

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN XUÂN LỘC
DƯƠNG HIỀN GIA KHANG

KHÓA LUẬN TỐT NGHIỆP
PHÁT TRIỂN TRÌNH MÔ PHỎNG HỆ THỐNG TRÊN
CHIP DỰA TRÊN KIẾN TRÚC TẬP LỆNH RISC-V
RV32IMF VÀ GIAO THỨC TILELINK

DEVELOPING A SYSTEM-ON-CHIP SIMULATOR BASED ON
RISC-V RV32IMF INSTRUCTION SET ARCHITECTURE AND
TILELINK PROTOCOL

CỦA NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

NGUYỄN XUÂN LỘC - 22520793
DƯƠNG HIỂN GIA KHANG - 22520610

KHÓA LUẬN TỐT NGHIỆP
PHÁT TRIỂN TRÌNH MÔ PHỎNG HỆ THỐNG TRÊN
CHIP DỰA TRÊN KIẾN TRÚC TẬP LỆNH RISC-V
RV32IMF VÀ GIAO THỨC TILELINK

DEVELOPING A SYSTEM-ON-CHIP SIMULATOR BASED ON
RISC-V RV32IMF INSTRUCTION SET ARCHITECTURE AND
TILELINK PROTOCOL

CỬ NHÂN NGÀNH KỸ THUẬT MÁY TÍNH

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN THÀNH NHÂN
KS. TRẦN ĐẠI DƯƠNG

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số
ngày của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Lời nói đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến quý nhà Trường - Trường Đại học Công nghệ Thông tin, Đại học Quốc gia Thành phố Hồ Chí Minh nói chung và Khoa Kỹ thuật Máy tính nói riêng. Sau gần 4 năm học tập, rèn luyện dưới môi trường học tập của Trường cũng như của Khoa, những điều em nhận lại là các kiến thức và kỹ năng quý giá, đó hành trang giúp em luôn tự tin và sẵn sàng cho những bước tiến tiếp theo trong tương lai.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến ThS. Nguyễn Thành Nhân và KS. Trần Đại Dương, hai giảng viên hướng dẫn đã tận tình định hướng, góp ý và hỗ trợ chúng em trong suốt quá trình thực hiện đề tài. Những nhận xét, chỉ dẫn và sự động viên của quý thầy đã giúp chúng em từng bước hoàn thiện hướng nghiên cứu, giải quyết các vấn đề phát sinh trong quá trình phát triển hệ thống, cũng như hoàn thiện nội dung khóa luận một cách nghiêm túc và khoa học.

Chúng em cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các anh chị, các bạn sinh viên cùng khóa đã luôn quan tâm, động viên và hỗ trợ chúng em trong quá trình học tập cũng như trong thời gian thực hiện khóa luận. Sự đồng hành và khích lệ đó là nguồn động lực quý báu giúp chúng em vượt qua những khó khăn, nỗ lực hoàn thành đề tài “Phát triển trình mô phỏng hệ thống trên chip dựa trên kiến trúc tập lệnh RISC-V RV32IMF và giao thức TileLink”.

Một lần nữa, chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Nguyễn Xuân Lộc

Dương Hiên Gia Khang

MỤC LỤC

Chương 1.	TỔNG QUAN ĐỀ TÀI.....	4
1.1.	Lý do thực hiện đề tài.....	4
1.1.1.	Bối cảnh và nhu cầu.....	4
1.1.2.	Khảo sát các công cụ mô phỏng hiện có.....	4
1.1.3.	Khoảng trống và hướng giải quyết	7
1.2.	Mục tiêu của đề tài	7
1.2.1.	Mục tiêu tổng quát	7
1.2.2.	Mục tiêu kỹ thuật	7
1.3.	Đối tượng và phạm vi nghiên cứu.....	8
1.3.1.	Đối tượng nghiên cứu	8
1.3.2.	Phạm vi và giới hạn	8
1.3.3.	Định hướng đánh giá.....	9
1.4.	Phương pháp thực hiện.....	10
1.5.	Đóng góp của đề tài.....	10
1.6.	Cấu trúc báo cáo	11
Chương 2.	CƠ SỞ LÝ THUYẾT.....	12
2.1.	Hệ thống trên chip (SoC)	12
2.1.1.	Khái niệm và đặc điểm	12
2.1.2.	Các thành phần điển hình và vai trò	12
2.2.	Các mức mô phỏng và vị trí của đề tài.....	14
2.3.	Kiến trúc tập lệnh RISC-V	15
2.3.1.	Tổng quan và triết lý mô-đun hóa ISA	15
2.3.2.	Tập lệnh cơ sở RV32I.....	17

2.3.3.	Mở rộng nhân/chia số nguyên RV32M	19
2.3.4.	Mở rộng dấu phẩy động RV32F và chuẩn IEEE 754	19
2.3.5.	Thao tác nguyên tử RV32A (mở rộng ngoài tên đề tài)	21
2.4.	Quản lý bộ nhớ và đơn vị quản lý bộ nhớ (MMU)	21
2.5.	Bộ nhớ đệm (cache).....	23
2.6.	Giao thức kết nối TileLink	25
2.6.1.	Vai trò interconnect và ba cấp độ TileLink	25
2.6.2.	Mô hình master/slave và các kênh truyền.....	26
2.6.3.	Tập con TileLink được sử dụng trong đề tài	29
2.7.	Truy cập bộ nhớ trực tiếp (DMA)	29
2.8.	Vào/ra ánh xạ bộ nhớ (MMIO) và thiết bị ngoại vi	31
2.9.	Công nghệ nền tảng web	32
Chương 3.	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	34
3.1.	Phân tích yêu cầu.....	34
3.1.1.	Yêu cầu chức năng	34
3.1.2.	Yêu cầu phi chức năng.....	35
3.1.3.	Mô hình use case tổng quát.....	36
3.2.	Kiến trúc tổng quan hệ thống	37
3.2.1.	Sơ đồ kiến trúc tổng quan	37
3.2.2.	Danh sách mô-đun và vai trò	39
3.2.3.	Bản đồ địa chỉ bộ nhớ	41
3.3.	Mô hình mô phỏng theo chu kỳ và trừu tượng kết nối.....	42
3.3.1.	Mô hình tick theo chu kỳ	42
3.3.2.	Trừu tượng cổng kết nối (Port).....	44

3.4.	Thiết kế giao diện web	46
3.4.1.	Bố cục và điều hướng	46
3.4.2.	Các khung quan sát trạng thái	48
3.5.	Luồng thực thi chương trình hợp ngữ	49
Chương 4.	HIỆN THỰC HỆ THỐNG	53
4.1.	Trình biên dịch hợp ngữ (Assembler)	53
4.1.1.	Kiến trúc hai lượt (two-pass)	53
4.1.2.	Chỉ thị và lệnh giả	56
4.1.3.	Mã hóa lệnh và ánh xạ thanh ghi	57
4.1.4.	Hỗ trợ soạn thảo	59
4.2.	Bộ xử lý CPU RV32IMF	60
4.2.1.	Tổ chức thanh ghi và trạng thái	60
4.2.2.	Chu trình nạp–giải mã–thực thi và truy cập bộ nhớ bất đồng bộ	62
4.2.3.	Hiện thực các nhóm lệnh I/M/F	64
4.2.4.	Xử lý dịch vụ hệ thống (syscall)	65
4.3.	Hệ thống bộ nhớ	66
4.3.1.	Bộ nhớ chính	66
4.3.2.	Đơn vị quản lý bộ nhớ và TLB	67
4.3.3.	Phân cấp bộ nhớ đệm L1I/L1D/L2	69
4.4.	Hệ thống bus TileLink	71
4.4.1.	Lỗi giao thức: mã thao tác, mặt nạ và ảnh chụp kênh	71
4.4.2.	Định tuyến master/slave và mô hình giao dịch	72
4.4.3.	Hai phân hệ bus TileLink-UH và TileLink-UL	74
4.4.4.	Cầu nối UH↔UL	75

4.5.	Bộ điều khiển DMA	76
4.6.	Hệ thống ngoại vi và MMIO	79
4.6.1.	Tổng quan kiến trúc ngoại vi	79
4.6.2.	UART	81
4.6.3.	Ma trận LED	82
4.6.4.	Bàn phím và chuột	83
4.7.	Trực quan hóa và gỡ lỗi.....	84
4.7.1.	Sơ đồ SoC động	84
4.7.2.	Nhật ký hệ thống và các khung quan sát trạng thái	85
4.8.	Vòng điều khiển thực thi trên giao diện	87
4.9.	Một số thách thức hiện thực và cách xử lý.....	90
Chương 5.	KIỂM THỬ VÀ ĐÁNH GIÁ	92
5.1.	Phương pháp và môi trường kiểm thử.....	92
5.2.	Kiểm thử đơn vị các mô-đun.....	92
5.3.	Đối chiếu mã hóa lệnh với GNU binutils.....	93
5.4.	Đối chiếu thực thi với Spike trên riscv-tests	94
5.5.	Nhận xét và hạn chế	94
Chương 6.	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	96
6.1.	Kết luận	96
6.2.	Hạn chế.....	97
6.3.	Hướng phát triển.....	97

DANH MỤC HÌNH

Hình 2.1: Sơ đồ khối của một hệ thống SoC — CPU	14
Hình 2.2: Sáu khuôn dạng lệnh RV32I (R/I/S/B/U/J) với bố trí các trường opcode, rd, funct3, rs1, rs2, funct7 và các trường giá trị tức thời tương ứng.	18
Hình 2.3: Cơ chế dịch địa chỉ ảo sang vật lý.	23
Hình 2.4: Tổ chức bộ nhớ đệm kết hợp tập.	25
Hình 2.5: Mô hình giao dịch TileLink trên hai kênh A và D.	28
Hình 2.6: So sánh luồng sao chép dữ liệu.	31
Hình 3.1: Sơ đồ use case tổng quát.	37
Hình 3.2: Kiến trúc tổng quan của trình mô phỏng SoC	38
Hình 3.3: Trình tự tiến bước trong một chu kỳ và sự lan truyền yêu cầu qua nhiều chu kỳ.	44
Hình 3.4: Mô hình cổng kết nối giữa các mô-đun.	46
Hình 3.5: Giao diện web tổng quan.	48
Hình 3.6: Các khung quan sát thanh ghi và bộ nhớ	49
Hình 3.7: Sơ đồ hoạt động luồng thực thi chương trình hợp ngữ (đầu–cuối)	52
Hình 4.1: Tóm tắt quy trình hai lượt của trình biên dịch.	55
Hình 4.2: Minh họa cơ chế gợi ý cú pháp trong trình soạn thảo.	60
Hình 4.3: Minh họa tổ chức bên trong của lõi xử lý.	61
Hình 4.4: Mô tả máy trạng thái truy cập bộ nhớ nói trên.	63
Hình 4.5: Minh họa cấu trúc MMU và đường dịch địa chỉ.	69
Hình 4.6: Minh họa phân cấp bộ nhớ đệm trong hệ thống.	71
Hình 4.7: Minh họa cấu trúc một giao dịch trên hai kênh A và D.	72
Hình 4.8: Minh họa cấu trúc bus và mô hình giao dịch.	74
Hình 4.9: Minh họa luồng giao dịch qua cầu nối.	76
Hình 4.10: Mô tả quy trình truyền của bộ điều khiển DMA.	79
Hình 4.11: Minh họa kiến trúc ngoại vi và các đường sự kiện từ giao diện.	81
Hình 4.12: Minh họa ánh xạ giữa VRAM và điểm ảnh.	83
Hình 4.13: Minh họa khung nhìn sơ đồ SoC khi có giao dịch trên bus.	85

Hình 4.14: Minh họa terminal nhật ký với bộ lọc theo mô-đun.

87

Hình 4.15: Minh họa vòng lặp chạy mô phỏng trên giao diện.

89

DANH MỤC BẢNG

Bảng 1.1. So sánh một số công cụ mô phỏng liên quan	6
Bảng 1.2. Tóm tắt phạm vi chức năng của hệ thống	9
Bảng 2.1. Các phần mở rộng RISC-V và phạm vi trong đề tài	16
Bảng 2.2. Các nhóm lệnh của tập cơ sở RV32I	18
Bảng 2.3. Các nhóm lệnh RV32M và RV32F được hỗ trợ	21
Bảng 2.4. So sánh ba cấp độ của giao thức TileLink	26
Bảng 2.5. Mã thao tác (opcode) trên kênh A và kênh D của TileLink	28
Bảng 3.1. Danh sách yêu cầu chức năng của hệ thống	34
Bảng 3.2. Danh sách mô-đun của hệ thống và vai trò mô phỏng	39
Bảng 3.3. Bản đồ địa chỉ bộ nhớ của hệ thống	42
Bảng 3.4. Bầy khung nhìn của giao diện và nội dung chính	47
Bảng 4.1. Các chỉ thị được hỗ trợ	56
Bảng 4.2. Lệnh giả và quy tắc khai triển	57
Bảng 4.3. Ánh xạ tên thanh ghi ABI	58
Bảng 4.4. Các dịch vụ hệ thống được hỗ trợ (`a7` chọn dịch vụ)	65
Bảng 4.5. Cấu hình đơn vị quản lý bộ nhớ và TLB	67
Bảng 4.6. Cấu hình bộ nhớ đệm L1I/L1D/L2	69
Bảng 4.7. Mã thao tác kênh A được phép trên TileLink-UH và TileLink-UL	74
Bảng 4.8. Bản đồ thanh ghi DMA	77
Bảng 4.9. Danh sách thiết bị ngoại vi và vùng địa chỉ	80
Bảng 4.10. Bản đồ thanh ghi UART	81
Bảng 4.11. Bản đồ thanh ghi bàn phím và chuột	83
Bảng 4.12. Các nhóm nhật ký và tiêu chí phân loại	86
Bảng 5.1. Ví dụ mã hóa lệnh do đề tài sinh ra, đối chiếu với GNU binutils	93

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên viết đầy đủ	Mô tả ý nghĩa
ABI	Application Binary Interface	Giao diện nhị phân ứng dụng
ALU	Arithmetic Logic Unit	Đơn vị số học – logic
ASCII	American Standard Code for Information Interchange	Bộ mã ký tự chuẩn ASCII
CAN	Controller Area Network	Mạng điều khiển vùng (bus CAN)
CISC	Complex Instruction Set Computer	Máy tính tập lệnh phức tạp
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSR	Control and Status Register	Thanh ghi điều khiển và trạng thái
CSS	Cascading Style Sheets	Bảng định kiểu xếp tầng
DMA	Direct Memory Access	Truy cập bộ nhớ trực tiếp
DOM	Document Object Model	Mô hình đối tượng tài liệu
ELF	Executable and Linkable Format	Định dạng tập tin thực thi và liên kết
FCSR	Floating-Point Control and Status Register	Thanh ghi điều khiển và trạng thái dấu phẩy động

Từ viết tắt	Tên viết đầy đủ	Mô tả ý nghĩa
FIFO	First In, First Out	Vào trước, ra trước
FPU	Floating-Point Unit	Đơn vị dấu phẩy động
GNU	GNU's Not Unix	Bộ công cụ phần mềm tự do GNU
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
IEEE	Institute of Electrical and Electronics Engineers	Viện Kỹ sư Điện và Điện tử
IRQ	Interrupt Request	Yêu cầu ngắt
ISA	Instruction Set Architecture	Kiến trúc tập lệnh
L1D	Level 1 Data Cache	Bộ nhớ đệm dữ liệu cấp 1
L1I	Level 1 Instruction Cache	Bộ nhớ đệm lệnh cấp 1
L2	Level 2 Cache	Bộ nhớ đệm cấp 2
LED	Light-Emitting Diode	Điốt phát quang
LRU	Least Recently Used	Thay thế phần tử ít dùng gần đây nhất
MIPS	Microprocessor without Interlocked Pipeline Stages	Kiến trúc tập lệnh MIPS
MMIO	Memory-Mapped I/O	Vào/ra ánh xạ bộ nhớ

Từ viết tắt	Tên viết đầy đủ	Mô tả ý nghĩa
MMU	Memory Management Unit	Đơn vị quản lý bộ nhớ
PC	Program Counter	Bộ đếm chương trình
PPN	Physical Page Number	Số hiệu trang vật lý
PTE	Page Table Entry	Mục bảng trang
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
RARS	RISC-V Assembler and Runtime Simulator	Trình hợp dịch và mô phỏng thực thi RISC-V
RISC	Reduced Instruction Set Computer	Máy tính tập lệnh thu gọn
RISC-V	Reduced Instruction Set Computer – Five	Kiến trúc tập lệnh mở RISC-V
RTL	Register-Transfer Level	Mức truyền thanh ghi
RV32A	RISC-V 32-bit Atomic Extension	Mở rộng thao tác nguyên tử 32-bit của RISC-V
RV32F	RISC-V 32-bit Single-Precision Floating-Point Extension	Mở rộng dấu phẩy động đơn 32-bit của RISC-V
RV32I	RISC-V 32-bit Base Integer Instruction Set	Tập lệnh số nguyên cơ sở 32-bit của RISC-V
RV32IMF	RISC-V 32-bit Integer, Multiply/Divide and Floating-Point	Tổ hợp tập lệnh RV32I + M + F

Từ viết tắt	Tên viết đầy đủ	Mô tả ý nghĩa
RV32M	RISC-V 32-bit Integer Multiplication and Division Extension	Mở rộng nhân/chia số nguyên 32-bit của RISC-V
SoC	System-on-Chip	Hệ thống trên chip
SVG	Scalable Vector Graphics	Đồ họa vector co giãn
TLB	Translation Lookaside Buffer	Bộ đệm dịch địa chỉ
TileLink	TileLink Interconnect Protocol	Giao thức kết nối TileLink
TileLink-UH	TileLink Uncached Heavyweight	Phân hệ TileLink không lưu đệm, đầy đủ
TileLink-UL	TileLink Uncached Lightweight	Phân hệ TileLink không lưu đệm, rút gọn
UART	Universal Asynchronous Receiver–Transmitter	Bộ thu–phát bất đồng bộ đa năng
VPN	Virtual Page Number	Số hiệu trang ảo
VRAM	Video Random Access Memory	Bộ nhớ truy cập ngẫu nhiên đồ họa

TÓM TẮT KHÓA LUẬN

Kiến trúc máy tính và hệ thống trên chip (System-on-Chip) là những nội dung nền tảng nhưng tương đối trừu tượng trong đào tạo ngành Kỹ thuật Máy tính. Một hệ thống tính toán thực tế không chỉ gồm một lõi xử lý mà là sự phối hợp của nhiều thành phần: lõi xử lý, phân cấp bộ nhớ, đơn vị quản lý bộ nhớ, hệ thống bus kết nối, bộ điều khiển truy cập bộ nhớ trực tiếp (Direct Memory Access – DMA) và các thiết bị ngoại vi. Các cơ chế bên trong như dịch địa chỉ ảo sang vật lý, hoạt động của bộ nhớ đệm, các giao dịch trên bus hay luồng dữ liệu của DMA khó quan sát trực tiếp và cũng không hiển thị rõ nếu chỉ chạy chương trình ở mức tập lệnh. Khảo sát các công cụ hiện có cho thấy một khoảng trống: các công cụ chạy trên web như RISC-V Interpreter hay WebRISC-V thuận tiện về khả năng tiếp cận nhưng dừng ở mức lõi xử lý; Ripes mạnh về trực quan hóa vi kiến trúc nhưng hạn chế về tùy biến hệ thống bus và DMA; Spike chính xác nhưng chỉ chạy ở dòng lệnh, không trực quan; và khóa luận liên quan thực hiện trước đó tuy đã tiếp cận hướng mô phỏng SoC cho RISC-V nhưng còn giới hạn ở tập lệnh cơ sở RV32I cùng thao tác nguyên tử và số lượng thiết bị ngoại vi còn ít.

Trên cơ sở đó, khóa luận đặt mục tiêu xây dựng một trình mô phỏng SoC dựa trên kiến trúc tập lệnh RISC-V RV32IMF và giao thức kết nối TileLink, hoạt động hoàn toàn trên trình duyệt web nhằm phục vụ mục tiêu giảng dạy và tự học. Hướng tiếp cận là mô phỏng ở mức hành vi/chức năng theo chu kỳ (behavioral/functional), trong đó mỗi thành phần phần cứng được mô hình hóa thành một mô-đun phần mềm và được lắp ghép tại một thành phần khởi tạo trung tâm; hệ thống không mô phỏng các đặc tính vật lý như độ trễ định thời theo công nghệ chế tạo hay mức tiêu thụ năng lượng. Phù hợp với định hướng trung thực của toàn khóa luận, hệ thống hỗ trợ một tập con của RV32IMF phục vụ mô phỏng cho các chương trình ở chế độ người dùng, còn các thành phần như lệnh truy cập thanh ghi điều khiển/trạng thái (CSR) và chế độ đặc quyền, thanh ghi điều khiển dấu phẩy động (FCSR/fflags/frm) và việc thực thi lệnh fence được nêu rõ là nằm ngoài phạm vi hiện tại.

Kết quả chính của khóa luận là một hệ thống tích hợp gồm nhiều thành phần phối hợp với nhau. Trình biên dịch hợp ngữ được hiện thực theo cơ chế hai lượt (two-pass) với đầy đủ chỉ thị, lệnh giả và mã hóa lệnh cho ba nhóm RV32I, RV32M và RV32F. Lỗi xử lý mô phỏng tập con RV32IMF với tệp thanh ghi số nguyên và tệp thanh ghi dấu phẩy động, thực hiện chu trình nạp – giải mã – thực thi và truy cập bộ nhớ theo mô hình bất đồng bộ. Hệ thống bộ nhớ gồm đơn vị quản lý bộ nhớ với phân trang và bộ đệm dịch địa chỉ (TLB), cùng phân cấp bộ nhớ đệm L1I/L1D/L2. Hệ thống bus được tổ chức thành hai phân hệ TileLink-UH và TileLink-UL kèm cầu nối, mô phỏng ở mức giao dịch trên hai kênh yêu cầu (A) và phản hồi (D); cả bộ xử lý và bộ điều khiển DMA cùng đóng vai trò thành phần chủ động phát yêu cầu, với cơ chế điều phối ở mức hàng đợi xử lý một giao dịch tại một thời điểm. Bộ điều khiển DMA minh họa khả năng vận chuyển khối dữ liệu độc lập với luồng thực thi của CPU, với việc nhận biết hoàn tất bằng cơ chế đọc thăm dò (polling). Hệ thống tích hợp bốn thiết bị ngoại vi gồm UART, ma trận LED, bàn phím và chuột, cùng các khung trực quan hóa: sơ đồ SoC động làm nổi bật giao dịch, khung quan sát MMU và bộ nhớ đệm, khung quan sát thanh ghi và bộ nhớ, và nhật ký hệ thống có khả năng lọc theo từng mô-đun.

Về kiểm chứng, khóa luận tổ chức nhiều lớp kiểm thử: kiểm thử đơn vị bằng Node.js cho các mô-đun trọng tâm đã đạt kết quả ở các kịch bản cho trình biên dịch hợp ngữ, MMU, TileLink và DMA; bên cạnh đó là quy trình đối chiếu mã hóa lệnh với bộ công cụ GNU RISC-V binutils và đối chiếu hành vi thực thi với trình mô phỏng tham chiếu Spike trên bộ riscv-tests của cộng đồng RISC-V.

Các đóng góp chính của khóa luận gồm: (1) một trình mô phỏng SoC dựa trên RISC-V RV32IMF chạy trên nền web, tích hợp trong cùng một công cụ trình biên dịch hợp ngữ, lỗi xử lý, phân cấp bộ nhớ (cache và MMU), hệ thống bus TileLink, bộ điều khiển DMA và nhiều thiết bị ngoại vi cùng các khung trực quan hóa trạng thái hệ thống; (2) mở rộng phạm vi tập lệnh so với khóa luận liên quan trước đó bằng cách bổ sung nhóm lệnh nhân/chia số nguyên (RV32M) và nhóm lệnh dấu phẩy động đơn (RV32F) cùng tệp thanh ghi dấu phẩy động; (3) bổ sung số lượng và chủng loại

thiết bị ngoại vi tương tác cùng các khung quan sát hỗ trợ như sơ đồ SoC động, khung quan sát MMU/bộ nhớ đệm và nhật ký hệ thống lọc theo mô-đun; (4) một quy trình kiểm chứng đối chiếu với các công cụ được cộng đồng công nhận (GNU binutils và Spike) trên bộ riscv-tests, kèm bộ kiểm thử nội bộ cho các mô-đun. Những đóng góp này hướng đến một công cụ phục vụ giảng dạy và học tập kiến trúc máy tính ở mức hệ thống thay vì chỉ ở mức tập lệnh. Các hạn chế còn lại và đề xuất hướng phát triển được trình bày trong Chương 6.

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Lý do thực hiện đề tài

1.1.1. Bối cảnh và nhu cầu

Kiến trúc máy tính, hệ thống nhúng và hệ thống trên chip (System-on-Chip – SoC) là những nội dung nền tảng trong chương trình đào tạo ngành Kỹ thuật Máy tính. Một hệ thống tính toán thực tế không chỉ gồm một lõi xử lý đơn lẻ mà là sự phối hợp của nhiều thành phần: lõi xử lý, phân cấp bộ nhớ, đơn vị quản lý bộ nhớ, hệ thống bus kết nối (interconnect), bộ điều khiển truy cập bộ nhớ trực tiếp (Direct Memory Access – DMA) và các thiết bị ngoại vi. Tuy nhiên, cách các thành phần này phối hợp ở mức hệ thống – như dịch địa chỉ ảo sang vật lý, hoạt động của bộ nhớ đệm, các giao dịch trên bus hay luồng dữ liệu của DMA – tương đối trừu tượng, khó quan sát trực tiếp trên phần cứng và không hiển thị rõ nếu chỉ chạy chương trình ở mức tập lệnh.

Kiến trúc tập lệnh (Instruction Set Architecture – ISA) RISC-V được quan tâm rộng rãi trong giảng dạy và nghiên cứu nhờ tính mở, miễn phí bản quyền và thiết kế mô-đun: một tập lệnh cơ sở nhỏ gọn (RV32I) cùng các phần mở rộng tùy chọn như nhân/chia số nguyên (M), dấu phẩy động (F) và thao tác nguyên tử (A) [1]. Hệ sinh thái phần cứng mở quanh RISC-V cũng dùng các giao thức kết nối mở, tiêu biểu là TileLink – một giao thức bus công khai được dùng trong nhiều thiết kế SoC dựa trên RISC-V [2]. Trong bối cảnh đó, một công cụ mô phỏng trực quan hóa trạng thái hệ thống theo từng bước và chạy trực tiếp trên trình duyệt web mà không cần cài đặt là phương tiện hỗ trợ hiệu quả cho việc giảng dạy và tự học kiến trúc máy tính ở mức hệ thống [3].

1.1.2. Khảo sát các công cụ mô phỏng hiện có

Để xác định khoảng trống mà đề tài hướng đến, phần này khảo sát một số công cụ mô phỏng phổ biến cùng một khóa luận liên quan thực hiện trước đó tại đơn vị.

MARS (MIPS Assembler and Runtime Simulator) là công cụ mô phỏng hợp ngữ và thực thi cho kiến trúc MIPS, được viết bằng Java và có giao diện đồ họa tích hợp

trình soạn thảo, bảng thanh ghi và khung quan sát bộ nhớ [3]. MARS hữu ích cho việc học tập ở mức tập lệnh, tuy nhiên nó hướng đến kiến trúc MIPS thay vì RISC-V, yêu cầu môi trường chạy Java, và tập trung ở mức mô phỏng tập lệnh của một bộ xử lý đơn lẻ; các thành phần ở mức hệ thống như bus kết nối và DMA không nằm trong trọng tâm của công cụ này [1].

RISC-V Interpreter (Đại học Cornell) là một công cụ chạy trên web, đơn giản và dễ sử dụng, cho phép biên dịch và quan sát giá trị thanh ghi cơ bản [2]. Tuy nhiên công cụ này không tập trung mô phỏng đường đi dữ liệu (datapath) và không mô phỏng các thiết bị ngoại vi, do đó phù hợp cho việc làm quen với hợp ngữ RISC-V hơn là quan sát hoạt động ở mức hệ thống.

WebRISC-V là công cụ mô phỏng trên web tập trung vào việc mô phỏng đường đi dữ liệu theo cơ chế nhiều chu kỳ và pipeline của bộ xử lý RISC-V [3]. Trọng tâm của công cụ là hoạt động bên trong lõi xử lý; phạm vi của nó chưa mở rộng đến kiến trúc SoC và chưa hỗ trợ tương tác với các thiết bị ngoại vi.

Ripes là một công cụ mã nguồn mở nổi bật trong việc trực quan hóa đường đi dữ liệu và pipeline của bộ xử lý, đồng thời có hỗ trợ minh họa bộ nhớ đệm [4]. Ripes tập trung khá sâu vào vi kiến trúc của CPU; theo định hướng so sánh trong đề cương đề tài, mức độ tùy biến hệ thống bus và tích hợp bộ điều khiển DMA phục vụ mục đích mô phỏng SoC ở công cụ này còn hạn chế.

Spike là trình mô phỏng tham chiếu chuẩn của RISC-V International, có độ chính xác cao và thường được dùng làm mô hình tham chiếu để kiểm chứng hành vi thực thi [5]. Tuy nhiên Spike hoạt động qua giao diện dòng lệnh và không cung cấp khả năng trực quan hóa trạng thái hệ thống, nên ít thuận lợi cho việc hình dung hoạt động của hệ thống đối với người học.

Khóa luận liên quan thực hiện trước đó. Khóa luận "Phát triển trình mô phỏng SoC" của tác giả Nguyễn Gia Bảo Ngọc (2025) [8] là công trình liên quan trực tiếp nhất, cùng hướng xây dựng trình mô phỏng SoC cho RISC-V với đề tài này. Theo phạm vi được trình bày trong khóa luận đó, hệ thống bao gồm lõi xử lý RISC-V ở phạm vi tập

lệnh cơ sở RV32I cùng các thao tác nguyên tử trên bộ nhớ, đơn vị quản lý bộ nhớ (MMU), giao thức kết nối TileLink, bộ điều khiển DMA và thiết bị ngoại vi dạng ma trận LED [8]. Phạm vi tập lệnh của khóa luận này tập trung ở RV32I và thao tác nguyên tử, chưa mở rộng sang nhóm lệnh nhân/chia số nguyên (M) và nhóm lệnh dấu phẩy động (F); số lượng thiết bị ngoại vi cũng còn giới hạn. Đây chính là những điểm mà đề tài hiện tại kế thừa và mở rộng.

Bảng 1.1 so sánh các công cụ trên theo một số tiêu chí ở mức hệ thống, với các ký hiệu: "Có" – hỗ trợ rõ ràng; "Hạn chế" – có nhưng ở mức giới hạn hoặc không phải trọng tâm; "Không" – không thuộc phạm vi công cụ.

Bảng 1.1. So sánh một số công cụ mô phỏng liên quan

Tiêu chí	MARS	RISC-V Interpreter	WebRISC-V	Nguyễn Gia bảo Ngọc	Đề tài này
Nền tảng	Java	Web	Web	Web	Web
Kiến trúc tập lệnh	MIPS	RISC-V	RISC-V	RV32I	RV32IMF
Trình biên dịch hợp ngữ tích hợp	Có	Có	Hạn chế	Có	Có
Trực quan hóa thực thi CPU	Hạn chế	Hạn chế	Có (pipeline)	Có	Có
Phân cấp bộ nhớ	Hạn chế	Không	Không	Có (MMU)	Có (Cache + MMU)
Bus	Không	Không	Không	Có (TileLink)	Có (TileLink UH/UL)
I/O	Hạn chế	Không	Không	Hạn chế (LED)	Có (UART, LED, bàn phím, chuột)
DMA	Không	Không	Không	Có	Có
Mô phỏng ở mức SoC	Không	Không	Không	Có	Có

Bảng 1.1 cho thấy các công cụ web hiện có dừng ở mức lỗi xử lý, còn các công cụ mạnh về vi kiến trúc hoặc có độ chính xác tham chiếu cao lại không nhắm tới mô phỏng SoC đầy đủ trên nền web. Cột "Đề tài này" tương ứng với các thành phần đã hiện thực trong mã nguồn (chi tiết ở Chương 3 và Chương 4).

1.1.3. Khoảng trống và hướng giải quyết

Khảo sát trên cho thấy còn thiếu một công cụ web hội đủ đồng thời: mô phỏng ở mức hệ thống SoC với đầy đủ lỗi xử lý, phân cấp bộ nhớ, bus kết nối, DMA và nhiều loại ngoại vi; hỗ trợ phạm vi tập lệnh RV32IMF; tích hợp trình biên dịch hợp ngữ và trực quan hóa từng thành phần; đồng thời được kiểm chứng đối chiếu với các công cụ tham chiếu được cộng đồng công nhận. Đề tài hướng đến lấp khoảng trống đó: phát triển một trình mô phỏng SoC dựa trên RISC-V RV32IMF và giao thức TileLink chạy hoàn toàn trên trình duyệt web, tích hợp trình biên dịch hợp ngữ, phân cấp bộ nhớ (cache và MMU), bus TileLink với bộ điều khiển DMA và các thiết bị ngoại vi tương tác, cùng các khung quan sát trực quan cho từng thành phần.

1.2. Mục tiêu của đề tài

1.2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một bộ công cụ mô phỏng hành vi của một hệ thống trên chip điển hình dựa trên kiến trúc RISC-V, kèm dịch vụ quản lý bộ nhớ, phục vụ mục tiêu giáo dục. Công cụ cho phép người dùng biên dịch mã nguồn hợp ngữ, mô phỏng hoạt động của các thành phần phần cứng và quan sát trạng thái của chúng trên nền tảng web dễ tiếp cận.

1.2.2. Mục tiêu kỹ thuật

Trên cơ sở mục tiêu tổng quát, đề tài đặt ra các mục tiêu kỹ thuật cụ thể, bám theo định hướng trong đề cương và phù hợp với hiện trạng đã hiện thực:

- Mô phỏng lỗi xử lý RISC-V 32-bit hỗ trợ các nhóm lệnh RV32IMF (số nguyên cơ sở I, nhân/chia M, dấu phẩy động đơn F), với tệp thanh ghi số nguyên và dấu phẩy động cùng chu trình nạp – giải mã – thực thi lệnh.

- Tái hiện hệ thống bus kết nối dựa trên giao thức TileLink, trong đó cả bộ xử lý và bộ điều khiển DMA đều đóng vai trò master phát yêu cầu trên bus (tách thành hai phân hệ TileLink-UH và TileLink-UL kèm cầu nối); đồng thời minh họa khả năng giảm tải cho bộ xử lý khi DMA vận chuyển khối dữ liệu độc lập với luồng thực thi lệnh.
- Hiện thực dịch vụ quản lý bộ nhớ gồm MMU với cơ chế phân trang và bộ đệm dịch địa chỉ (Translation Lookaside Buffer – TLB), cùng phân cấp bộ nhớ đệm nhiều mức.
- Tích hợp trình biên dịch hợp ngữ và hệ thống thiết bị ngoại vi mô phỏng cùng các khung trực quan hóa, cho phép người dùng nạp mã nguồn và quan sát kết quả điều khiển phần cứng.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

- Kiến trúc tập lệnh RISC-V (tập cơ sở RV32I cùng các phần mở rộng nhân/chia số nguyên M và dấu phẩy động đơn F) [1] và giao thức kết nối TileLink ở hai cấp độ TileLink-UL (Uncached Lightweight) và TileLink-UH (Uncached Heavyweight) [2].
- Kiến trúc hệ thống trên chip và cách tổ chức các thành phần: lõi xử lý, phân cấp bộ nhớ, bus kết nối, DMA và thiết bị ngoại vi.
- Các cơ chế quản lý bộ nhớ (phân trang, dịch địa chỉ ảo sang vật lý, TLB, cache) cùng các kỹ thuật xây dựng và trực quan hóa trình mô phỏng trên nền tảng web.

1.3.2. Phạm vi và giới hạn

Hệ thống mô phỏng ở mức hành vi/chức năng (behavioral/functional), tập trung tái hiện đúng hành vi logic của các thành phần; hệ thống không mô phỏng các đặc tính vật lý như độ trễ định thời chi tiết theo công nghệ chế tạo hay mức tiêu thụ năng lượng. Do được hiện thực bằng JavaScript chạy đơn luồng trên trình duyệt, tốc độ mô phỏng phụ thuộc vào máy của người dùng và không phản ánh tần số hoạt động của một chip thực tế.

Hệ thống hỗ trợ một tập con của RV32IMF phục vụ các chương trình ở chế độ người dùng; phần dấu phẩy động tính theo chuẩn IEEE 754 [10]. Một số thành phần của kiến trúc đầy đủ nằm ngoài phạm vi: lệnh truy cập thanh ghi điều khiển và trạng thái (Control and Status Register – CSR) cùng chế độ đặc quyền; thanh ghi điều khiển dấu phẩy động (FCSR), cờ ngoại lệ (fflags) và chế độ làm tròn động (frm); lệnh fence được mã hóa nhưng chưa thực thi; lệnh AMOADD.W thuộc phần mở rộng RV32A chỉ là mở rộng minh họa, nằm ngoài tên đề tài.

Về giao thức TileLink, hệ thống mô phỏng ở mức giao dịch trên hai kênh yêu cầu (A) và phản hồi (D) tương ứng TileLink-UL và TileLink-UH; chưa hiện thực các kênh B, C, E cùng cơ chế nhất quán bộ nhớ đệm (coherence) của TileLink-C. Cơ chế điều phối bus ở mức hàng đợi một giao dịch và chưa có đường tín hiệu ngắt (Interrupt Request – IRQ) tới bộ xử lý; trạng thái hoàn tất của DMA và ngoại vi được nhận biết qua cơ chế đọc thăm dò (polling). Các ngoại vi đã hiện thực gồm UART (Universal Asynchronous Receiver–Transmitter), ma trận LED, bàn phím và chuột; thiết bị CAN chỉ được đề ngỏ cho hướng phát triển. Bảng 1.2 tóm tắt phạm vi chức năng của hệ thống theo ba mức.

Bảng 1.2. Tóm tắt phạm vi chức năng của hệ thống

Mức độ	Nội dung
Đã hỗ trợ	Trình biên dịch hợp ngữ hai lượt; lỗi xử lý với các nhóm lệnh RV32I, RV32M, RV32F; tệp thanh ghi số nguyên và dấu phẩy động; MMU với phân trang và TLB; phân cấp bộ nhớ đệm; bus TileLink-UH/UL và cầu nối; DMA; ngoại vi UART, LED, bàn phím, chuột; trực quan hóa sơ đồ SoC, bộ nhớ, thanh ghi và nhật ký hệ thống
Hỗ trợ một phần	Dấu phẩy động đơn theo biểu diễn IEEE 754 nhưng chưa đủ FCSR/fflags/frm; lệnh fence được mã hóa nhưng chưa thực thi; thao tác nguyên tử mới có AMOADD.W; điều phối bus ở mức hàng đợi một giao dịch

Mức độ	Nội dung
Ngoài phạm vi	Lệnh CSR và chế độ đặc quyền; các kênh B/C/E và nhất quán bộ nhớ đệm của TileLink-C; cơ chế ngắt phần cứng tới CPU; thiết bị CAN; mô phỏng đặc tính định thời/năng lượng vật lý

1.3.3. Định hướng đánh giá

Tính đúng đắn của hệ thống được đánh giá bằng cách đối chiếu kết quả với các công cụ tham chiếu được cộng đồng công nhận, kết hợp với bộ kiểm thử nội bộ: đối chiếu kết quả mã hóa lệnh với bộ công cụ GNU RISC-V binutils và đối chiếu hành vi thực thi với trình mô phỏng tham chiếu Spike [8] trên bộ riscv-tests. Quy trình và kết quả chi tiết được trình bày trong Chương 5.

1.4. Phương pháp thực hiện

Nghiên cứu kiến trúc tập lệnh RISC-V RV32IMF [1], cấu trúc hệ thống trên chip, các cơ chế quản lý bộ nhớ (phân trang, TLB, cache) và giao thức TileLink ở hai cấp độ TL-UL và TL-UH [2]; đồng thời tìm hiểu cách triển khai của một số công cụ đã công bố [4]–[8] để rút ra yêu cầu thiết kế.

Hiện thực bộ mô phỏng SoC trên nền web bằng HTML, CSS và JavaScript theo mô hình mô-đun (ES modules), trong đó mỗi thành phần phần cứng là một mô-đun phần mềm riêng, được lắp ghép tại một thành phần khởi tạo trung tâm; giao diện dùng thư viện CodeMirror cho vùng nhập hợp ngữ và phần tử canvas để hiển thị ma trận LED.

Đối chiếu kết quả thực thi (trạng thái thanh ghi, bộ nhớ và kết quả mã hóa lệnh) với các công cụ mô phỏng tham chiếu thông qua các bộ chương trình kiểm thử, đồng thời xây dựng các kịch bản và chương trình minh họa chạy trực tiếp trên hệ thống.

1.5. Đóng góp của đề tài

Các đóng góp chính của đề tài gồm:

- Một trình mô phỏng SoC dựa trên RISC-V RV32IMF hoạt động trên nền web, tích hợp trong cùng một công cụ: trình biên dịch hợp ngữ, lõi xử lý, phân cấp bộ nhớ (cache và MMU), hệ thống bus TileLink, bộ điều khiển DMA và nhiều thiết bị ngoại vi, cùng các khung trực quan hóa trạng thái hệ thống.
- Mở rộng phạm vi tập lệnh so với khóa luận liên quan trước đó [9]: bổ sung nhóm lệnh nhân/chia số nguyên (RV32M) và nhóm lệnh dấu phẩy động đơn (RV32F) cùng tệp thanh ghi dấu phẩy động.
- Bổ sung số lượng và chủng loại thiết bị ngoại vi tương tác (UART có cấu hình tốc độ, ma trận LED, bàn phím, chuột) cùng các khung quan sát hỗ trợ: sơ đồ SoC động, khung quan sát MMU và bộ nhớ đệm, và nhật ký hệ thống có khả năng lọc theo từng mô-đun.
- Một quy trình kiểm chứng đối chiếu với các công cụ được cộng đồng công nhận (GNU binutils và Spike) trên bộ riscv-tests, kèm bộ kiểm thử nội bộ cho các mô-đun. Những đóng góp này hướng đến một công cụ phục vụ giảng dạy và học tập kiến trúc máy tính ở mức hệ thống, thay vì chỉ ở mức tập lệnh.

1.6. Cấu trúc báo cáo

Phần còn lại của báo cáo gồm năm chương. Chương 2 trình bày cơ sở lý thuyết: hệ thống trên chip, kiến trúc tập lệnh RISC-V (RV32I, RV32M, RV32F), quản lý bộ nhớ và bộ nhớ đệm, giao thức TileLink, cơ chế DMA, thiết bị ngoại vi và các công nghệ nền tảng web. Chương 3 phân tích yêu cầu và thiết kế kiến trúc tổng quan, danh sách mô-đun, bản đồ địa chỉ bộ nhớ, mô hình mô phỏng theo chu kỳ và giao diện web. Chương 4 mô tả chi tiết hiện thực của trình biên dịch hợp ngữ, lõi xử lý, hệ thống bộ nhớ, bus TileLink, DMA, thiết bị ngoại vi và trực quan hóa. Chương 5 trình bày phương pháp, môi trường và kết quả kiểm thử (đối chiếu với GNU binutils và Spike trên bộ riscv-tests). Chương 6 tổng kết kết quả, hạn chế và hướng phát triển.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Hệ thống trên chip (SoC)

2.1.1. Khái niệm và đặc điểm

Hệ thống trên chip (System-on-Chip – SoC) là một mạch tích hợp tập hợp phần lớn hoặc toàn bộ các khối chức năng của một hệ thống tính toán vào trong một vi mạch duy nhất [11]. Khác với mô hình máy tính cổ điển, nơi bộ xử lý, bộ nhớ và các bộ điều khiển vào/ra là những vi mạch riêng biệt nối với nhau qua bo mạch chủ, một SoC tích hợp các khối này cùng hệ thống kết nối nội bộ trên cùng một đế bán dẫn. Cách tổ chức này giúp giảm kích thước, giảm tiêu thụ năng lượng và rút ngắn đường truyền tín hiệu giữa các khối, do đó được sử dụng rộng rãi trong các thiết bị nhúng, thiết bị di động và nhiều hệ thống chuyên dụng.

Một đặc điểm quan trọng của SoC, xét từ góc độ kiến trúc, là sự hiện diện đồng thời của nhiều thành phần hoạt động tương đối độc lập nhưng phải phối hợp trao đổi dữ liệu qua một hạ tầng kết nối chung. Bộ xử lý không còn là thành phần duy nhất khởi tạo truy cập bộ nhớ; các thành phần khác như bộ điều khiển truy cập bộ nhớ trực tiếp cũng có thể chủ động phát yêu cầu trên hệ thống kết nối. Vì vậy, việc hiểu một SoC không chỉ dừng lại ở việc hiểu tập lệnh của bộ xử lý, mà còn đòi hỏi nắm được cách các thành phần chia sẻ không gian địa chỉ, cách hệ thống bus điều phối các yêu cầu và cách dữ liệu di chuyển giữa bộ nhớ và thiết bị ngoại vi.

2.1.2. Các thành phần điển hình và vai trò

Một SoC điển hình bao gồm các nhóm thành phần sau, được minh họa khái quát ở Hình 2.1.

Bộ xử lý trung tâm (Central Processing Unit – CPU). CPU là thành phần thực thi chương trình, thực hiện chu trình nạp lệnh, giải mã và thực thi (fetch – decode – execute). Bên trong CPU có tập thanh ghi để lưu trạng thái tính toán và đơn vị số học – luận lý (Arithmetic Logic Unit – ALU) để thực hiện các phép tính số nguyên; nếu hỗ trợ số thực, CPU còn có đơn vị dấu phẩy động (Floating-Point Unit – FPU) cùng

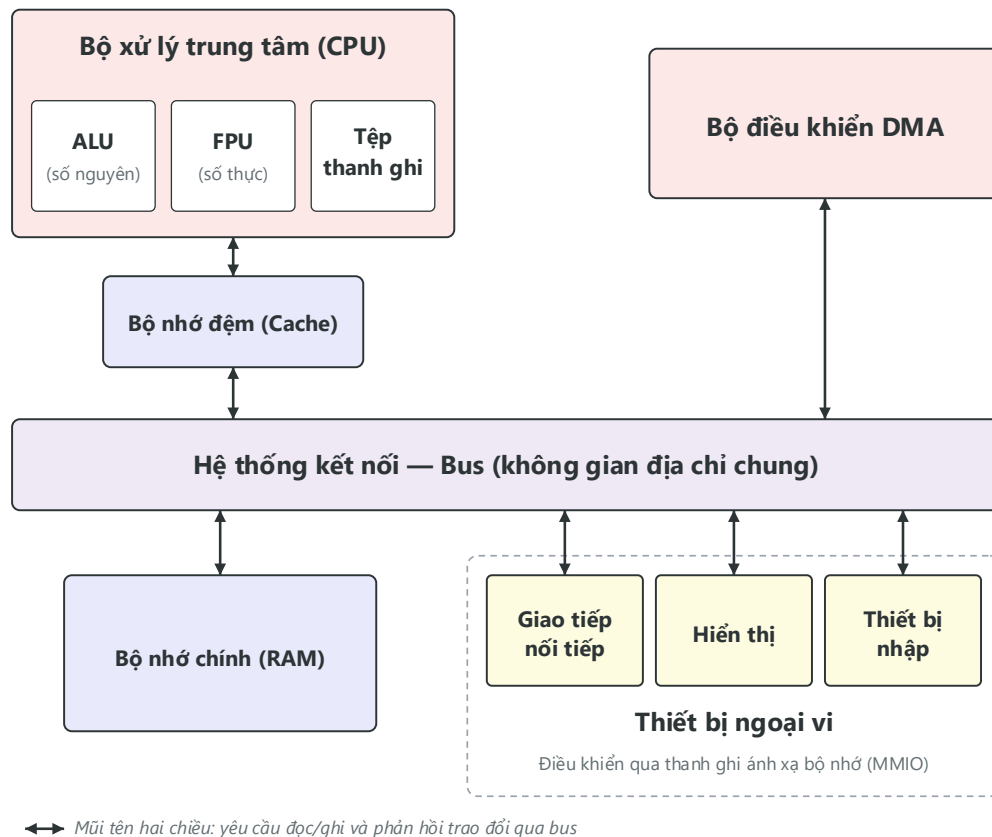
tệp thanh ghi dấu phẩy động riêng. CPU là thành phần chủ động (master) khởi tạo phần lớn các truy cập bộ nhớ trong hệ thống.

Bộ nhớ (memory). Bộ nhớ lưu trữ mã lệnh và dữ liệu. Trong thực tế, bộ nhớ được tổ chức thành nhiều cấp với đặc tính dung lượng – tốc độ khác nhau, từ các thanh ghi và bộ nhớ đệm tốc độ cao nằm gần CPU đến bộ nhớ chính (RAM) có dung lượng lớn nhưng độ trễ truy cập cao hơn [11], [12]. Tổ chức phân cấp này được trình bày chi tiết hơn ở các mục 2.4 và 2.5.

Hệ thống kết nối (interconnect/bus). Đây là hạ tầng truyền tải các yêu cầu đọc/ghi và phản hồi giữa các thành phần. Hệ thống kết nối định tuyến một yêu cầu từ thành phần chủ động đến thành phần đích dựa trên địa chỉ, đồng thời điều phối khi có nhiều thành phần cùng muốn truy cập tài nguyên chung. Giao thức kết nối quy định khuôn dạng giao dịch, các loại thao tác được phép và cách bắt tay giữa hai phía. Giao thức TileLink được sử dụng trong đề tài được trình bày ở mục 2.6.

Bộ điều khiển truy cập bộ nhớ trực tiếp (Direct Memory Access – DMA). DMA là một thành phần chuyên trách việc vận chuyển khối dữ liệu giữa các vùng nhớ hoặc giữa bộ nhớ và thiết bị ngoại vi mà không cần CPU thực hiện từng thao tác đọc/ghi. Khi hoạt động, DMA đóng vai trò là một thành phần chủ động trên hệ thống kết nối, tương tự CPU. Cơ chế DMA được trình bày ở mục 2.7.

Thiết bị ngoại vi (peripheral). Các thiết bị ngoại vi đảm nhận chức năng giao tiếp giữa hệ thống và môi trường bên ngoài, ví dụ giao tiếp nối tiếp, hiển thị hoặc nhận tín hiệu đầu vào từ người dùng. Các thiết bị này thường được điều khiển thông qua các thanh ghi được ánh xạ vào không gian địa chỉ bộ nhớ (mục 2.8).



Hình 2.1: Sơ đồ khối của một hệ thống SoC — CPU

2.2. Các mức mô phỏng và vị trí của đề tài

Việc mô phỏng một hệ thống tính toán có thể được thực hiện ở nhiều mức trừu tượng khác nhau, mỗi mức đánh đổi giữa độ chi tiết và tốc độ mô phỏng. Phân biệt các mức này giúp xác định rõ những gì một trình mô phỏng tái hiện và những gì nằm ngoài phạm vi của nó.

Mô phỏng mức tập lệnh (Instruction Set Architecture – ISA). Ở mức này, trình mô phỏng chỉ tái hiện trạng thái kiến trúc nhìn thấy được từ phần mềm — tập thanh ghi, bộ đếm chương trình và bộ nhớ — và ngữ nghĩa của từng lệnh. Mỗi lệnh được thực thi cho ra kết quả đúng theo đặc tả, nhưng trình mô phỏng không quan tâm đến cách lệnh được thực hiện bên trong vi kiến trúc. Cách tiếp cận này cho tốc độ cao và đủ để kiểm chứng tính đúng đắn về mặt chức năng của chương trình.

Mô phỏng mức hành vi/chức năng (functional/behavioral). Mức này mở rộng mô phỏng ISA bằng cách mô hình hóa hành vi của các thành phần ngoài lõi xử lý — bộ nhớ đệm, đơn vị quản lý bộ nhớ, hệ thống bus, DMA, thiết bị ngoại vi — và quá trình các thành phần trao đổi giao dịch với nhau. Mức này có thể gán các giá trị độ trễ ở dạng số chu kỳ cho từng loại thao tác nhằm phản ánh tương quan chi phí giữa chúng (ví dụ truy cập bộ nhớ chính tốn nhiều chu kỳ hơn truy cập bộ nhớ đệm), nhưng không mô tả chi tiết cấu trúc vi kiến trúc bên trong.

Mô phỏng chính xác theo chu kỳ (cycle-accurate). Ở mức này, trình mô phỏng tái hiện hoạt động của vi kiến trúc theo từng chu kỳ đồng hồ, bao gồm các tầng của đường ống lệnh (pipeline), hiện tượng xung đột (hazard) và cơ chế chuyển tiếp dữ liệu. Mức này cho thông tin định thời chính xác hơn nhưng phức tạp và chậm hơn đáng kể. Ở mức chi tiết nhất là mô phỏng tại mức truyền thanh ghi (Register-Transfer Level – RTL) hoặc mức cổng logic, thường phục vụ cho thiết kế và kiểm chứng phần cứng.

Ngoài cách phân loại theo độ chi tiết của lõi xử lý, người ta còn nói đến mô phỏng ở mức hệ thống (system-level) khi đối tượng mô phỏng là toàn bộ SoC — bao gồm CPU, bộ nhớ, bus và ngoại vi — thường ở mức giao dịch hoặc mức hành vi, nhằm quan sát sự tương tác giữa nhiều thành phần.

2.3. Kiến trúc tập lệnh RISC-V

2.3.1. Tổng quan và triết lý mô-đun hóa ISA

Kiến trúc tập lệnh (Instruction Set Architecture – ISA) là lớp giao tiếp trừu tượng giữa phần mềm và phần cứng: nó định nghĩa tập các lệnh, tập thanh ghi, các kiểu dữ liệu, mô hình bộ nhớ và quy ước mã hóa lệnh mà bộ xử lý phải tuân theo. RISC-V là một ISA theo trường phái máy tính với tập lệnh rút gọn (Reduced Instruction Set Computer – RISC), trong đó các lệnh có khuôn dạng đều đặn và đa số thao tác tính toán chỉ làm việc trên thanh ghi, tách biệt với các lệnh truy cập bộ nhớ (mô hình nạp – lưu, hay load–store) [1]. Đặc điểm này khác với trường phái máy tính với tập lệnh

phức tạp (Complex Instruction Set Computer – CISC), nơi một lệnh có thể vừa truy cập bộ nhớ vừa thực hiện tính toán và có độ dài thay đổi.

Điểm nổi bật trong triết lý thiết kế của RISC-V là tính mô-đun [1]. Thay vì định nghĩa một tập lệnh lớn cố định, RISC-V quy định một tập lệnh cơ sở nhỏ gọn bắt buộc và một loạt phần mở rộng tùy chọn, mỗi phần bổ sung một nhóm chức năng. Tập cơ sở cho kiến trúc 32-bit là RV32I (Base Integer). Các phần mở rộng tiêu biểu gồm: M cho phép nhân/chia số nguyên, A cho thao tác nguyên tử trên bộ nhớ, F cho số thực dấu phẩy động độ chính xác đơn, D cho độ chính xác kép, C cho lệnh nén 16-bit, cùng nhiều phần khác. Tên một biến thể được ghép từ chiều rộng thanh ghi và danh sách phần mở rộng; ví dụ "RV32IMF" chỉ kiến trúc 32-bit gồm tập cơ sở I và hai phần mở rộng M, F. Cách tổ chức mô-đun này cho phép một hệ thống chỉ hiện thực đúng những phần cần thiết, đồng thời thuận tiện cho việc trình bày từng lớp khái niệm một cách tuần tự. Bảng 2.1 tóm tắt các phần mở rộng RISC-V liên quan đến đề tài và phạm vi được hỗ trợ.

Bảng 2.1. Các phần mở rộng RISC-V và phạm vi trong đề tài

Ký hiệu	Tên đầy đủ	Nội dung chính	Phạm vi trong đề tài
I	Base Integer Instruction Set	Tập lệnh số nguyên cơ sở: tính toán, nạp/lưu, rẽ nhánh, nhảy	Có (tập con ở chế độ người dùng)
M	Standard Extension for Integer Multiplication and Division	Nhân và chia số nguyên có dấu/không dấu	Có
F	Standard Extension for Single-Precision Floating-Point	Số thực độ chính xác đơn theo IEEE 754	Có (tập con, chưa có FCSR/fflags/frm)
A	Standard Extension for Atomic Instructions	Thao tác đọc–sửa–ghi nguyên tử trên bộ nhớ	Một phần (chỉ AMOADD.W) — ngoài tên đề tài
Các phần khác (D, C, ...)	—	Số thực kép, lệnh nén, ...	Ngoài phạm vi

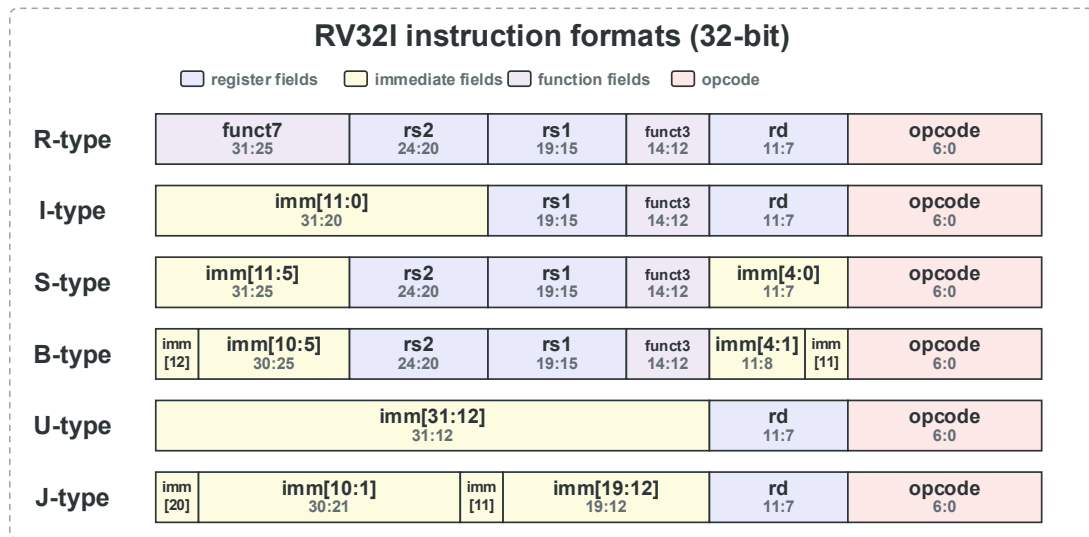
Ghi chú: Tên đề tài là RV32IMF; phần mở rộng A được liệt kê vì hệ thống có hiện thực một lệnh nguyên tử minh họa, được trình bày ở mục 2.3.5.

2.3.2. Tập lệnh cơ sở RV32I

Tập lệnh cơ sở RV32I định nghĩa 32 thanh ghi số nguyên rộng 32 bit, ký hiệu x0 đến x31, cùng một bộ đếm chương trình (Program Counter – PC) trở tới lệnh đang thực thi [1]. Thanh ghi x0 được nối cứng giá trị 0: mọi thao tác đọc x0 đều trả về 0 và mọi thao tác ghi vào x0 đều không có hiệu lực; đặc điểm này được dùng để biểu diễn nhiều thao tác thông dụng một cách gọn gàng. Các thanh ghi còn lại có công dụng chung, với quy ước sử dụng (lưu địa chỉ trả về, con trỏ ngăn xếp, tham số hàm...) được quy định bởi giao diện nhị phân ứng dụng (ABI); ánh xạ tên thanh ghi theo ABI được trình bày ở Chương 4.

Các lệnh RV32I có chiều dài cố định 32 bit và được tổ chức theo một số khuôn dạng (format) thống nhất, giúp việc giải mã trở nên đơn giản và đều đặn. Sáu khuôn dạng cơ bản gồm R, I, S, B, U và J, khác nhau ở cách bố trí các trường thanh ghi và trường giá trị tức thời (immediate), được minh họa ở Hình 2.2:

- Khuôn dạng R dùng cho các lệnh tính toán thanh ghi – thanh ghi (hai toán hạng nguồn rs1, rs2, một đích rd), với các trường funct3 và funct7 xác định cụ thể phép toán.
- Khuôn dạng I dùng cho các lệnh có một toán hạng tức thời 12 bit, gồm lệnh tính toán với hằng số, lệnh nạp dữ liệu và lệnh nhảy gián tiếp JALR.
- Khuôn dạng S dùng cho lệnh lưu dữ liệu, trong đó giá trị tức thời bị tách thành hai phần để giữ nguyên vị trí các trường thanh ghi.
- Khuôn dạng B là biến thể của S dùng cho rẽ nhánh có điều kiện, với giá trị tức thời mã hóa độ dịch địa chỉ đích (là bội của 2).
- Khuôn dạng U dùng cho các lệnh mang giá trị tức thời 20 bit ở phần cao (LUI, AUIPC).
- Khuôn dạng J dùng cho lệnh nhảy JAL, với giá trị tức thời mã hóa độ dịch tương đối so với PC.



Hình 2.2: Sáu khuôn dạng lệnh RV32I (R/I/S/B/U/J) với bố trí các trường opcode, rd, funct3, rs1, rs2, funct7 và các trường giá trị tức thời tương ứng.

Về chức năng, các lệnh RV32I được chia thành các nhóm chính như tổng hợp ở Bảng 2.2. Riêng lệnh FENCE dùng để đặt ràng buộc thứ tự cho các truy cập bộ nhớ; trong các hệ thống đơn lõi đơn giản, thao tác này thường không tạo ra hiệu ứng quan sát được.

Bảng 2.2. Các nhóm lệnh của tập cơ sở RV32I

Nhóm lệnh	Khuôn dạng	Lệnh tiêu biểu	Ý nghĩa
Tính toán thanh ghi – Thanh ghi	R	ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL, SRA	Phép số học, luận lý, so sánh và dịch bit giữa hai thanh ghi
Tính toán với hằng số	I	ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI	Như trên nhưng một toán hạng là giá trị tức thời
Nạp dữ liệu (load)	I	LB, LH, LW, LBU, LHU	Đọc byte/nửa từ/từ từ bộ nhớ vào thanh ghi
Lưu dữ liệu (store)	S	SB, SH, SW	Ghi byte/nửa từ/từ từ thanh ghi xuống bộ nhớ

Nhóm lệnh	Khuôn dạng	Lệnh tiêu biểu	Ý nghĩa
Rẽ nhánh có điều kiện	B	BEQ, BNE, BLT, BGE, BLTU, BGEU	Thay đổi luồng thực thi theo điều kiện so sánh
Nhảy vô điều kiện	J, I	JAL, JALR	Nhảy và lưu địa chỉ trả về (gọi/trả về chương trình con)
Nạp hằng số lớn	U	LUI, AUIPC	Nạp 20 bit cao của hằng số / tính địa chỉ tương đối PC
Lệnh hệ thống	I	ECALL, EBREAK	Gọi dịch vụ hệ thống / điểm dừng gỡ lỗi
Đồng bộ bộ nhớ	I	FENCE	Đặt ràng buộc thứ tự truy cập bộ nhớ

2.3.3. Mở rộng nhân/chia số nguyên RV32M

Phần mở rộng M bổ sung cho RV32I tám lệnh nhân và chia số nguyên, vốn không có trong tập cơ sở [1]. Nhóm nhân gồm MUL lấy 32 bit thấp của tích, và ba lệnh lấy 32 bit cao của tích là MULH (cả hai toán hạng có dấu), MULHU (cả hai không dấu) và MULHSU (một có dấu, một không dấu) — việc tách phần cao/phần thấp cho phép thu được kết quả nhân đầy đủ 64 bit từ hai thanh ghi 32 bit. Nhóm chia gồm DIV và DIVU (chia có dấu và không dấu) cùng REM và REMU (lấy phần dư tương ứng).

Đặc tả RISC-V quy định rõ hành vi cho các trường hợp biên mà không sinh ngoại lệ phần cứng: khi chia cho 0, lệnh chia trả về giá trị toàn bit 1 (tức -1 với phép có dấu) và lệnh lấy dư trả về chính số bị chia; khi xảy ra tràn trong phép chia có dấu (số bị chia là giá trị âm nhỏ nhất biểu diễn được chia cho -1), thương trả về chính số bị chia còn phần dư bằng 0 [1]. Việc quy định tường minh các trường hợp này giúp kết quả là xác định và nhất quán giữa các hiện thực.

2.3.4. Mở rộng dấu phẩy động RV32F và chuẩn IEEE 754

Phần mở rộng F bổ sung khả năng tính toán trên số thực dấu phẩy động độ chính xác đơn và một tệp thanh ghi dấu phẩy động riêng gồm 32 thanh ghi $f0$ đến $f31$ [1].

Việc tách riêng tệp thanh ghi số thực khỏi tệp thanh ghi số nguyên cho phép thực hiện đồng thời các thao tác trên hai miền dữ liệu và phản ánh sự tồn tại của một đơn vị dấu phẩy động độc lập.

Cách biểu diễn và quy tắc tính toán số thực trong RISC-V tuân theo chuẩn IEEE 754 về số học dấu phẩy động [10]. Với độ chính xác đơn, một số thực được biểu diễn bằng 32 bit gồm ba trường: 1 bit dấu, 8 bit số mũ (lưu theo dạng lệch với độ lệch 127) và 23 bit phần định trị (fraction). Chuẩn này định nghĩa một số giá trị đặc biệt ngoài các số chuẩn tắc: số 0 có dấu (+0 và -0), vô cùng có dấu ($\pm\infty$), các số dưới chuẩn tắc (subnormal) cho phép biểu diễn các giá trị rất nhỏ gần 0, và giá trị "không phải số" (Not-a-Number – NaN) dùng cho các kết quả không xác định, được phân biệt thành NaN báo hiệu (signaling) và NaN im lặng (quiet). Chuẩn cũng quy định năm chế độ làm tròn — trong đó thông dụng là làm tròn tới giá trị gần nhất, lấy chẵn khi ở giữa (round to nearest, ties to even) và làm tròn về 0 (round toward zero) — cùng năm cờ ngoại lệ ghi nhận các tình huống như chia cho 0, tràn trên, tràn dưới, kết quả không chính xác và thao tác không hợp lệ.

Trong RISC-V, chế độ làm tròn và các cờ ngoại lệ nói trên được quản lý qua thanh ghi điều khiển và trạng thái dấu phẩy động (Floating-point Control and Status Register – FCSR), gồm trường chọn chế độ làm tròn động (frm) và trường cờ ngoại lệ tích lũy (fflags) [1]. Mỗi lệnh dấu phẩy động cũng có một trường chỉ định chế độ làm tròn tĩnh ngay trong mã lệnh.

Phần mở rộng F cung cấp nhiều nhóm lệnh: nạp/lưu số thực (FLW, FSW); số học cơ bản (FADD.S, FSUB.S, FMUL.S, FDIV.S, FSQRT.S); nhân-cộng kết hợp (FMADD.S, FMSUB.S, FNMSUB.S, FNMADD.S); cực trị (FMIN.S, FMAX.S); thao tác trên bit dấu (FSGNJ.S, FSGNJN.S, FSGNJX.S); so sánh (FEQ.S, FLT.S, FLE.S); phân loại giá trị (FCLASS.S); chuyển đổi giữa số thực và số nguyên (FCVT.W.S, FCVT.WU.S, FCVT.S.W, FCVT.S.WU); và di chuyển bit giữa hai tệp thanh ghi (FMV.X.W, FMV.W.X). Các nhóm RV32M và RV32F được tổng hợp ở Bảng 2.3.

Bảng 2.3. Các nhóm lệnh RV32M và RV32F được hỗ trợ

Phần mở rộng	Nhóm chức năng	Lệnh
RV32M	Nhân	MUL, MULH, MULHSU, MULHU
RV32M	Chia và lấy dư	DIV, DIVU, REM, REMU
RV32F	Nạp/lưu	FLW, FSW
RV32F	Số học	FADD.S, FSUB.S, FMUL.S, FDIV.S, FSQRT.S
RV32F	Nhân–cộng kết hợp	FMADD.S, FMSUB.S, FNMSUB.S, FNMADD.S
RV32F	Cực trị	FMIN.S, FMAX.S
RV32F	Thao tác bit dấu	FSGNJ.S, FSGNJN.S, FSGNJX.S
RV32F	So sánh	FEQ.S, FLT.S, FLE.S
RV32F	Phân loại	FCLASS.S
RV32F	Chuyển đổi	FCVT.W.S, FCVT.WU.S, FCVT.S.W, FCVT.S.WU
RV32F	Di chuyển bit	FMV.X.W, FMV.W.X

2.3.5. Thao tác nguyên tử RV32A (mở rộng ngoài tên đề tài)

Phần mở rộng A bổ sung các lệnh thao tác nguyên tử trên bộ nhớ, phục vụ cho việc đồng bộ giữa nhiều thành phần cùng truy cập một vùng nhớ chung [1]. Nhóm lệnh tiêu biểu là các thao tác bộ nhớ nguyên tử (Atomic Memory Operation – AMO), trong đó một lệnh thực hiện trọn vẹn chuỗi đọc giá trị hiện có, tính toán với một toán hạng, rồi ghi kết quả trở lại — toàn bộ chuỗi này được bảo đảm không bị xen kẽ bởi truy cập khác. Ví dụ, AMOADD.W đọc một từ 32-bit từ bộ nhớ, cộng với giá trị trong thanh ghi và ghi tổng trở lại địa chỉ đó, đồng thời trả về giá trị cũ. Trên hệ thống kết nối, thao tác này tương ứng với giao dịch số học nguyên tử (mục 2.6).

2.4. Quản lý bộ nhớ và đơn vị quản lý bộ nhớ (MMU)

Trong các hệ thống hiện đại, chương trình không truy cập trực tiếp bộ nhớ vật lý mà làm việc trên một không gian địa chỉ ảo (virtual address – VA). Một thành phần

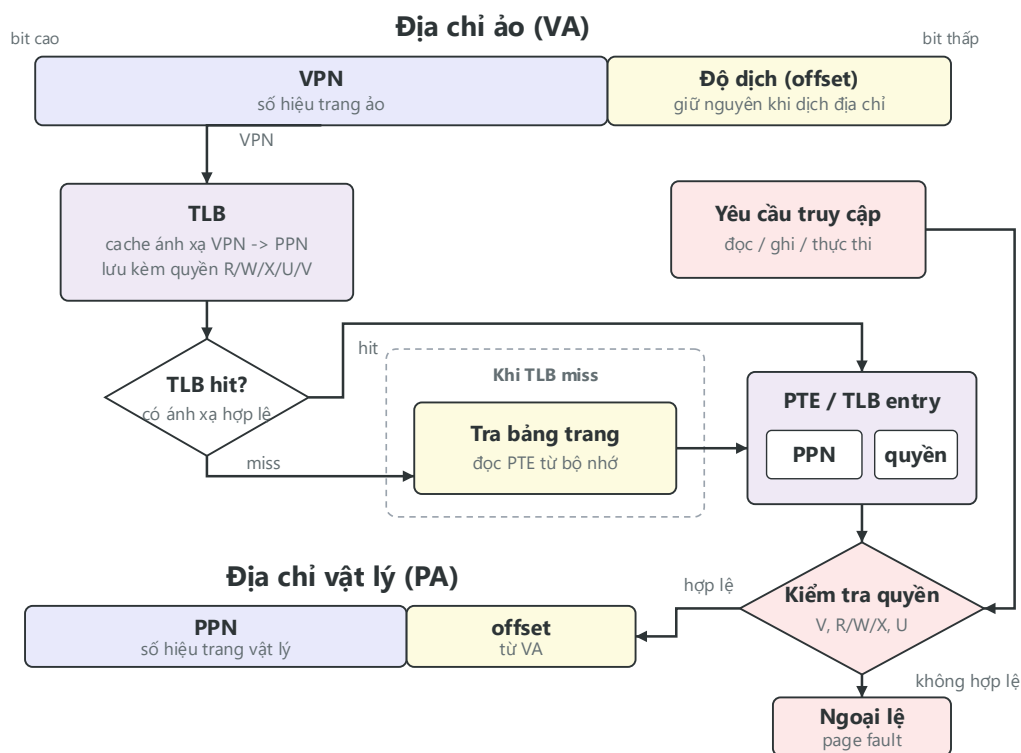
phần cứng gọi là đơn vị quản lý bộ nhớ (Memory Management Unit – MMU) chịu trách nhiệm dịch mỗi địa chỉ ảo thành địa chỉ vật lý (physical address – PA) tương ứng trước khi truy cập bộ nhớ thực [11]. Cơ chế này tạo nên một lớp trừu tượng giữa khung nhìn bộ nhớ của chương trình và bố trí bộ nhớ vật lý, mang lại nhiều lợi ích như cô lập không gian nhớ giữa các tiến trình, kiểm soát quyền truy cập và cho phép bố trí bộ nhớ linh hoạt.

Kỹ thuật phổ biến để tổ chức dịch địa chỉ là phân trang (paging). Không gian địa chỉ được chia thành các trang (page) có kích thước cố định, thường là 4 KiB. Tương ứng, bộ nhớ vật lý được chia thành các khung trang (page frame) cùng kích thước. Khi đó, một địa chỉ ảo được tách thành hai phần: số hiệu trang ảo (Virtual Page Number – VPN) ở phần cao và độ dịch trong trang (page offset) ở phần thấp. Việc dịch địa chỉ chỉ thay thế VPN bằng số hiệu khung trang vật lý (Physical Page Number – PPN) tương ứng, còn độ dịch trong trang được giữ nguyên (Hình 2.3). Ánh xạ từ VPN sang PPN được lưu trong bảng trang (page table); mỗi mục của bảng, gọi là mục bảng trang (Page Table Entry – PTE), chứa số hiệu khung trang vật lý cùng các bit quyền truy cập.

Các bit quyền trong PTE cho phép kiểm soát thao tác nào được phép trên mỗi trang, thông thường gồm quyền đọc (R), ghi (W) và thực thi (X). Khi một truy cập vi phạm quyền (ví dụ ghi vào trang chỉ đọc), MMU phát hiện và báo lỗi thay vì cho phép truy cập. Ngoài ra, mỗi vùng địa chỉ còn được gắn thuộc tính cho biết có được phép lưu vào bộ nhớ đệm hay không: vùng có thể lưu đệm (cacheable) — điển hình là bộ nhớ chính — cho phép sao chép tạm vào bộ nhớ đệm để tăng tốc; ngược lại, vùng không lưu đệm (non-cacheable) — điển hình là vùng thanh ghi của thiết bị ngoại vi — phải được truy cập trực tiếp vì mỗi lần đọc/ghi có thể gây ra hiệu ứng phụ (mục 2.8).

Việc tra bảng trang cho mỗi truy cập bộ nhớ sẽ rất tốn kém nếu phải thực hiện đầy đủ mỗi lần. Để khắc phục, MMU sử dụng một bộ nhớ đệm chuyên dụng cho kết quả dịch địa chỉ gần đây gọi là bộ đệm dịch địa chỉ (Translation Lookaside Buffer – TLB). TLB khai thác nguyên lý cục bộ (locality) — xu hướng chương trình truy cập lặp lại

một số trang trong khoảng thời gian gần nhau — nên phần lớn các lần dịch có thể được phục vụ nhanh từ TLB mà không cần tra bảng trang. TLB thường được tổ chức theo kiểu kết hợp tập (set-associative), tương tự bộ nhớ đệm dữ liệu được trình bày ở mục 2.5.



Hình 2.3: Cơ chế dịch địa chỉ ảo sang vật lý.

2.5. Bộ nhớ đệm (cache)

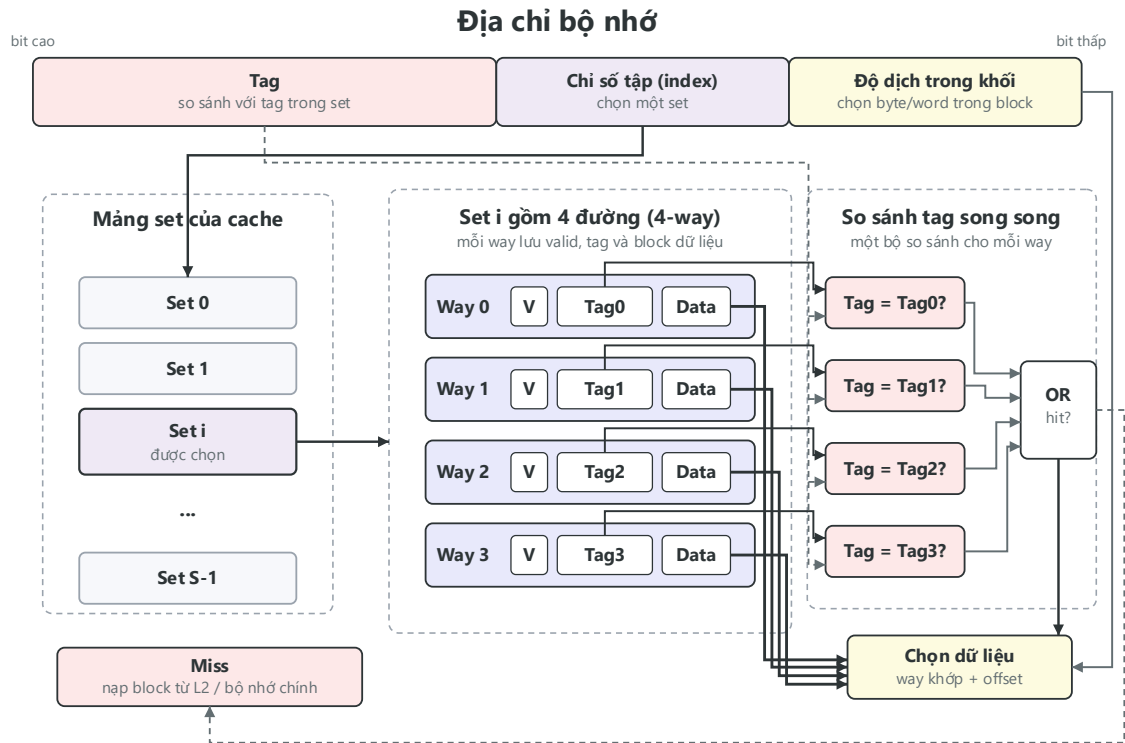
Bộ nhớ đệm (cache) là một bộ nhớ dung lượng nhỏ nhưng tốc độ cao, đặt giữa bộ xử lý và bộ nhớ chính, nhằm thu hẹp khoảng cách lớn về tốc độ truy cập giữa hai thành phần này [11], [12]. Hiệu quả của bộ nhớ đệm dựa trên nguyên lý cục bộ, gồm cục bộ theo thời gian (một ô nhớ vừa được truy cập có nhiều khả năng sẽ được truy cập lại trong tương lai gần) và cục bộ theo không gian (các ô nhớ lân cận một ô vừa truy cập có nhiều khả năng được truy cập kế tiếp). Nhờ đó, việc giữ lại bản sao của dữ liệu được dùng gần đây — và nạp theo từng khối liên kề — giúp phần lớn các truy cập sau được phục vụ nhanh từ bộ nhớ đệm.

Tổ chức kết hợp tập. Dữ liệu trong bộ nhớ đệm được quản lý theo đơn vị khối (block, hay dòng – line), mỗi khối gồm một số byte liên kề trong bộ nhớ. Tổ chức phổ biến là kết hợp tập (set-associative): bộ nhớ đệm được chia thành nhiều tập (set), mỗi tập gồm một số đường (way) có thể chứa khối. Để xác định vị trí của một địa chỉ, địa chỉ được tách thành ba phần (Hình 2.4): độ dịch trong khối (block offset) chọn byte bên trong khối; chỉ số tập (index) chọn tập; và thẻ (tag) được lưu kèm và so sánh với phần thẻ của các đường trong tập để xác định có khối cần tìm hay không. Khi dữ liệu cần tìm có trong bộ nhớ đệm thì gọi là trúng (hit); ngược lại là trượt (miss), khi đó khối tương ứng được nạp từ cấp bộ nhớ thấp hơn, thường mất nhiều chu kỳ hơn đáng kể.

Chính sách ghi. Khi bộ xử lý ghi dữ liệu, có hai chính sách cập nhật chính. Với ghi xuyên (write-through), dữ liệu được ghi đồng thời vào bộ nhớ đệm và cấp bộ nhớ thấp hơn, giúp giữ nhất quán đơn giản nhưng tạo nhiều lưu lượng ghi. Với ghi trả sau (write-back), dữ liệu chỉ được ghi vào bộ nhớ đệm và khối được đánh dấu "bẩn" (dirty); việc cập nhật xuống bộ nhớ chính chỉ diễn ra khi khối bị thay thế, giúp giảm lưu lượng ghi nhưng cần thêm cơ chế theo dõi trạng thái.

Chính sách thay thế. Khi cần nạp khối mới vào một tập đã đầy, một khối hiện có phải bị thay thế. Một chính sách thường dùng là thay thế khối dùng lâu nhất chưa dùng (Least Recently Used – LRU), tức loại bỏ khối có lần truy cập gần nhất xa nhất trong quá khứ, dựa trên giả định về cục bộ theo thời gian.

Phân cấp bộ nhớ đệm. Để cân bằng giữa tốc độ và dung lượng, bộ nhớ đệm thường được tổ chức thành nhiều cấp. Cấp một (L1) nhỏ và nhanh nhất, thường tách riêng phần chứa lệnh (L1I) và phần chứa dữ liệu (L1D); cấp hai (L2) lớn hơn và chậm hơn, đóng vai trò đệm chung phía sau L1. Một truy cập trượt ở L1 sẽ tìm tiếp ở L2 trước khi xuống bộ nhớ chính. Riêng các vùng được đánh dấu không lưu đệm — điển hình là vùng vào/ra ánh xạ bộ nhớ — được truy cập bỏ qua (bypass) bộ nhớ đệm để bảo đảm mỗi thao tác đọc/ghi tác động trực tiếp lên thiết bị.



Hình 2.4: Tổ chức bộ nhớ đệm kết hợp tập.

2.6. Giao thức kết nối TileLink

2.6.1. Vai trò interconnect và ba cấp độ TileLink

Trong một SoC có nhiều thành phần cùng truy cập tài nguyên chung, hệ thống kết nối (interconnect) đảm nhận việc truyền tải các yêu cầu và phản hồi giữa chúng theo một giao thức thống nhất. Giao thức kết nối quy định khuôn dạng của một giao dịch, tập các thao tác được phép, cách định tuyến yêu cầu tới đúng thành phần đích theo địa chỉ, và quy tắc bắt tay (handshake) bảo đảm hai phía đồng bộ khi trao đổi. TileLink là một giao thức kết nối được công bố công khai, sử dụng trong nhiều thiết kế dựa trên RISC-V, với mô hình bộ nhớ chia sẻ và không gian địa chỉ phẳng [2].

TileLink định nghĩa ba cấp độ tuân thủ, tăng dần về mức độ phức tạp và tập thao tác hỗ trợ [2]:

- TileLink Uncached Lightweight (TL-UL) là cấp độ đơn giản nhất, chỉ hỗ trợ các thao tác đọc và ghi đơn lẻ, mỗi giao dịch truyền một đơn vị dữ liệu. Cấp độ

này phù hợp cho các thành phần đơn giản như thanh ghi điều khiển của thiết bị ngoại vi.

- TileLink Uncached Heavyweight (TL-UH) mở rộng TL-UL bằng cách bổ sung khả năng truyền theo chuỗi nhiều đơn vị dữ liệu (burst), các thao tác nguyên tử (số học và luận lý) và các lệnh gợi ý (hint). Cấp độ này phù hợp cho truy cập bộ nhớ khối lượng lớn.
- TileLink Cached (TL-C) là cấp độ đầy đủ nhất, bổ sung cơ chế bảo đảm nhất quán bộ nhớ đệm (cache coherence) giữa nhiều thành phần có bộ nhớ đệm riêng, thông qua các thao tác xin quyền và thu hồi quyền sở hữu khối dữ liệu.

Bảng 2.4 so sánh ba cấp độ này theo tập thao tác và số kênh truyền sử dụng (khái niệm kênh được trình bày ở mục 2.6.2).

Bảng 2.4. So sánh ba cấp độ của giao thức TileLink

Đặc điểm	TL-UL	TL-UH	TL-C
Đọc/ghi đơn lẻ	Có	Có	Có
Truyền chuỗi (burst)	Không	Có	Có
Thao tác nguyên tử (số học/luận lý)	Không	Có	Có
Lệnh gợi ý (hint)	Tùy chọn	Có	Có
Nhất quán bộ nhớ đệm (coherence)	Không	Không	Có
Các kênh sử dụng	A, D	A, D	A, B, C, D, E

2.6.2. Mô hình master/slave và các kênh truyền

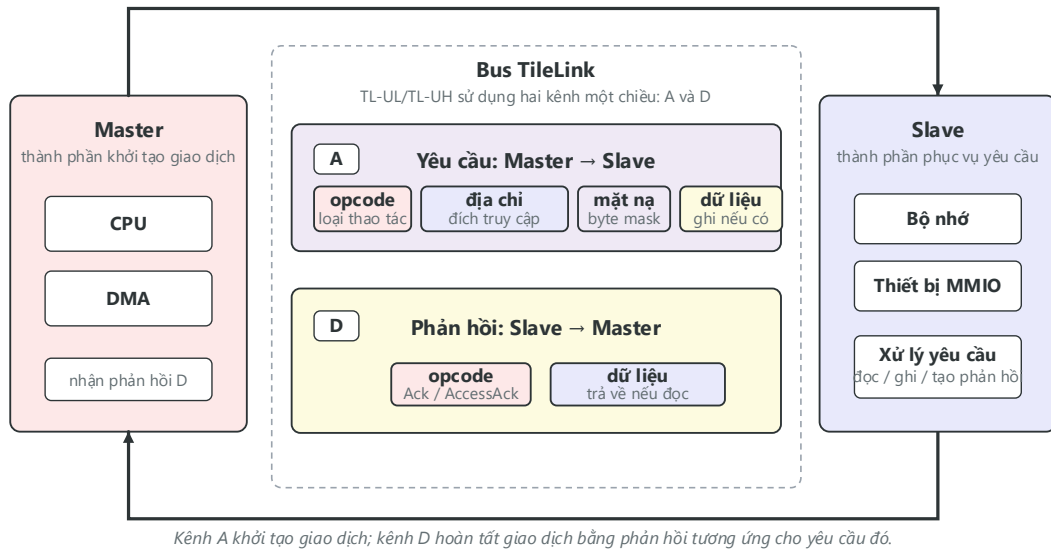
TileLink hoạt động theo mô hình chủ động – bị động (master–slave). Một thành phần chủ động (master) là bên khởi tạo giao dịch bằng cách phát yêu cầu (ví dụ CPU hoặc DMA); một thành phần bị động (slave) là bên tiếp nhận và phục vụ yêu cầu rồi trả phản hồi (ví dụ bộ nhớ hoặc thiết bị ngoại vi). Mỗi giao dịch gồm một yêu cầu đi từ master đến slave và một phản hồi đi theo chiều ngược lại.

Giao thức tổ chức việc truyền tin trên tối đa năm kênh một chiều, ký hiệu A đến E, với vai trò như sau [2]:

- Kênh A mang yêu cầu từ master đến slave (đọc, ghi, thao tác nguyên tử, gợi ý). Đây là kênh khởi tạo giao dịch.
- Kênh B mang yêu cầu từ slave đến master, dùng trong cơ chế nhất quán để dò/thu hồi khối (chỉ có ở TL-C).
- Kênh C mang phản hồi của master cho kênh B, ví dụ trả lại hoặc ghi xuống khối bị thu hồi (chỉ có ở TL-C).
- Kênh D mang phản hồi từ slave về master cho yêu cầu trên kênh A (báo nhận, dữ liệu trả về...).
- Kênh E mang xác nhận cuối cùng của master để hoàn tất một giao dịch cấp quyền (chỉ có ở TL-C).

Như vậy, các cấp độ không nhất quán (TL-UL, TL-UH) chỉ cần hai kênh A và D, còn cấp độ nhất quán (TL-C) cần đủ năm kênh. Mỗi giao dịch trên kênh A và D mang một mã thao tác (opcode) xác định loại thao tác. Hình 2.5 minh họa một giao dịch cơ bản trên hai kênh A và D giữa master và slave, và Bảng 2.5 liệt kê các opcode của hai kênh này. Ngoài opcode, kênh A còn dùng một mặt nạ byte (byte mask) để chỉ định những byte nào trong một từ thực sự được ghi, phục vụ các thao tác ghi một phần.

Mô hình giao dịch TileLink trên hai kênh A và D



Hình 2.5: Mô hình giao dịch TileLink trên hai kênh A và D.

Bảng 2.5. Mã thao tác (opcode) trên kênh A và kênh D của TileLink

Kênh	Opcode	Mã số	Ý nghĩa	Phạm vi đề tài
A	PutFullData	0	Ghi đầy đủ một khối dữ liệu	Có (UL, UH)
A	PutPartialData	1	Ghi một phần, dùng mặt nạ byte	Có (UL, UH)
A	ArithmeticData	2	Thao tác số học nguyên tử (MIN/MAX/ADD...)	Có (UH)
A	LogicalData	3	Thao tác luận lý nguyên tử (XOR/OR/AND/SWAP)	Có (UH)
A	Get	4	Đọc dữ liệu	Có (UL, UH)
A	Intent	5	Lệnh gợi ý/đặt trước (hint)	Có (UL, UH)
D	AccessAck	0	Báo nhận cho thao tác ghi (không kèm dữ liệu)	Có
D	AccessAckData	1	Báo nhận kèm dữ liệu (cho đọc/nguyên tử)	Có
D	HintAck	2	Báo nhận cho lệnh gợi ý	Có
D	Grant / GrantData	4 / 5	Cấp quyền sở hữu khối (kèm/không kèm dữ liệu)	Ngoài phạm vi (TL-C)

Kênh	Opcode	Mã số	Ý nghĩa	Phạm vi đề tài
D	ReleaseAck	6	Báo nhận thao tác thu hồi khối	Ngoài phạm vi (TL-C)

Ghi chú: Mã số opcode theo định nghĩa trong source `src/js/tilelink.js`, nhất quán với đặc tả TileLink [2]. Các opcode kênh D thuộc nhóm cấp quyền (Grant/GrantData/ReleaseAck) liên quan đến cơ chế nhất quán của TL-C, được định nghĩa để tham chiếu nhưng không được sử dụng trong luồng giao dịch của hệ thống.

2.6.3. Tập con TileLink được sử dụng trong đề tài

Để bảo đảm tính chính xác khi trình bày, cần nêu rõ phân giao thức TileLink mà hệ thống thực sự hiện thực. Hệ thống mô phỏng ở mức giao dịch trên hai kênh A và D, tương ứng hai cấp độ TL-UL và TL-UH. Hệ thống không hiện thực ba kênh B, C, E cùng các luồng dò/thu hồi/cấp quyền, và do đó không bảo đảm nhất quán bộ nhớ theo cấp độ TL-C (xem mục 7 trong phân tích phạm vi). Về điều phối truy cập, cơ chế được mô hình hóa ở mức hàng đợi yêu cầu với một giao dịch được xử lý tại một thời điểm; trên cơ sở đó, hệ thống thể hiện việc nhiều thành phần (CPU và DMA) cùng đóng vai trò master phát yêu cầu, nhưng đây chưa phải là một cơ chế phân xử (arbitration) đa master đầy đủ.

2.7. Truy cập bộ nhớ trực tiếp (DMA)

Khi cần di chuyển một khối lượng lớn dữ liệu giữa hai vùng nhớ hoặc giữa bộ nhớ và thiết bị ngoại vi, việc để CPU thực hiện từng cặp lệnh nạp – lưu là không hiệu quả: CPU bị chiếm dụng hoàn toàn cho thao tác sao chép và không thể làm việc khác trong suốt quá trình đó. Truy cập bộ nhớ trực tiếp (Direct Memory Access – DMA) giải quyết vấn đề này bằng cách giao việc vận chuyển dữ liệu cho một bộ điều khiển chuyên trách [11]. CPU chỉ cần thiết lập thông số cho lần truyền rồi tiếp tục thực thi công việc khác, trong khi bộ điều khiển DMA tự thực hiện việc đọc và ghi dữ liệu với vai trò một thành phần chủ động (master) trên hệ thống kết nối. Nhờ đó, việc di

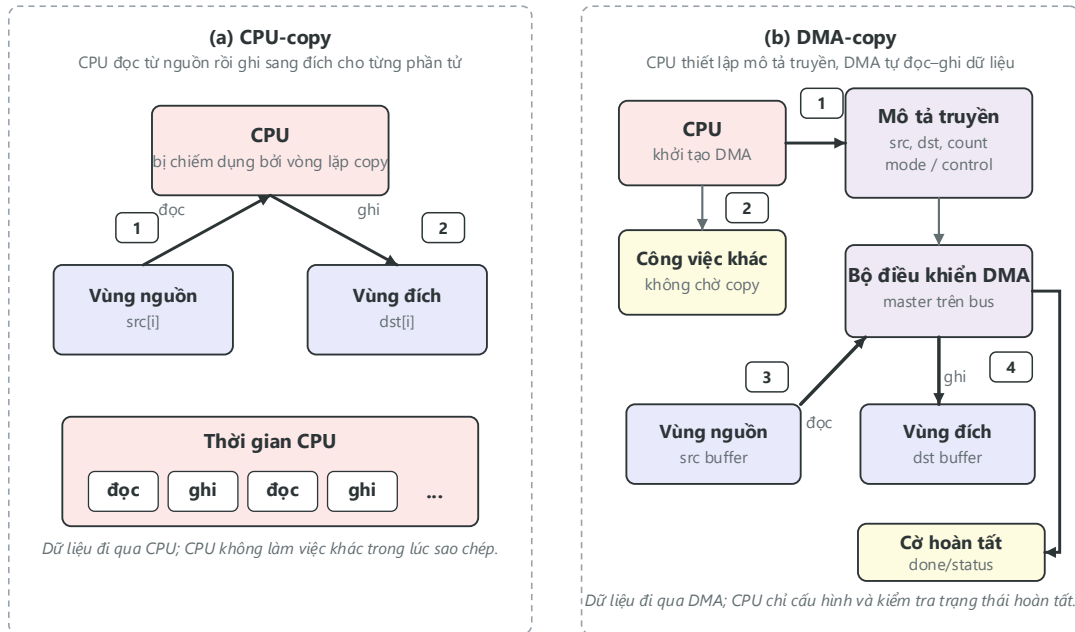
chuyển dữ liệu có thể diễn ra song song với tính toán của CPU, giúp giảm tải cho CPU.

Thông số của một lần truyền thường được tổ chức thành một cấu trúc gọi là mô tả truyền (descriptor), bao gồm địa chỉ nguồn, địa chỉ đích, số lượng dữ liệu cần truyền và các tham số cấu hình khác. Bộ điều khiển đọc mô tả này để biết cần truyền gì và truyền như thế nào.

Trong quá trình truyền, địa chỉ nguồn và địa chỉ đích có thể được xử lý theo các chế độ khác nhau. Với địa chỉ cố định, mỗi phần tử được đọc từ hoặc ghi vào cùng một địa chỉ — phù hợp khi một đầu của lần truyền là một thanh ghi dữ liệu của thiết bị ngoại vi. Với địa chỉ tăng dần, địa chỉ được tăng sau mỗi phần tử — phù hợp khi đầu kia là một vùng đệm liên tục trong bộ nhớ.

Phần mềm cần biết khi nào lần truyền kết thúc. Có hai cách phổ biến: đọc thăm dò (polling), trong đó CPU đọc lặp một thanh ghi trạng thái cho đến khi thấy cờ báo hoàn tất; và ngắt (interrupt), trong đó bộ điều khiển chủ động phát tín hiệu báo cho CPU khi hoàn tất, giúp CPU không phải chờ. Hình 2.6 đối chiếu luồng dữ liệu khi sao chép bằng CPU và khi sao chép bằng DMA.

So sánh luồng sao chép dữ liệu



Hình 2.6: So sánh luồng sao chép dữ liệu.

2.8. Vào/ra ánh xạ bộ nhớ (MMIO) và thiết bị ngoại vi

Vào/ra ánh xạ bộ nhớ (Memory-Mapped I/O – MMIO) là phương thức điều khiển thiết bị ngoại vi bằng cách ánh xạ các thanh ghi điều khiển và dữ liệu của thiết bị vào cùng không gian địa chỉ với bộ nhớ [11]. Khi đó, bộ xử lý không cần một tập lệnh vào/ra riêng: việc đọc hoặc ghi tới một địa chỉ thuộc vùng của thiết bị sẽ tác động trực tiếp lên thanh ghi tương ứng của thiết bị, sử dụng chính các lệnh nạp/lưu thông thường. Cách tiếp cận này đơn giản hóa kiến trúc tập lệnh và thống nhất cơ chế truy cập bộ nhớ với truy cập thiết bị.

Một đặc điểm quan trọng của vùng MMIO là các truy cập tới nó thường có hiệu ứng phụ (side effect): chẳng hạn, đọc một thanh ghi dữ liệu có thể lấy đi một ký tự khỏi bộ đệm nhận của thiết bị, hoặc ghi một thanh ghi có thể kích hoạt một hành động. Vì vậy, các vùng MMIO phải được đánh dấu không lưu đệm (mục 2.4, 2.5) để mọi thao tác đọc/ghi đều tác động trực tiếp lên thiết bị thay vì lên một bản sao trong

bộ nhớ đệm. Việc giao tiếp với thiết bị qua MMIO cũng có thể theo kiểu đọc thăm dò hoặc theo kiểu hướng ngắt, tương tự như đã trình bày ở mục 2.7.

Ở mức khái niệm, hệ thống của đề tài có bốn thiết bị ngoại vi, mỗi thiết bị chiếm một vùng địa chỉ MMIO riêng:

- Bộ thu phát không đồng bộ đa năng (Universal Asynchronous Receiver/Transmitter – UART): thiết bị giao tiếp nối tiếp, dùng làm cổng console để truyền và nhận dữ liệu dạng ký tự, có thể cấu hình tốc độ truyền (baud).
- Ma trận đèn (Light-Emitting Diode – LED): thiết bị hiển thị dạng lưới điểm ảnh, trạng thái các điểm ảnh được lưu trong một vùng bộ nhớ hiển thị (video memory) được ánh xạ vào không gian địa chỉ.
- Bàn phím (keyboard): thiết bị nhập, dùng một bộ đệm để lưu các ký tự gõ vào theo thứ tự, phần mềm đọc lần lượt qua thanh ghi dữ liệu.
- Chuột (mouse): thiết bị nhập, cung cấp tọa độ con trỏ và trạng thái nút nhấn qua các thanh ghi tương ứng.

2.9. Công nghệ nền tảng web

Hệ thống được hiện thực dưới dạng một ứng dụng web tĩnh nhằm bảo đảm khả năng tiếp cận không cần cài đặt, như đã nêu trong mục tiêu ở Chương 1. Phần này giới thiệu ngắn gọn các công nghệ nền tảng được sử dụng, ở mức đủ để hiểu các lựa chọn hiện thực.

HTML, CSS và JavaScript - Ba công nghệ này là nền tảng của ứng dụng web phía trình duyệt [13]: HTML (HyperText Markup Language) mô tả cấu trúc nội dung; CSS (Cascading Style Sheets) quy định trình bày và bố cục; còn JavaScript là ngôn ngữ lập trình thực thi logic trong trình duyệt. JavaScript chạy theo mô hình đơn luồng dựa trên vòng lặp sự kiện (event loop): các tác vụ được xử lý tuần tự trên một luồng chính, do đó việc cập nhật giao diện theo từng bước mô phỏng cần được tổ chức hợp lý để không chặn luồng này. Để cập nhật hiển thị mượt mà theo khung hình, trình

duyet cung cap co che dang ky ve lai dong bo voi chu ky lam tuoi man hinh (requestAnimationFrame).

Phiên bản hiện đại của JavaScript hỗ trợ cơ chế mô-đun chuẩn hóa (ECMAScript modules), cho phép tách mã nguồn thành nhiều tệp độc lập và khai báo quan hệ phụ thuộc qua các câu lệnh nhập/xuất (import/export) [13]. Cơ chế này hỗ trợ tổ chức mã theo hướng mô-đun, trong đó mỗi đơn vị chức năng được đóng gói riêng.

Phần tử canvas cho phép vẽ đồ họa điểm ảnh hai chiều bằng JavaScript [13]. Phần tử này phù hợp để hiển thị các nội dung thay đổi liên tục theo trạng thái, chẳng hạn một lưới điểm ảnh được cập nhật theo nội dung vùng bộ nhớ hiển thị.

Thư viện soạn thảo CodeMirror là một thư viện soạn thảo mã nguồn chạy trong trình duyệt, cung cấp các tính năng như tô màu cú pháp theo ngôn ngữ, hiển thị số dòng và hỗ trợ gợi ý khi nhập [14]. Thư viện này thường được dùng để xây dựng vùng nhập mã trong các công cụ lập trình trên nền web.

Chương 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

Hệ thống được xác định là một công cụ phục vụ học tập và minh họa, có đối tượng sử dụng chính là người học kiến trúc máy tính ở mức hệ thống. Yêu cầu của hệ thống được phân thành hai nhóm: yêu cầu chức năng (mô tả những việc hệ thống phải làm được) và yêu cầu phi chức năng (mô tả các thuộc tính chất lượng mà hệ thống cần đạt). Các yêu cầu được rút ra từ mục tiêu đề tài (mục 1.2), từ mô tả chức năng trong tài liệu dự án và từ các thành phần đã hiện thực trong mã nguồn.

3.1.1. Yêu cầu chức năng

Nhóm yêu cầu chức năng bao quát toàn bộ vòng đời thao tác của người học với một chương trình hợp ngữ: soạn thảo, biên dịch, nạp, thực thi (chạy liên tục hoặc theo từng bước), và quan sát trạng thái của các thành phần phần cứng được mô phỏng. Bảng 3.1 liệt kê các yêu cầu chức năng chính kèm mô tả ngắn và tệp nguồn hiện thực tương ứng; chi tiết cách mỗi chức năng được hiện thực được trình bày ở Chương 4.

Bảng 3.1. Danh sách yêu cầu chức năng của hệ thống

Mã	Yêu cầu chức năng	Mô tả ngắn
FR1	Soạn thảo chương trình	Nhập mã hợp ngữ RISC-V trong trình soạn thảo có tô màu cú pháp và gợi ý nhập
FR2	Biên dịch (assemble)	Biên dịch mã hợp ngữ sang mã máy, báo lỗi theo từng dòng
FR3	Nạp chương trình	Ghi mã máy và dữ liệu vào bộ nhớ, đặt bộ đếm chương trình về địa chỉ bắt đầu
FR4	Chạy liên tục	Thực thi liên tục với điều khiển Run/Pause/Stop
FR5	Chạy từng bước	Thực thi từng lệnh một (Step) để quan sát thay đổi trạng thái
FR6	Đặt điểm dừng	Đặt điểm dừng (breakpoint) theo dòng lệnh; Run dừng khi tới điểm dừng
FR7	Điều chỉnh tốc độ	Điều chỉnh số chu kỳ mỗi khung hình và hiển thị tốc độ thực thi ước lượng (IPS)

Mã	Yêu cầu chức năng	Mô tả ngắn
FR8	Quan sát thanh ghi	Hiển thị tệp thanh ghi số nguyên (x0–x31) và dấu phẩy động (f0–f31)
FR9	Quan sát bộ nhớ	Hiển thị vùng mã lệnh (disassembly) và vùng dữ liệu (hex/ASCII)
FR10	Quan sát MMU	Hiển thị cấu hình, bảng trang, bộ đệm dịch địa chỉ (TLB) và lịch sử dịch địa chỉ
FR11	Quan sát bộ nhớ đệm	Hiển thị trạng thái và thống kê trùng/trượt của L1I, L1D, L2
FR12	Quan sát sơ đồ SoC	Hiển thị sơ đồ khối động, làm nổi bật các giao dịch trên bus theo thời gian thực
FR13	Tương tác ngoại vi	Tương tác với UART (console), ma trận LED, bàn phím và chuột
FR14	Nhật ký hệ thống	Ghi và lọc nhật ký hệ thống theo từng mô-đun (CPU, MMU, cache, bus, DMA...)
FR15	Khởi tạo lại hệ thống	Đặt lại trạng thái mô phỏng (Reset) mà vẫn giữ mã nguồn trong trình soạn thảo

Các yêu cầu FR8–FR12 và FR14 phản ánh đặc thù của một công cụ giáo dục: ngoài việc thực thi đúng chương trình, hệ thống còn phải làm cho các trạng thái nội bộ vốn khó quan sát trên phần cứng thật — nội dung bộ nhớ đệm, các mục trong TLB, các giao dịch trên bus — trở nên hiển thị được và theo dõi được theo từng bước.

3.1.2. Yêu cầu phi chức năng

Bên cạnh các chức năng, hệ thống cần đáp ứng một số thuộc tính chất lượng sau:

- Khả năng tiếp cận không cần cài đặt: Hệ thống phải chạy được trực tiếp trên trình duyệt web hiện đại, không yêu cầu cài đặt môi trường hay biên dịch trước. Đây là một mục tiêu đã nêu ở Chương 1 và định hình lựa chọn nền tảng web tĩnh.
- Tính trực quan: Trạng thái của các thành phần phải được trình bày một cách trực quan, có cấu trúc và có làm nổi bật thay đổi (ví dụ tô sáng thanh ghi vừa

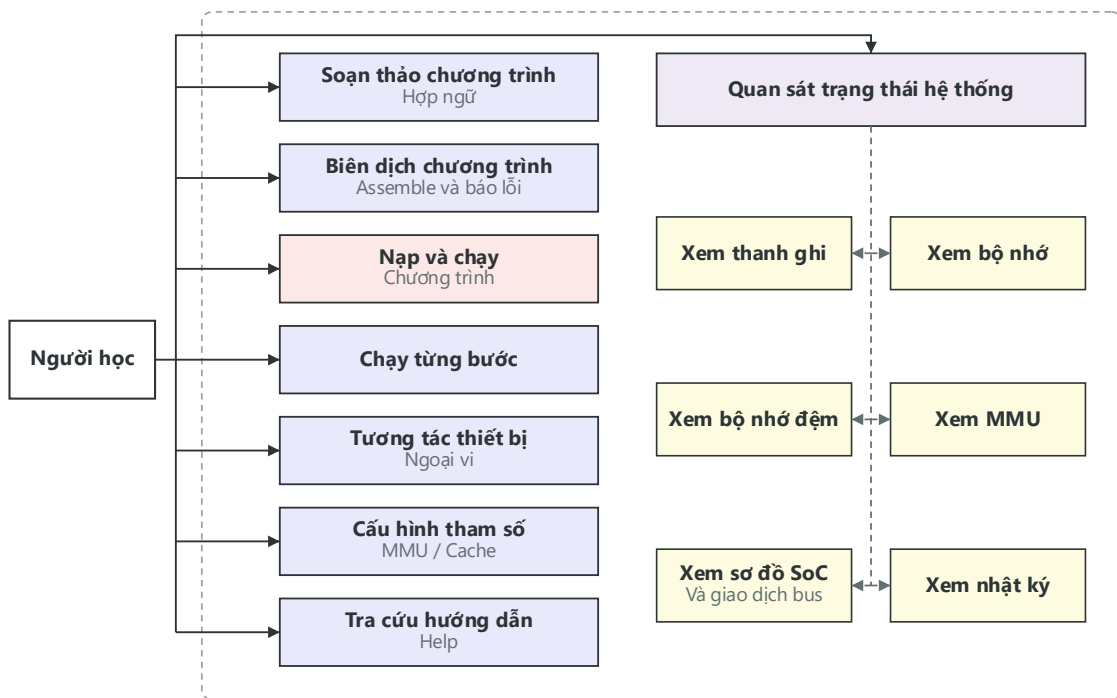
thay đổi, làm nổi bật đường bus đang có giao dịch), nhằm hỗ trợ việc quan sát và suy luận của người học.

- Hiệu năng đủ cho tương tác: Tốc độ mô phỏng phải đủ để thao tác mượt mà ở chế độ tương tác. Vòng lặp chạy được tổ chức theo cơ chế vẽ lại theo khung hình của trình duyệt, cho phép điều chỉnh số chu kỳ thực thi trên mỗi khung hình và hiển thị tốc độ thực thi ước lượng. Hệ thống mô phỏng ở mức hành vi nên không nhằm đạt tần số của một chip thật (xem mục 1.3.2).
- Kiến trúc mã mô-đun và khả năng kiểm thử: Mã nguồn được tổ chức theo mô-đun ES, trong đó mỗi thành phần phần cứng là một mô-đun JavaScript độc lập và được lắp ghép tại một thành phần khởi tạo trung tâm. Việc các mô-đun được tách rời qua lớp trừu tượng cổng kết nối (mục 3.3.2) giúp mỗi mô-đun có thể được khởi tạo và kiểm thử một cách độc lập, kể cả trong môi trường không có trình duyệt: nhiều mô-đun có thể được nạp và chạy kiểm thử ở chế độ dòng lệnh bằng Node.js. Đây là một định hướng thiết kế hướng tới khả năng kiểm thử; quy trình và kết quả kiểm thử cụ thể được trình bày ở Chương 5.

3.1.3. Mô hình use case tổng quát

Hệ thống có người dùng chính là người học - người sử dụng công cụ để viết, chạy và quan sát chương trình. Vì là một ứng dụng chạy hoàn toàn phía trình duyệt, hệ thống không có đối tượng ngoài (hệ thống bên thứ ba, dịch vụ mạng) trong luồng sử dụng chính. Các trường hợp sử dụng (use case) được nhóm theo vòng đời thao tác đã nêu ở mục 3.1.1.

Mô hình use case cho thấy hệ thống được thiết kế xoay quanh một chu trình lặp: người học soạn thảo, biên dịch, chạy, quan sát kết quả, rồi quay lại chỉnh sửa. Toàn bộ thiết kế kiến trúc và giao diện ở các mục tiếp theo phục vụ cho chu trình này.



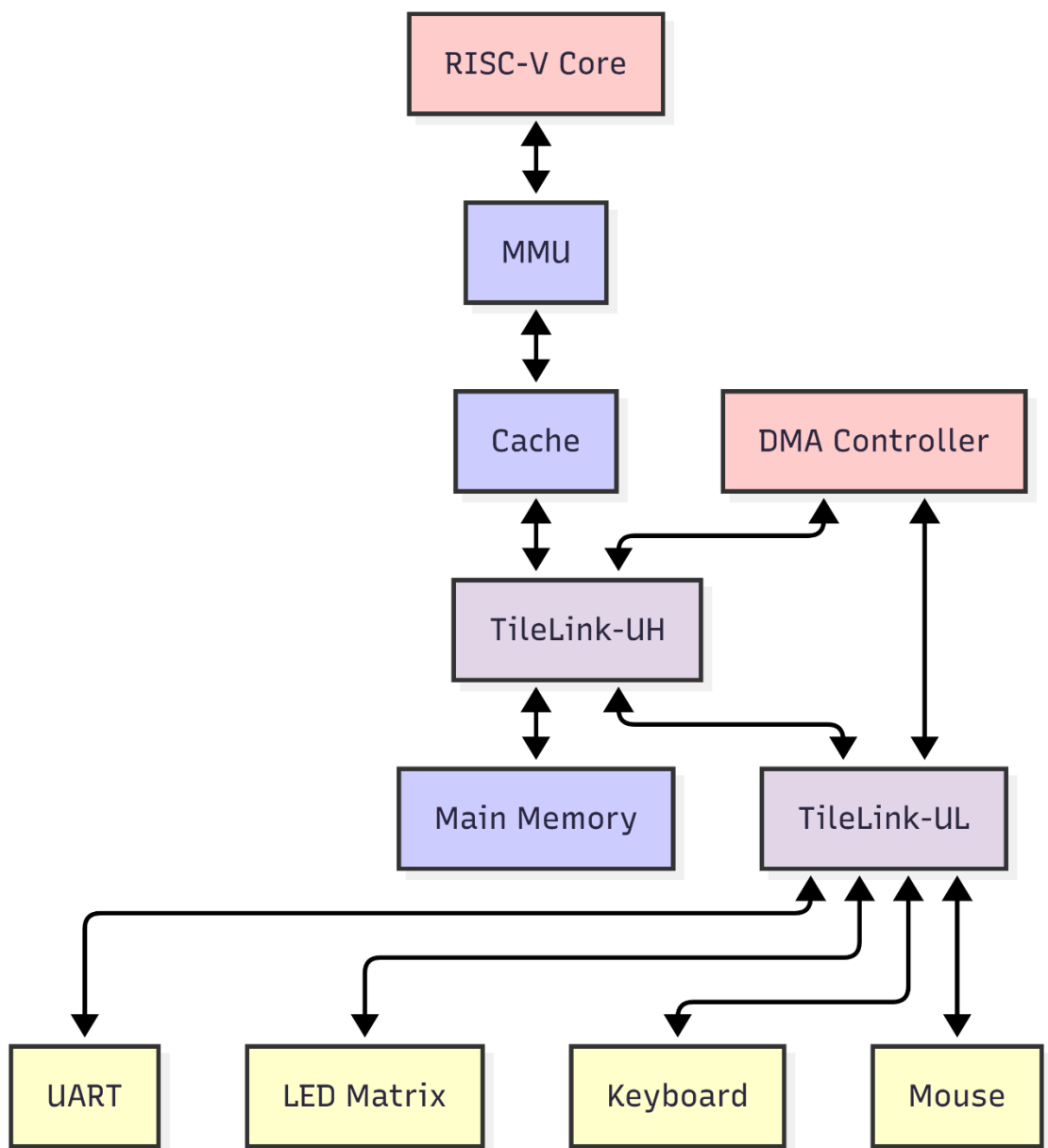
Hình 3.1: Sơ đồ use case tổng quát.

3.2. Kiến trúc tổng quan hệ thống

3.2.1. Sơ đồ kiến trúc tổng quan

Về tổng thể, hệ thống tái hiện một SoC RISC-V điển hình theo đúng năm nhóm thành phần đã nêu ở mục 2.1.2: lõi xử lý, phân cấp bộ nhớ, hệ thống kết nối, bộ điều khiển DMA và các thiết bị ngoại vi. Các thành phần này được khởi tạo và nối với nhau tại thành phần khởi tạo trung tâm của trình mô phỏng, trong khi mô tả trực quan của chúng được định nghĩa độc lập cho sơ đồ động.

Đường truy cập bộ nhớ chính đi theo một chuỗi từ lõi xử lý xuống bộ nhớ chính: lõi xử lý phát yêu cầu qua đơn vị quản lý bộ nhớ; đơn vị này dịch địa chỉ rồi chuyển tiếp yêu cầu tới bộ nhớ đệm cấp một (tách riêng phần lệnh và phần dữ liệu); khi trượt, yêu cầu đi tiếp xuống bộ nhớ đệm cấp hai dùng chung; khi vẫn trượt, yêu cầu đi ra bus hiệu năng cao TileLink-UH để tới bộ nhớ chính. Song song với đường này, bộ điều khiển DMA và cầu nối bus nằm bên cạnh lõi xử lý và cùng tham gia vào hệ thống kết nối. Hình 3.2 minh họa kiến trúc tổng quan này.



Hình 3.2: Kiến trúc tổng quan của trình mô phỏng SoC

Có hai đặc điểm thiết kế cần nêu rõ để tránh hiểu nhầm về phạm vi.

Thứ nhất, về vai trò của bus TileLink-UH. Trong nhãn hiển thị của giao diện, bus này được gọi là "coherent bus (high-speed)"; tuy nhiên, đúng như đã phân tích ở mục 2.6.3, hệ thống chỉ hiện thực mô hình giao dịch trên hai kênh A và D, không hiện thực các kênh B/C/E cùng cơ chế bảo đảm nhất quán bộ nhớ đệm. Vì vậy, trong báo cáo này, TileLink-UH được hiểu là bus hiệu năng cao dành cho truy cập bộ nhớ khối lượng lớn (cho phép truyền chuỗi và thao tác nguyên tử), còn TileLink-UL là bus nhẹ

dành cho các thiết bị ngoại vi đơn giản; cả hai đều ở mức giao dịch, không phải bus có nhất quán bộ nhớ đệm theo nghĩa đầy đủ của TileLink-C [2].

Thứ hai, về tính "đa master". Hệ thống có hai thành phần cùng đóng vai trò master phát yêu cầu trên bus là lõi xử lý (thông qua đường L2 → TileLink-UH) và bộ điều khiển DMA. Đây là cơ sở để minh họa việc nhiều thành phần cùng truy cập tài nguyên chung. Tuy nhiên, cơ chế điều phối hiện được mô hình hóa ở mức hàng đợi yêu cầu với một giao dịch được xử lý tại một thời điểm, chưa phải là một cơ chế phân xử (arbitration) đa master đầy đủ (xem mục 2.6.3). Hệ thống cũng chưa có đường tín hiệu ngắt (IRQ) tới lõi xử lý; trạng thái hoàn tất của DMA và trạng thái thiết bị ngoại vi được nhận biết qua đọc thăm dò (xem mục 1.3.2).

3.2.2. Danh sách mô-đun và vai trò

Mỗi khối phần cứng trong sơ đồ kiến trúc được hiện thực bằng một hoặc một nhóm mô-đun phần mềm. Bảng 3.2 liệt kê các thành phần thiết kế, vai trò mô phỏng, tệp hiện thực và ghi chú về cổng kết nối. Bảng được trình bày ở mức vai trò và giao diện kết nối; chi tiết nội bộ của từng mô-đun được trình bày ở Chương 4.

Bảng 3.2. Danh sách mô-đun của hệ thống và vai trò mô phỏng

Thành phần thiết kế	Vai trò mô phỏng	Ghi chú (cổng/kết nối)
Lõi xử lý (RV32IMF)	Thực thi lệnh theo chu trình nạp–giải mã–thực thi; tệp thanh ghi số nguyên và dấu phẩy động; phát yêu cầu truy cập bộ nhớ	Master chính; có cổng trên (upper) và cổng dưới (lower) nối xuống MMU; truy cập bộ nhớ theo mô hình bất đồng bộ
Đơn vị quản lý bộ nhớ (MMU)	Dịch địa chỉ ảo sang vật lý, TLB, kiểm tra quyền, xác định vùng có thể lưu đệm	Nằm trên đường CPU → bộ nhớ đệm; có cổng lệnh và cổng dữ liệu riêng; không phải tầng được cấp xung riêng
Bộ nhớ đệm L1I / L1D	Bộ nhớ đệm cấp một, tách riêng phần lệnh và phần dữ liệu	Nối lên MMU, nối xuống L2; bỏ qua đệm với vùng không lưu đệm
Bộ nhớ đệm L2	Bộ nhớ đệm cấp hai dùng chung, phục vụ trượt của L1	Nhận từ L1I/L1D, nối xuống TileLink-UH

Thành phần thiết kế	Vai trò mô phỏng	Ghi chú (cổng/kết nối)
Bộ nhớ chính (RAM)	Lưu mã lệnh và dữ liệu; mô hình độ trễ và đáp ứng theo beat	Slave trên TileLink-UH cho mọi địa chỉ không thuộc MMIO
Lỗi giao thức TileLink	Định nghĩa mã thao tác kênh A/D, mặt nạ byte, thao tác nguyên tử, ảnh chụp giao dịch	Thư viện dùng chung cho các bus và endpoint
Hạ tầng interconnect	Hàng đợi yêu cầu/phản hồi, định tuyến master/slave theo địa chỉ	Lớp cơ sở của hai bus; một giao dịch in-flight
Bus TileLink-UH	Bus hiệu năng cao cho RAM, thanh ghi DMA và cầu nối; cho phép đủ tập thao tác (kể cả nguyên tử)	Nối L2, RAM, DMA (master + thanh ghi), cầu nối UH→UL
Bus TileLink-UL	Bus nhẹ cho thiết bị ngoại vi; cho phép đọc/ghi và lệnh gọi ý	Nối UART, LED, bàn phím, chuột, DMA (master), cầu nối UL→UH
Cầu nối UH↔UL	Chuyển giao dịch giữa bus hiệu năng cao và bus ngoại vi	Hai cầu nối một chiều; slave trên bus này, truy cập trực tiếp sang bus kia
Lớp cổng kết nối (Port)	Trừu tượng hóa việc nối hai mô-đun hoặc đăng ký endpoint vào bus	Cung cấp các loại cổng link/upper/lower/memory
Bộ điều khiển DMA	Truyền khối dữ liệu độc lập với CPU theo mô tả truyền	Master trên cả UH và UL; thanh ghi điều khiển là slave trên UH (0xFFED0000)
UART	Giao tiếp nối tiếp dạng console, có cấu hình tốc độ	Endpoint MMIO không lưu đệm trên UL (0x10000000)
Ma trận LED	Hiển thị lưới 32×32 điểm ảnh từ vùng bộ nhớ hiển thị	Endpoint MMIO trên UL (0xFF000000); vẽ bằng canvas
Bàn phím	Bộ đệm ký tự nhập vào để phần mềm đọc thăm dò	Endpoint MMIO trên UL (0xFFFF0000)
Chuột	Cung cấp tọa độ con trỏ và trạng thái nút qua thanh ghi	Endpoint MMIO trên UL (0xFF100000)
Sơ đồ SoC động	Vẽ sơ đồ khối SVG và làm nổi bật giao dịch bus	Đọc dấu vết giao dịch từ thành phần khởi tạo trung tâm

Thành phần thiết kế	Vai trò mô phỏng	Ghi chú (cổng/kết nối)
Nhật ký hệ thống	Bắt, phân loại và lọc nhật ký theo từng mô-đun	Hỗ trợ quan sát và gỡ lỗi
Trình biên dịch hợp ngữ	Biên dịch hai lượt mã hợp ngữ sang mã máy	Sinh dữ liệu nạp vào bộ nhớ
Vỏ giao diện web	Bố cục, thanh bên, các view, bảng thanh ghi/bộ nhớ, vùng I/O, log	Khung trình bày
Vòng điều khiển và cập nhật giao diện	Điều phối assemble/run/step/reset, vẽ lại trạng thái, đo IPS	Lớp kết nối giao diện với lõi mô phỏng
Thành phần khởi tạo trung tâm	Khởi tạo mô-đun, lập bản đồ địa chỉ, nối cổng, điều phối vòng tick	Nơi lắp ghép toàn hệ thống

3.2.3. Bản đồ địa chỉ bộ nhớ

Các thành phần của hệ thống chia sẻ một không gian địa chỉ phẳng 32 bit. Việc một địa chỉ thuộc về bộ nhớ chính hay thuộc về một thiết bị ngoại vi, và việc một truy cập đi qua bus nào, được quyết định bởi bản đồ địa chỉ cấu hình tại thành phần khởi tạo trung tâm. Bản đồ này cũng xác định thuộc tính có thể lưu đệm của từng vùng: các vùng thuộc thiết bị ngoại vi và vùng thanh ghi DMA được đánh dấu không lưu đệm để mọi truy cập tác động trực tiếp lên thiết bị, phù hợp với đặc tính của vùng MMIO (xem mục 2.8); mọi địa chỉ còn lại được coi là bộ nhớ chính và có thể lưu đệm. Bảng 3.3 tổng hợp bản đồ địa chỉ.

Trong vùng bộ nhớ chính, trình biên dịch hợp ngữ sử dụng hai địa chỉ nền quy ước để nạp chương trình: vùng mã lệnh (.text) bắt đầu tại 0x00400000 và vùng dữ liệu (.data) bắt đầu tại 0x10010000. Cả hai vùng này đều là địa chỉ bộ nhớ chính, do đó được truy cập qua TileLink-UH và có thể lưu đệm. Có thể thấy vùng dữ liệu 0x10010000 nằm cao hơn dải của UART (0x10000000–0x10000013) nên không xảy

ra chồng lấn giữa vùng dữ liệu chương trình và vùng MMIO. Bản đồ địa chỉ này là cơ sở để hệ thống định tuyến mỗi yêu cầu tới đúng thành phần đích; cơ chế định tuyến cụ thể được trình bày ở mục 3.3.2 và Chương 4.

Bảng 3.3. Bản đồ địa chỉ bộ nhớ của hệ thống

Vùng / thiết bị	Địa chỉ nền (base)	Kích thước	Dải địa chỉ	Bus	Có thể lưu đệm
Bộ nhớ chính (RAM)	(mọi địa chỉ không thuộc MMIO)	—	Phần còn lại của không gian địa chỉ	TileLink-UH	Có
UART	0x10000000	20 byte (0x14)	0x10000000–0x10000013	TileLink-UL	Không
DMA (thanh ghi điều khiển)	0xFFED0000	8 byte (0x08)	0xFFED0000–0xFFED0007	TileLink-UH	Không
LED Matrix (VRAM)	0xFF000000	4096 byte (32×32×4)	0xFF000000–0xFF000FFF	TileLink-UL	Không
Mouse	0xFF100000	20 byte (0x14)	0xFF100000–0xFF100013	TileLink-UL	Không
Keyboard	0xFFFF0000	8 byte (0x08)	0xFFFF0000–0xFFFF0007	TileLink-UL	Không

3.3. Mô hình mô phỏng theo chu kỳ và trừu tượng kết nối

3.3.1. Mô hình tick theo chu kỳ

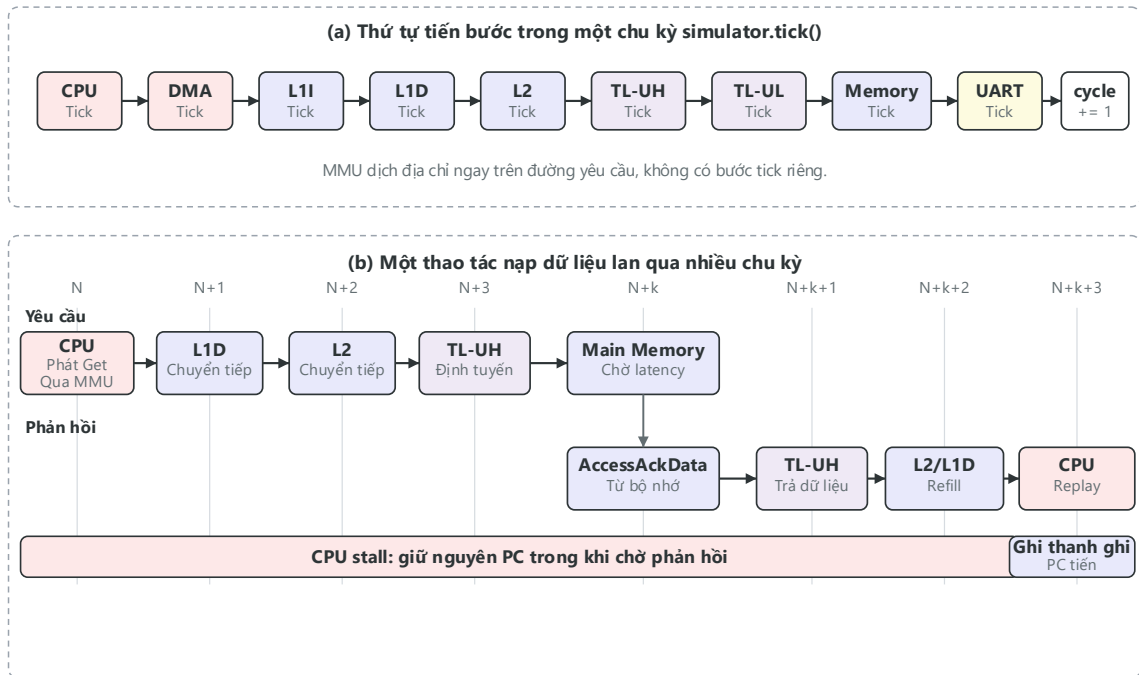
Toàn hệ thống tiến triển theo các bước chu kỳ rời rạc. Mỗi lần gọi thao tác tiến một bước (tick) tại thành phần khởi tạo trung tâm tương ứng với một chu kỳ mô phỏng và làm tăng bộ đếm chu kỳ lên một. Trong mỗi chu kỳ, các mô-đun được cho tiến một bước theo một thứ tự cố định từ phía phát yêu cầu xuống phía phục vụ yêu cầu: lõi xử lý, rồi bộ điều khiển DMA, rồi bộ nhớ đệm L1I, L1D, rồi bộ nhớ đệm L2, rồi bus TileLink-UH, bus TileLink-UL, rồi bộ nhớ chính, và cuối cùng là UART. Đơn vị quản lý bộ nhớ không được cấp một bước tiến riêng trong vòng này: nó thực hiện

việc dịch địa chỉ ngay trên đường yêu cầu đi từ lõi xử lý xuống bộ nhớ đệm (mục 3.3.2), chứ không phải là một tầng được cấp xung độc lập.

Việc cố định thứ tự tiến bước theo chiều truyền yêu cầu nhằm mục đích để mỗi chặng (hop) trên đường đi của một yêu cầu tiêu tốn đúng một chu kỳ, qua đó phản ánh tương quan chi phí giữa các chặng một cách nhất quán. Để biểu diễn tương quan chi phí giữa các loại truy cập, hệ thống còn gán các giá trị độ trễ theo số chu kỳ cho từng thao tác — ví dụ truy cập trúng/trượt ở mỗi cấp bộ nhớ đệm và độ trễ của bộ nhớ chính — như đã nêu ở mục 2.5; các tham số cụ thể được trình bày ở Chương 4. Cần nhắc lại rằng đây là mô hình ở mức hành vi, không phải mô phỏng chính xác theo chu kỳ của vi kiến trúc (xem mục 2.2).

Một hệ quả quan trọng của mô hình này là lõi xử lý truy cập bộ nhớ theo cách bất đồng bộ: khi cần nạp lệnh hoặc thực hiện một lệnh nạp/lưu, lõi xử lý phát một yêu cầu xuống cổng dưới rồi tạm dừng, giữ nguyên bộ đếm chương trình, cho đến khi phản hồi tương ứng quay về ở một chu kỳ sau; khi đó lệnh được "thực hiện lại" để hoàn tất và bộ đếm chương trình mới tiến tới lệnh kế tiếp. Nhờ vậy, một lệnh có truy cập bộ nhớ có thể trải dài qua nhiều chu kỳ tương ứng với thời gian dữ liệu di chuyển qua các tầng. Chi tiết máy trạng thái truy cập bộ nhớ của lõi xử lý được trình bày ở Chương 4.

Thao tác khởi tạo lại (reset) đặt bộ đếm chu kỳ về 0 và khôi phục trạng thái ban đầu cho toàn bộ mô-đun và thiết bị ngoại vi, được thực hiện bằng cách khởi tạo lại thành phần trung tâm. Ngoài chế độ chạy liên tục, hệ thống còn hỗ trợ chạy theo từng lệnh: thao tác bước-một-lệnh lặp việc tiến chu kỳ cho đến khi số lệnh hoàn tất tăng thêm một, tức là tới khi đúng một lệnh được hoàn thành (kể cả khi lệnh đó phải chờ qua nhiều chu kỳ truy cập bộ nhớ).



Hình 3.3: Trình tự tiến bước trong một chu kỳ và sự lan truyền yêu cầu qua nhiều chu kỳ.

3.3.2. Trừu tượng cổng kết nối (Port)

Để các mô-đun không phụ thuộc trực tiếp vào nhau, hệ thống đưa vào một lớp trừu tượng trung gian gọi là cổng kết nối (Port). Một đối tượng cổng đóng vai trò trung gian: hoặc nối trực tiếp hai mô-đun theo kiểu điểm–điểm, hoặc đăng ký một mô-đun như một endpoint vào một bus/interconnect. Nhờ lớp này, mỗi mô-đun chỉ cần cung cấp một giao diện thống nhất gồm các thao tác gửi/nhận yêu cầu và gửi/nhận phản hồi, mà không cần biết phía bên kia là mô-đun cụ thể nào hay được đấu nối ra sao.

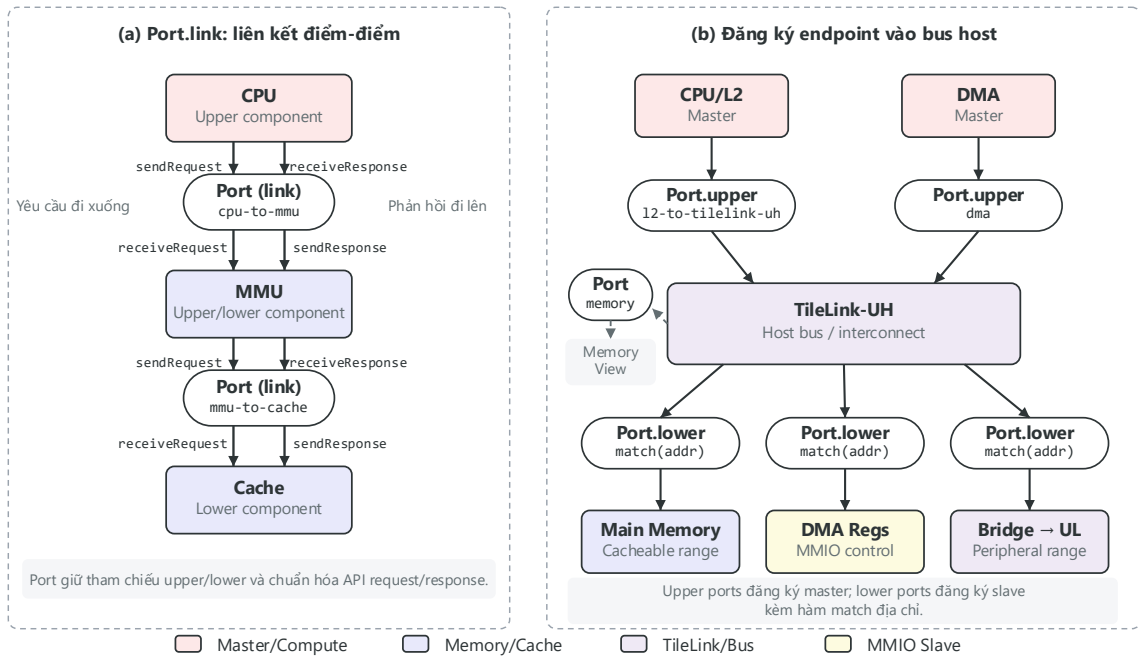
Lớp cổng định nghĩa bốn loại cổng tương ứng với bốn tình huống đầu nối:

- Cổng liên kết (link) nối trực tiếp hai mô-đun theo kiểu điểm–điểm. Khi gắn, cổng yêu cầu mô-đun phía trên (upper) và mô-đun phía dưới (lower) cùng đăng ký tham chiếu tới nhau qua cổng này. Đây là loại cổng dùng cho chuỗi lỗi xử lý $\rightarrow$ MMU $\rightarrow$ bộ nhớ đệm.

- Cổng phía trên (upper) đăng ký một thành phần chủ động (master/requester) vào một bus host. Lỗi xử lý và bộ điều khiển DMA được đưa lên bus theo loại cổng này.
- Cổng phía dưới (lower) đăng ký một thành phần bị động (slave/target) vào bus host, kèm một hàm phán đoán địa chỉ (match) để bus quyết định những địa chỉ nào sẽ được định tuyến tới thành phần này. Bộ nhớ chính, các thanh ghi DMA, các endpoint ngoại vi và cầu nối đều được đăng ký theo loại cổng này, mỗi loại kèm điều kiện địa chỉ tương ứng từ bản đồ địa chỉ ở mục 3.2.3.
- Cổng bộ nhớ (memory) cung cấp một "khung nhìn bộ nhớ" để một số thao tác truy cập trực tiếp hoặc gỡ lỗi có thể nhìn thấy bộ nhớ phía dưới.

Trên giao diện thống nhất đó, cổng chuyển tiếp một yêu cầu từ phía trên xuống phía dưới (thao tác gửi yêu cầu được ánh xạ thành thao tác nhận yêu cầu của mô-đun phía dưới) và chuyển tiếp một phản hồi từ phía dưới ngược lên phía trên. Cổng được thiết kế để dung hòa hai quy ước đặt tên thao tác khác nhau có thể tồn tại giữa các mô-đun (một số mô-đun trả phản hồi bằng thao tác "gửi phản hồi", số khác bằng "nhận phản hồi"), nhờ đó các mô-đun viết độc lập vẫn ghép nối được với nhau. Ngoài ra, các hàm tiện ích đọc/ghi trực tiếp được chuyển tiếp qua cổng để phục vụ các endpoint MMIO và bộ nhớ.

Việc lắp ghép toàn hệ thống được thực hiện tập trung bằng các hàm trợ giúp gắn cổng: một hàm tổng quát để nối hai mô-đun hoặc gắn một mô tả cổng vào bus, cùng hai hàm chuyên biệt để gắn riêng cổng lệnh và cổng dữ liệu cho đơn vị quản lý bộ nhớ (phản ánh việc MMU có hai đường tách biệt tới L1I và L1D). Nhờ tách bạch giữa "định nghĩa mô-đun" và "cách đấu nối", thành phần khởi tạo trung tâm có thể mô tả toàn bộ tô-pô kết nối của hệ thống một cách tường minh tại một nơi.



Hình 3.4: Mô hình cổng kết nối giữa các mô-đun.

3.4. Thiết kế giao diện web

Giao diện web là nơi người học tương tác với toàn bộ hệ thống. Thiết kế giao diện hướng tới hai mục tiêu đã nêu trong yêu cầu phi chức năng: cho phép thao tác trọn vẹn chu trình soạn thảo - biên dịch - chạy - quan sát, và trình bày trạng thái nội bộ của hệ thống một cách trực quan, có cấu trúc.

3.4.1. Bố cục và điều hướng

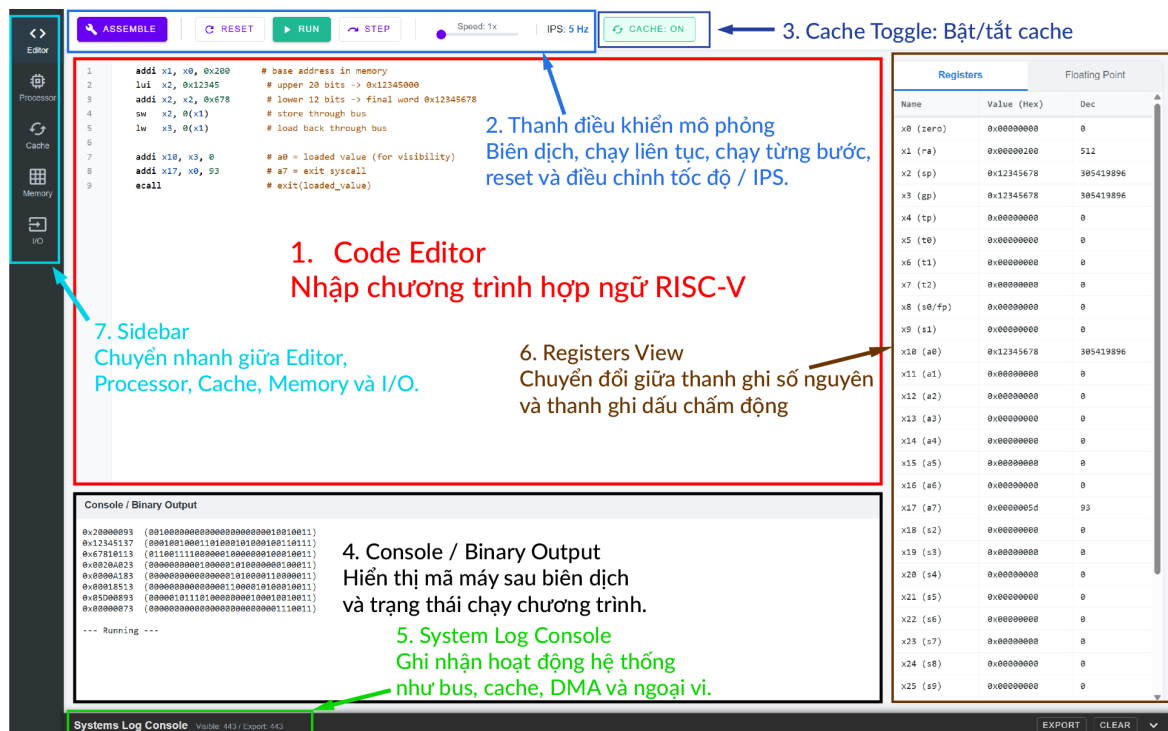
Giao diện được tổ chức thành ba khu vực: một thanh bên dọc bên trái để điều hướng giữa các khung nhìn (view), một thanh công cụ ngang phía trên để điều khiển thực thi, và một vùng nội dung chính ở giữa hiển thị khung nhìn đang được chọn.

Thanh bên gồm bảy mục điều hướng, mỗi mục tương ứng một khung nhìn: Editor (soạn thảo và thanh ghi), SoC (sơ đồ khối hệ thống), MMU (đơn vị quản lý bộ nhớ), Cache (bộ nhớ đệm), Memory (bộ nhớ lệnh và dữ liệu), I/O (thiết bị ngoại vi) và Help (hướng dẫn). Tại mỗi thời điểm chỉ một khung nhìn được hiển thị; việc chọn một mục trên thanh bên sẽ chuyển khung nhìn tương ứng. Bảng 3.4 tóm tắt vai trò của bảy khung nhìn.

Bảng 3.4. Bảy khung nhìn của giao diện và nội dung chính

Khung nhìn	Nội dung chính	Phục vụ yêu cầu
Editor	Trình soạn thảo mã hợp ngữ; bảng kết quả mã máy (console/binary output); bảng thanh ghi số nguyên và dấu phẩy động	FR1, FR2, FR8
SoC	Sơ đồ khối động của toàn hệ thống; làm nổi bật đường bus đang có giao dịch; chú giải và tooltip	FR12
MMU	Tổng quan và cấu hình; bảng trang phần mềm; bảng TLB; lịch sử dịch địa chỉ gần đây	FR10
Cache	Trạng thái L1I, L1D, L2 (tập, đường, hợp lệ, bẩn, thê); thống kê trùng/trượt	FR11
Memory	Bộ nhớ lệnh (địa chỉ, mã máy, disassembly, cột đặt điểm dừng); vùng dữ liệu theo hàng, chuyển đổi hex/ASCII	FR6, FR9
I/O	Ma trận LED (canvas); console UART (xuất/nhập); ô nhập bàn phím và trạng thái bộ đệm	FR13
Help	Tra cứu lệnh, pseudo-instruction, directive, syscall, bản đồ địa chỉ và hướng dẫn quy trình chạy	(hỗ trợ FR1–FR5)

Thanh công cụ phía trên chứa các nút điều khiển thực thi theo đúng vòng đời thao tác: Assemble (biên dịch và nạp), Reset (khởi tạo lại), Run (chạy liên tục), Pause (tạm dừng/tiếp tục), Stop (dừng hẳn) và Step (chạy từng bước). Bên cạnh đó là một thanh trượt điều chỉnh tốc độ (Speed) cho phép thay đổi số chu kỳ thực thi trên mỗi khung hình, và một chỉ số hiển thị tốc độ thực thi ước lượng (IPS — số lệnh/chu kỳ trên giây) cập nhật trong khi chạy. Trạng thái khả dụng của các nút thay đổi theo trạng thái chạy: ví dụ Pause và Stop chỉ được kích hoạt khi chương trình đang chạy, còn Step bị vô hiệu hóa khi đang chạy liên tục mà chưa tạm dừng.



Hình 3.5: Giao diện web tổng quan.

3.4.2. Các khung quan sát trạng thái

Ngoài khu vực soạn thảo và điều khiển, phần lớn giao diện dành cho việc quan sát trạng thái. Mục này mô tả hai nhóm khung quan sát tiêu biểu là khung thanh ghi và khung bộ nhớ; các khung còn lại (MMU, Cache, SoC, I/O, nhật ký hệ thống) hiển thị trạng thái của các mô-đun tương ứng và được trình bày chi tiết ở Chương 4.

Khung thanh ghi nằm trong khung nhìn Editor, gồm hai bảng chuyển đổi qua lại. Bảng thanh ghi số nguyên hiển thị 32 thanh ghi x0–x31 cùng giá trị ở dạng thập lục phân và thập phân; bảng thanh ghi dấu phẩy động hiển thị 32 thanh ghi f0–f31 cùng giá trị số thực và biểu diễn bit ở dạng thập lục phân. Khi một thanh ghi thay đổi giá trị sau một bước thực thi, ô tương ứng được tô sáng để người học dễ nhận ra.

Khung bộ nhớ nằm trong khung nhìn Memory, gồm hai bảng. Bảng bộ nhớ lệnh (Instruction Memory) liệt kê từng lệnh theo địa chỉ, kèm mã máy, dạng disassembly và mã nguồn gốc, cùng một cột để đặt/bỏ điểm dừng. Bảng vùng dữ liệu (Data Segment) hiển thị nội dung bộ nhớ theo hàng (mỗi hàng nhiều từ liên tiếp), cho phép

nhập địa chỉ để nhảy tới và chuyển đổi giữa hiển thị thập lục phân và hiển thị ký tự (ASCII).

a) Khung thanh ghi (trong khung nhìn Editor)

Registers			Floating Point		
Name	Value (Hex)	Dec	Register	Float Value	Hex (Bits)
x0 (zero)	0x00000000	0	f0 (ft0)	0.0	0x00000000
x1 (ra)	0x00400014	4194324	f10 (fa0)	3.14	0x4048f5c3
x2 (sp)	0x7fffffff0	2147483632	f12 (fa2)	-1.5	0xbf c00000
x10 (a0)	0x10010000	268500992	... đủ f0-f31 (RV32F single precision)		
x17 (a7)	0x00000004	4			
... đủ x0-x31; ô tô vàng = giá trị vừa thay đổi					

b) Khung bộ nhớ (trong khung nhìn Memory)

Instruction Memory				
Bkpt	Address	Code	Basic	Source
●	0x00400000	0x00500513	addi a0,zero,5	li a0, 5
	0x00400004	0x00400297	auipc t0,0x0	la a0, msg
	0x00400008	0x00428513	addi a0,t0,4	
	0x0040000c	0x00400893	addi a7,zero,4	li a7, 4
	0x00400010	0x00000073	ecall	ecall

Data Segment		Go to Address (e.g. 0x10010000)						
Address	+0	+4	+8	+c	+10	+14	+18	+1c
0x10010000	6c6c6548	00000a6f	00000000	00000000	00000000	00000000	00000000	00000000
0x10010020	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

Nút "Toggle Hex/ASCII" chuyển giữa hiển thị thập lục phân và ký tự ASCII

Hình 3.6: Các khung quan sát thanh ghi và bộ nhớ

3.5. Luồng thực thi chương trình hợp ngữ

Mục này mô tả luồng xử lý đầu-cuối (end-to-end) của một chương trình hợp ngữ, từ lúc người học nhập mã nguồn cho đến lúc chương trình kết thúc và kết quả được phản ánh trên giao diện. Luồng này gắn kết các thành phần đã trình bày ở các mục trước: trình biên dịch hợp ngữ (mục 3.2.2), bản đồ địa chỉ và việc nạp chương trình (mục 3.2.3), mô hình tiến triển theo chu kỳ (mục 3.3.1) và giao diện điều khiển (mục 3.4). Các điểm chính của luồng được dẫn nguồn tới src/js/javascript.js (điều phối giao

diện), `src/js/assembler.js` (biên dịch) và `src/js/cpu.js` (thực thi và xử lý dịch vụ hệ thống).

Bước 1 — Soạn thảo và biên dịch. Người học nhập hoặc chọn một chương trình hợp ngữ trong khung soạn thảo, rồi nhấn `Assemble`. Khi đó hệ thống trước hết khởi tạo lại thành phần trung tâm để đưa toàn bộ SoC về trạng thái sạch, sau đó gọi trình biên dịch hợp ngữ để biên dịch mã nguồn.

Bước 2 — Biên dịch hai lượt. Trình biên dịch thực hiện theo hai lượt. Lượt thứ nhất duyệt từng dòng để tách nhãn, tính địa chỉ và kích thước của từng lệnh/dữ liệu, đồng thời xử lý các chỉ thị (directive) — qua đó dựng được bảng nhãn và bố trí địa chỉ; vùng mã lệnh mặc định bắt đầu tại `0x00400000`. Lượt thứ hai mã hóa từng lệnh và dữ liệu thành mã máy và ghi vào ảnh bộ nhớ của chương trình. Kết thúc, trình biên dịch xác định địa chỉ bắt đầu thực thi (ưu tiên nhãn `_start` nếu có, nếu không thì lấy lệnh đầu tiên trong vùng mã) và trả về một gói dữ liệu gồm ảnh bộ nhớ, địa chỉ bắt đầu và danh sách lệnh đã mã hóa. Nếu phát hiện lỗi cú pháp hoặc lỗi mã hóa, lỗi được báo theo dòng và hiển thị ở khung kết quả, luồng dừng tại đây để người học sửa mã rồi biên dịch lại. Cấu trúc chi tiết của hai lượt biên dịch, danh mục chỉ thị, pseudo-instruction và bảng mã hóa được trình bày ở Chương 4.

Bước 3 — Nạp chương trình. Khi biên dịch thành công, hệ thống nạp ảnh bộ nhớ vào bộ nhớ chính, đặt bộ đếm chương trình của lõi xử lý về địa chỉ bắt đầu và chuyển lõi xử lý sang trạng thái sẵn sàng chạy; danh sách mã máy đồng thời được hiển thị ở khung kết quả và bộ nhớ lệnh. Hệ thống cũng thiết lập các hàm gọi lại (callback) để xuất kết quả vào/ra (console UART, dịch vụ hệ thống) phù hợp với trạng thái vừa khởi tạo.

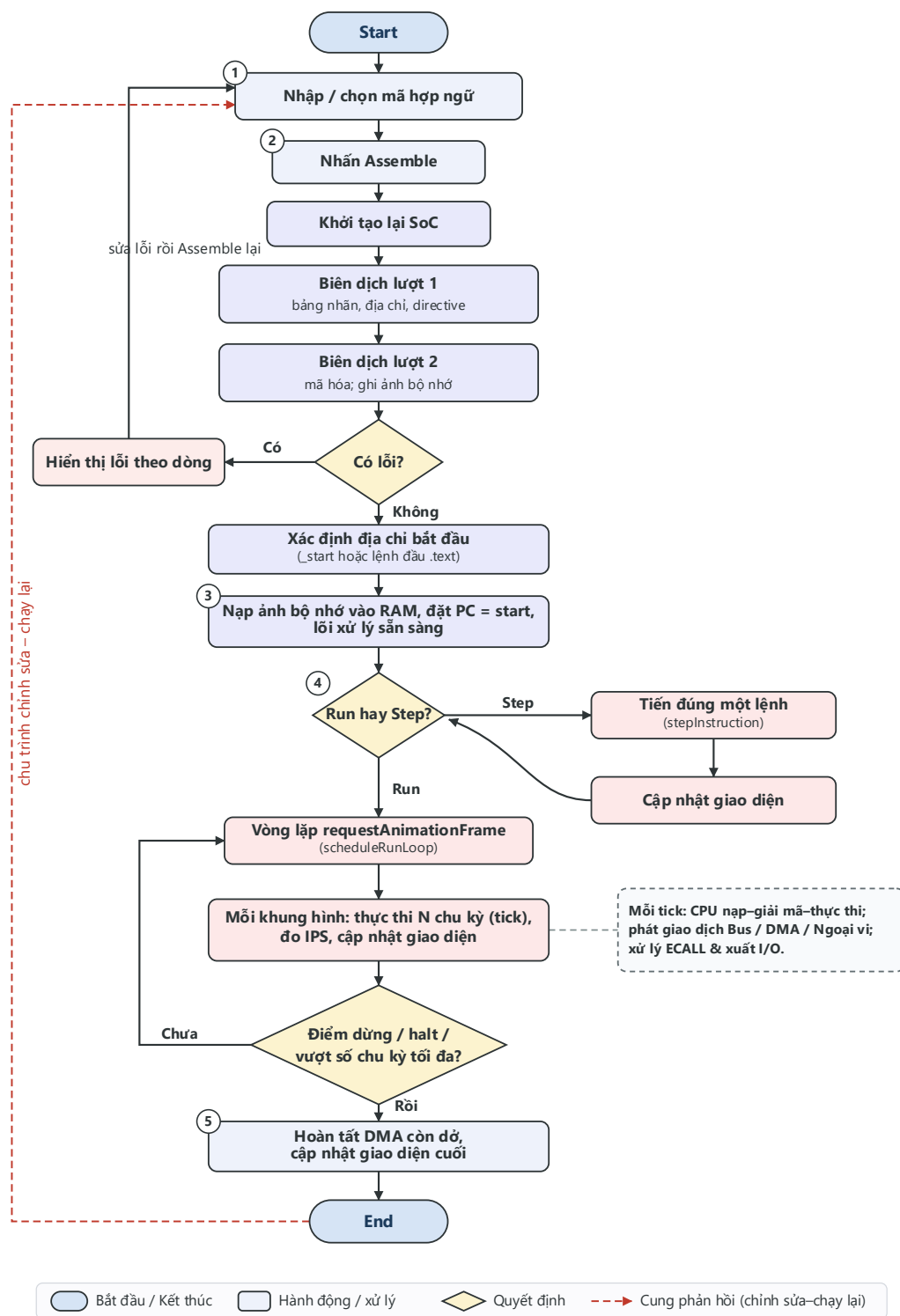
Bước 4 — Thực thi. Người học chọn chạy liên tục (Run) hoặc chạy từng bước (Step). Ở chế độ chạy liên tục, hệ thống dùng vòng lặp gắn với cơ chế vẽ lại theo khung hình của trình duyệt; mỗi khung hình thực thi một số chu kỳ tương ứng với mức tốc độ đã chọn bằng cách lặp lại thao tác tiến chu kỳ của thành phần trung tâm, đồng thời kiểm tra các điều kiện dừng (gấp điểm dừng tại địa chỉ hiện tại, lõi xử lý

dừng, hoặc vượt số chu kỳ tối đa), đo tốc độ thực thi ước lượng và cập nhật giao diện sau mỗi khung hình. Ở chế độ từng bước, hệ thống tiến đúng một lệnh rồi cập nhật giao diện. Trong mỗi chu kỳ, lõi xử lý thực hiện chu trình nạp—giải mã—thực thi và có thể phát sinh giao dịch trên bus, kích hoạt bộ điều khiển DMA hoặc tác động lên thiết bị ngoại vi; các giao dịch này lan truyền qua các tầng theo mô hình tiến chu kỳ đã trình bày ở mục 3.3.1.

Bước 5 — Dịch vụ hệ thống và xuất vào/ra. Trong quá trình thực thi, lệnh ECALL được dùng để gọi các dịch vụ hệ thống mô phỏng. Các dịch vụ bao gồm in số nguyên, in chuỗi và ghi ra đầu ra chuẩn (kết quả hiển thị ở khung console), kết thúc chương trình, cùng một số dịch vụ điều khiển ánh xạ bộ nhớ của đơn vị quản lý bộ nhớ; ngoài ra, vào/ra với thiết bị được thực hiện qua các thanh ghi MMIO như UART. Danh sách đầy đủ các dịch vụ hệ thống và hành vi của chúng được trình bày ở Chương 4.

Bước 6 — Điều kiện dừng và cập nhật giao diện. Quá trình chạy kết thúc khi xảy ra một trong các điều kiện: chương trình gọi dịch vụ kết thúc, lõi xử lý dừng (halt), gặp điểm dừng, hoặc vượt giới hạn số chu kỳ. Khi dừng, hệ thống có thể hoàn tất nốt các giao dịch DMA còn dở, đặt lại chỉ số tốc độ và cập nhật toàn bộ giao diện để phản ánh trạng thái cuối cùng (thanh ghi, bộ nhớ, bộ nhớ đệm, MMU, sơ đồ SoC, thiết bị ngoại vi và nhật ký). Người học có thể quan sát kết quả, đặt thêm điểm dừng, chạy tiếp hoặc khởi tạo lại để lặp chu trình.

Như vậy, luồng thực thi đầu—cuối gắn kết toàn bộ các thành phần thiết kế thành một chu trình khép kín phục vụ người học: từ mã nguồn hợp ngữ, qua biên dịch hai lượt và nạp vào không gian địa chỉ chung, đến thực thi theo mô hình tiến chu kỳ với sự tham gia của lõi xử lý, hệ thống kết nối, DMA và thiết bị ngoại vi, và cuối cùng phản ánh kết quả lên các khung quan sát của giao diện. Các quyết định thiết kế ở mức kiến trúc trình bày trong chương này là cơ sở cho phần hiện thực chi tiết của từng thành phần ở Chương 4.



Hình 3.7: Sơ đồ hoạt động luồng thực thi chương trình hợp ngữ (đầu–cuối)

Chương 4. HIỆN THỰC HỆ THỐNG

4.1. Trình biên dịch hợp ngữ (Assembler)

Trình biên dịch hợp ngữ được hiện thực như một đối tượng đơn assembler mang toàn bộ trạng thái biên dịch và các hàm xử lý, biên dịch mã hợp ngữ RV32IMF thành mã máy theo quy trình hai lượt.

4.1.1. Kiến trúc hai lượt (two-pass)

Trạng thái biên dịch gồm: ảnh bộ nhớ memory (ánh xạ địa chỉ byte sang giá trị byte), bảng nhãn labels, bảng hằng số equValues do chỉ thị .eqv định nghĩa, con trỏ địa chỉ hiện hành currentAddress, danh sách thông tin từng dòng instructionLines, và danh sách mã máy đã sinh binaryCode. Hai địa chỉ nền mặc định được quy ước: vùng mã .text bắt đầu tại 0x00400000 và vùng dữ liệu .data bắt đầu tại 0x10010000. Hàm điều phối assemble lần lượt gọi _reset, _pass1, _pass2, rồi xác định địa chỉ bắt đầu thực thi.

Lượt một (_pass1). Lượt một duyệt mã nguồn theo từng dòng. Với mỗi dòng, trình biên dịch cắt bỏ chú thích (ký tự # đến hết dòng), tách nhãn ở đầu dòng (mẫu tên:) và ghi nhãn đó vào bảng nhãn với địa chỉ hiện hành cùng loại (instruction nếu đang ở vùng mã, data nếu đang ở vùng dữ liệu). Phần còn lại của dòng được tách thành mnemonic và danh sách toán hạng bằng một bộ phân tích có nhận biết dấu ngoặc và chuỗi (_parseOperandsSmart), nhờ đó toán hạng dạng offset(base) hay chuỗi chứa dấu phẩy không bị tách sai. Tùy theo dòng là chỉ thị, lệnh thật hay lệnh giả, lượt một tính kích thước byte tương ứng và tăng currentAddress: mỗi lệnh RV32 chiếm 4 byte, riêng lệnh giả có thể chiếm 4 hoặc 8 byte và được ước lượng trước bởi _estimatePseudoInstructionSize. Mục tiêu của lượt một là dựng đầy đủ bảng nhãn và bố trí địa chỉ trước khi mã hóa, để các tham chiếu tiền (forward reference) tới nhãn nằm phía sau vẫn giải được ở lượt hai.

Lượt hai (_pass2). Lượt hai duyệt lại danh sách dòng đã ghi nhận, bỏ qua các dòng không phải lệnh, và mã hóa từng lệnh thành chuỗi nhị phân 32 bit. Lệnh thật được

mã hóa trực tiếp bởi `_encodeInstruction`; lệnh giả được khai triển thành một hoặc nhiều lệnh thật rồi mã hóa bởi `_expandAndEncodePseudo`. Mỗi từ lệnh 32 bit được ghi vào ảnh bộ nhớ theo thứ tự byte nhỏ trước (little-endian) bằng hàm `_writeBinaryToMemory`, đồng thời được thêm vào `binaryCode` kèm biểu diễn thập lục phân để hiển thị. Lỗi cú pháp hoặc lỗi mã hóa được bắt và ném lại kèm số dòng và nội dung dòng gốc, nhờ đó thông báo lỗi chỉ đúng vị trí dòng để người học sửa.

Xác định địa chỉ bắt đầu. Sau hai lượt, trình biên dịch ưu tiên nhãn `_start` (nếu tồn tại và nằm trong vùng `.text`) làm địa chỉ bắt đầu; nếu không có, nó lấy lệnh đầu tiên trong vùng mã, mặc định là `0x00400000`. Kết quả trả về là một gói gồm ảnh bộ nhớ, địa chỉ bắt đầu và danh sách lệnh đã mã hóa.

4.1.2. Chỉ thị và lệnh giả

Trình biên dịch điều phối các chỉ thị (directive) qua một bảng ánh xạ tên chỉ thị sang hàm xử lý. Bảng 4.1 liệt kê các chỉ thị được hỗ trợ; trong đó .single là tên gọi khác của .float, .string là tên gọi khác của .asciiz, và .global là tên gọi khác của .globl (cùng dùng chung một hàm xử lý).

Bảng 4.1. Các chỉ thị được hỗ trợ

Chỉ thị	Chức năng
.text [addr]	Chuyển sang vùng mã; đặt địa chỉ nếu có tham số
.data [addr]	Chuyển sang vùng dữ liệu; mặc định 0x10010000
.section <tên>	Chuyển section theo tên (hỗ trợ .text, .data, .rodata, .bss, .text.init...)
.word, .half, .byte	Lưu giá trị 4/2/1 byte (kèm căn lề tương ứng cho word/half)
.float (.single)	Lưu số thực đơn IEEE 754 (4 byte, little-endian)
.ascii	Lưu chuỗi, không có ký tự kết thúc
.asciiz (.string)	Lưu chuỗi, có ký tự null kết thúc
.space <n>	Cấp phát n byte giá trị 0
.align <n>	Căn lề địa chỉ hiện hành theo 2 ⁿ byte
.globl (.global)	Đánh dấu nhãn là toàn cục
.extern	Khai báo ký hiệu ngoài (tương thích đầu vào)
.eqv <tên>, <giá trị>	Định nghĩa hằng số ký hiệu
.org <addr>	Đặt địa chỉ biên dịch hiện hành

Các chỉ thị lưu dữ liệu tự động chuyển sang vùng dữ liệu nếu cần (_ensureDataSection) và căn lề trước khi ghi; chỉ thị .byte còn nhận ký tự dạng 'A'; chỉ thị .float dùng DataView.setFloat32 (little-endian) để biểu diễn IEEE 754; xử lý chuỗi có giải mã các chuỗi thoát (\n, \t, \", ...).

Lệnh giả (pseudo-instruction) được khai triển thành lệnh thật ở lượt hai. Bảng 4.2 liệt kê 13 lệnh giả và quy tắc khai triển tương ứng.

Bảng 4.2. Lệnh giả và quy tắc khai triển

Lệnh giả	Khai triển
nop	addi x0, x0, 0
mv rd, rs	addi rd, rs, 0
li rd, imm	addi (khi imm trong [-2048, 2047]); ngược lại lui + addi (hoặc chỉ lui khi phần thấp bằng 0)
la rd, sym	auipc rd, %hi'(sym-PC) + addi rd, rd, %lo'(sym-PC) (luôn sinh đủ 8 byte)
call sym	auipc ra, hi + jalr ra, ra, lo
j label	jal x0, label
jr rs	jalr x0, rs, 0
ret	jalr x0, ra, 0
bnez rs, label	bne rs, x0, label
beqz rs, label	beq rs, x0, label
fmv.s fd, fs	fsgnj.s fd, fs, fs
fabs.s fd, fs	fsgnjx.s fd, fs, fs
fneg.s fd, fs	fsgnjn.s fd, fs, fs

Hai lệnh giả li và la cần xử lý đặc biệt vì kích thước phụ thuộc giá trị/khoảng cách. Với li, khi hằng số nằm trong dải 12 bit có dấu thì chỉ cần một lệnh addi; ngược lại trình biên dịch tách hằng số thành phần cao 20 bit (lui) và phần thấp 12 bit có dấu (addi), áp dụng quy tắc làm tròn lên (cộng 0x800 trước khi dịch phải 12 bit) để bù khi bit 11 của phần thấp bằng 1. Với la, trình biên dịch tính khoảng cách tương đối giữa nhãn đích và địa chỉ lệnh rồi sinh cặp auipc + addi; vì lượt một đã dành sẵn 8 byte cho la, lượt hai luôn phát đủ cả hai lệnh kể cả khi phần thấp bằng 0, tránh để lại khoảng trống 4 byte mà lỗi xử lý có thể nạp nhầm thành lệnh không hợp lệ.

4.1.3. Mã hóa lệnh và ánh xạ thanh ghi

Bảng opcodes lưu thông tin mã hóa của từng lệnh: mã thao tác (opcode), trường funct3/funct7 (khi có), và loại định dạng (type). Hàm `_encodeInstruction` chọn nhánh

mã hóa theo loại định dạng và ghép các trường bit thành chuỗi 32 ký tự nhị phân; bước cuối kiểm tra độ dài đúng 32 bit trước khi trả về.

Các trường immediate tương đối với PC (nhánh B và J) được kiểm tra căn lẻ và phạm vi bởi `_validatePCRelativeImmediate` trước khi cắt ghép bit. Toán hạng dạng bộ nhớ `offset(base)` được phân tích bởi `_parseMemoryOperand`. Trình biên dịch hỗ trợ hai bộ định vị (relocation) `%hi(sym)` và `%lo(sym)` cho phép nạp địa chỉ 32 bit bằng cặp lui/addi: phần cao được tính bằng cách làm tròn lên $\text{roundedUpper} = \lfloor (\text{sym} + 0x800) / 0x1000 \rfloor$ và phần thấp là $\text{sym} - \text{roundedUpper} \cdot 0x1000$, bảo đảm tổng hai phần khôi phục đúng giá trị gốc khi phần thấp được mở rộng dấu.

Với các lệnh dấu phẩy động, chế độ làm tròn (rounding mode) được mã hóa vào trường `rm`: nếu lệnh có `funct3` cố định thì dùng giá trị đó, ngược lại đọc toán hạng chế độ làm tròn tùy chọn (`rne/rtz/rdn/rup/rmm/dyn`, mặc định `dyn`). Lệnh `fence` được mã hóa với hai tập quyền hạn (`predecessor/successor`) qua `_encodeFenceSet`; tuy vậy, đúng như giới hạn đã nêu ở mục 1.3.2, `fence` chỉ được mã hóa chứ chưa được xử lý thực thi.

Trình biên dịch chấp nhận nhiều cách viết tên thanh ghi và quy về tên chuẩn qua bảng `registerMapping`: tên ABI (ví dụ `sp`, `a0`, `t0`), tên kiểu MIPS có tiền tố `$`, và tên dạng chỉ số (`x0–x31`, `f0–f31`). Bảng 4.3 trình bày ánh xạ tên ABI của 32 thanh ghi số nguyên và 32 thanh ghi dấu phẩy động.

Bảng 4.3. Ánh xạ tên thanh ghi ABI

Số nguyên	ABI	Vai trò quy ước	Dấu phẩy động	ABI
x0	zero	hằng số 0	f0–f7	ft0–ft7 (tạm)
x1	ra	địa chỉ trả về	f8–f9	fs0–fs1 (được giữ)
x2	sp	con trỏ ngăn xếp	f10–f11	fa0–fa1 (đổi số/trả về)
x3	gp	con trỏ toàn cục	f12–f17	fa2–fa7 (đổi số)
x4	tp	con trỏ luồng	f18–f27	fs2–fs11 (được giữ)
x5–x7	t0–t2	tạm	f28–f31	ft8–ft11 (tạm)

Số nguyên	ABI	Vai trò quy ước	Dấu phẩy động	ABI
x8–x9	s0/fp, s1	được giữ / con trỏ khung		
x10–x17	a0–a7	đối số/trả về (a7 = mã syscall)		
x18–x27	s2–s11	được giữ		
x28–x31	t3–t6	tạm		

4.1.4. Hỗ trợ soạn thảo

Trình soạn thảo dựa trên thư viện CodeMirror [14]. Việc tô màu cú pháp được định nghĩa bằng một chế độ (mode) đơn giản tên `riscv` qua `CodeMirror.defineSimpleMode`, phân loại các phần tử của một dòng hợp ngữ thành: chú thích (`#...`), chuỗi, từ khóa chỉ thị (`.text`, `.data`, `.word`, ...), một số mnemonic thường gặp, tên thanh ghi (`x0–x31`, `f0–f31` và tên ABI), nhãn (tên:) và hằng số (thập lục phân hoặc thập phân/thực).

Hỗ trợ gợi ý nhập (autocompletion) được hiện thực trong mô-đun riêng `editor_hint.js` qua tiện ích `show-hint` của CodeMirror. Mô-đun này dựng một từ điển gợi ý từ chính bảng opcodes và bảng directives của trình biên dịch (nhờ đó danh sách gợi ý luôn đồng bộ với tập lệnh được hỗ trợ), kèm theo danh mục thanh ghi số nguyên/dấu phẩy động, chế độ làm tròn, tập quyền hạn của fence, và các trợ giúp immediate (`%hi`, `%lo`, mẫu `offset(base)`). Gợi ý có nhận biết ngữ cảnh: ở đầu câu lệnh, hệ thống gợi ý mnemonic hoặc chỉ thị; ở vị trí toán hạng, hệ thống suy ra loại toán hạng mong đợi (thanh ghi số nguyên, thanh ghi dấu phẩy động, nhãn, chế độ làm tròn, toán hạng bộ nhớ...) dựa trên loại định dạng của lệnh và chỉ số toán hạng. Hệ thống còn tự dò các nhãn và hằng số `.eqv` do người dùng định nghĩa trong mã để đưa vào danh sách gợi ý, và không kích hoạt gợi ý khi con trỏ nằm trong chú thích hoặc chuỗi.



Hình 4.2: Minh họa cơ chế gợi ý cú pháp trong trình soạn thảo.

4.2. Bộ xử lý CPU RV32IMF

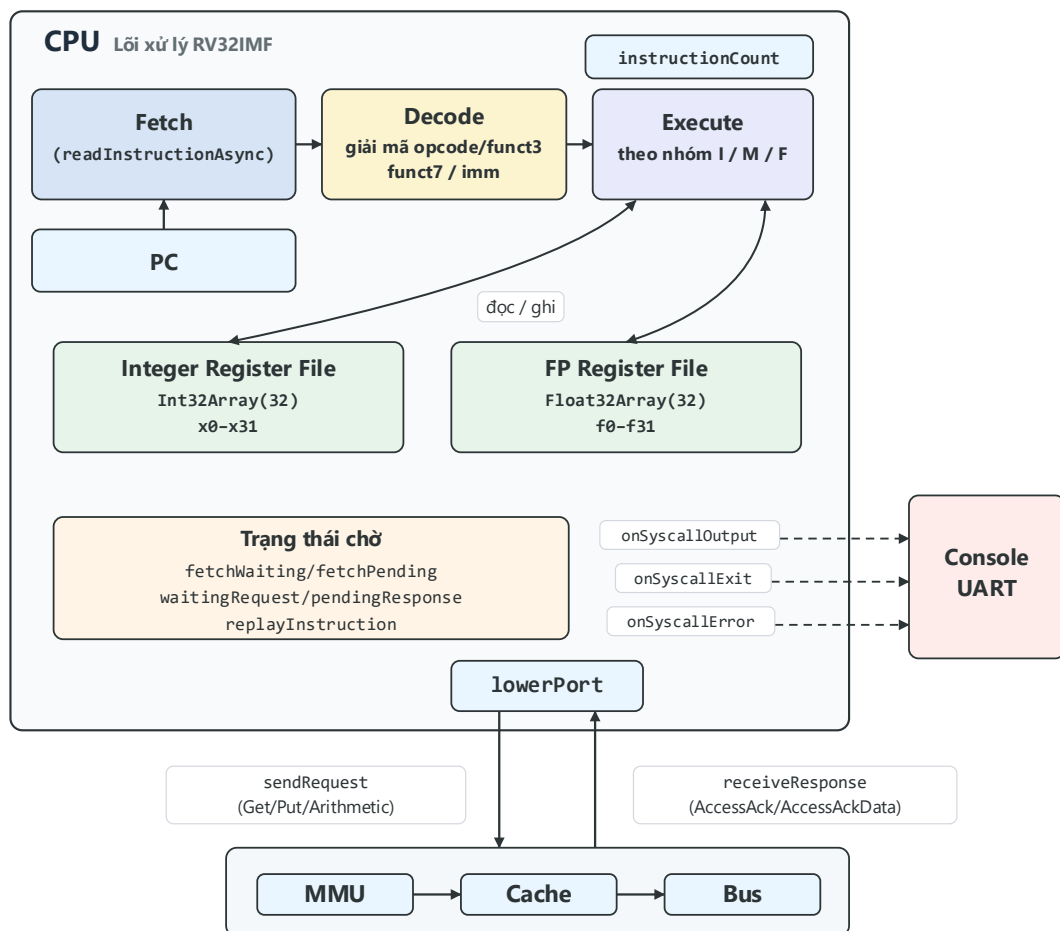
Lỗi xử lý được hiện thực trong lớp CPU. Phần này trình bày tổ chức trạng thái, chu trình thực thi cùng mô hình truy cập bộ nhớ bất đồng bộ, cách hiện thực ba nhóm lệnh I/M/F, và việc xử lý dịch vụ hệ thống.

4.2.1. Tổ chức thanh ghi và trạng thái

Tập thanh ghi số nguyên được biểu diễn bằng `Int32Array(32)` (`x0–x31`), tập thanh ghi dấu phẩy động bằng `Float32Array(32)` (`f0–f31`); việc dùng `Float32Array` khiến mỗi giá trị tự động được lưu ở độ chính xác đơn IEEE 754 [10] mà không cần cắt xén thủ công. Bộ đếm chương trình là `pc`. Ngoài ra, lỗi xử lý giữ bộ đếm số lệnh đã hoàn tất `instructionCount`, một trường giới hạn an toàn `maxSteps` (mặc định 1.000.000), cờ trạng thái chạy `isRunning`, và hai cổng kết nối: cổng trên (`upperPort`) và cổng dưới (`lowerPort`) — cổng dưới là đường phát yêu cầu xuống đơn vị quản lý bộ nhớ. Thanh

ghi x0 luôn được giữ bằng 0: mọi nhánh ghi kết quả đều kiểm tra rd != 0 trước khi ghi.

Để phục vụ mô hình truy cập bộ nhớ bất đồng bộ (mục 4.2.2), lõi xử lý còn mang một nhóm trường trạng thái chờ: fetchWaiting/fetchPending cho việc nạp lệnh, waitingRequest/pendingResponse cho yêu cầu dữ liệu đang chờ và phản hồi đã về, và replayInstruction lưu lệnh cần thực hiện lại. Việc xuất kết quả vào/ra của dịch vụ hệ thống được tách rời khỏi lõi xử lý qua ba hàm gọi lại có thể gắn từ bên ngoài: onSyscallOutput, onSyscallExit, onSyscallError.



Hình 4.3: Minh họa tổ chức bên trong của lõi xử lý.

4.2.2. Chu trình nạp–giải mã–thực thi và truy cập bộ nhớ bất đồng bộ

Như đã nêu ở mục 3.3.1, mỗi chu kỳ mô phỏng tương ứng một lần gọi tick của lõi xử lý. Tuy nhiên, do lõi xử lý truy cập bộ nhớ qua bus với độ trễ nhiều chu kỳ, một lệnh có truy cập bộ nhớ không thể hoàn tất trong một chu kỳ. Hệ thống giải quyết điều này bằng một mô hình **yêu cầu – chờ – phản hồi – thực hiện lại** (request–wait–response–replay) được điều khiển bởi các trường trạng thái nêu ở mục 4.2.1.

Giải mã (decode) trích các trường opcode, rd, funct3, rs1, rs2, funct7, và với lệnh dấu phẩy động còn trích rs3 và fmt. Việc nhận dạng tên lệnh được thực hiện bằng cách so khớp tổ hợp opcode/funct3/funct7 (và các trường con như rs2_subfield, fmt) với một bảng định dạng nội bộ; sau đó immediate được tái tạo theo từng loại định dạng, kể cả việc mở rộng dấu cho các định dạng I/S/B/J.

Thực thi (execute) trả về địa chỉ lệnh kế tiếp. Với lệnh số học/logic và rẽ nhánh, lệnh hoàn tất ngay trong một chu kỳ. Với lệnh nạp/lưu và lệnh nguyên tử, hàm thực thi hoạt động theo hai pha trong hai lần gọi khác nhau:

- **Pha phát yêu cầu.** Khi chưa có yêu cầu nào đang chờ (!waitingRequest && !pendingResponse), lõi xử lý tính địa chỉ hiệu dụng, phát một yêu cầu xuống cổng dưới (ví dụ Get cho nạp, PutFullData/PutPartialData cho lưu), đặt waitingRequest, rồi trả về nextPc = pc để giữ nguyên bộ đếm chương trình.
- **Pha hoàn tất.** Khi phản hồi đã về (pendingResponse khác rỗng), lõi xử lý lấy dữ liệu, thực hiện mở rộng dấu/cắt byte theo loại lệnh (ví dụ LB mở rộng dấu 8 bit, LBU mở rộng số 0), ghi vào thanh ghi đích, xóa các cờ chờ và để bộ đếm chương trình tiến tới lệnh kế.

Hàm tick điều phối toàn bộ máy trạng thái này:

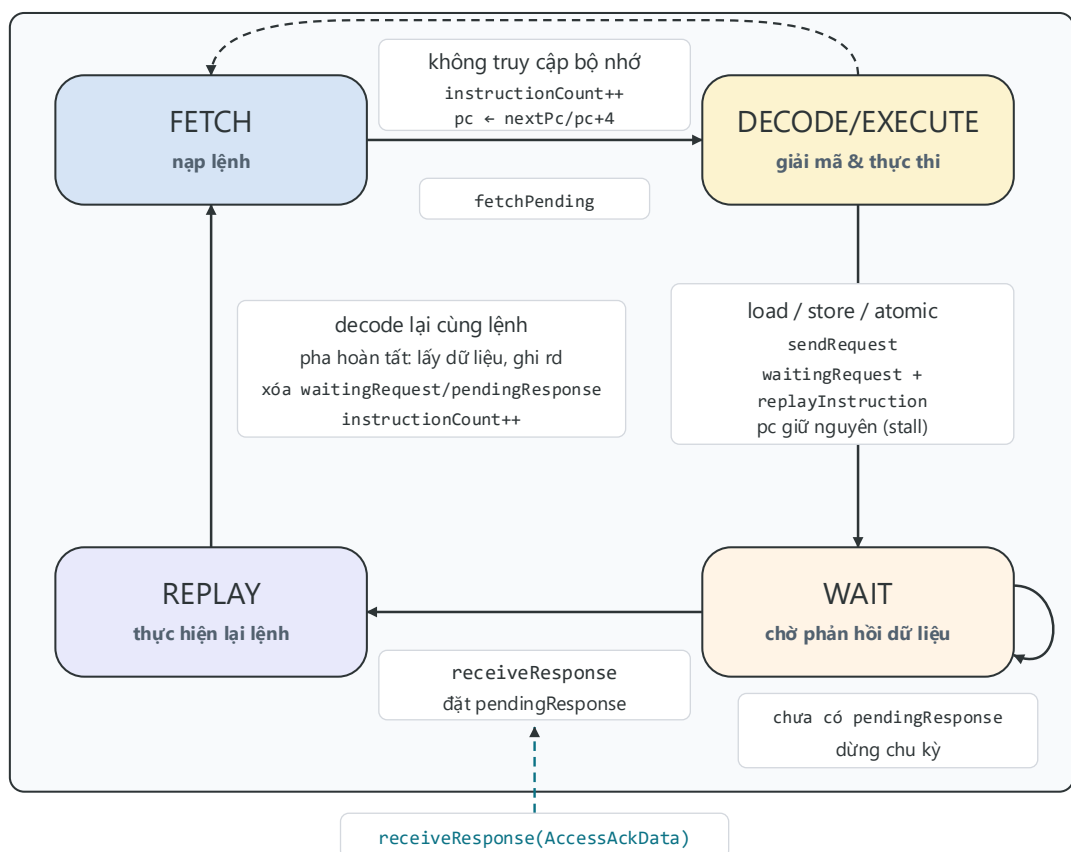
```
nếu (waitingRequest và chưa có pendingResponse) → dừng chu kỳ này (chờ phản hồi dữ liệu)
shouldReplay = (waitingRequest và pendingResponse và replayInstruction)
nếu (không replay):
    nếu (chưa phát fetch và chưa có fetchPending) → phát readInstructionAsync(pc); dừng
    nếu (chưa có fetchPending) → dừng (chờ phản hồi nạp lệnh)
inst = (replay ? replayInstruction : fetchPending)
```

```

decoded = decode(inst); execute(decoded)
nếu (vẫn waitingRequest) → replayInstruction = {pc, inst} // lệnh sẽ chạy lại
ngược lại → replayInstruction = null; instructionCount++ // lệnh hoàn tất
cập nhật pc (nextPc nếu có, ngược lại pc += 4)

```

Phản hồi từ phía dưới được nhận qua `receiveResponse`: phản hồi của nạp lệnh được định tuyến vào `fetchPending`, các phản hồi còn lại vào `pendingResponse`. Nhờ mô hình này, một lệnh nạp dữ liệu có thể trải dài qua nhiều chu kỳ — phát yêu cầu ở chu kỳ N, dữ liệu lan qua MMU/cache/bus/bộ nhớ rồi quay về sau một số chu kỳ tương ứng độ trễ — trong khi bộ đếm chương trình bị "đóng băng" cho đến khi lệnh hoàn tất, đúng với mô hình tiến chu kỳ đã mô tả ở mục 3.3.1.



Hình 4.4: Mô tả máy trạng thái truy cập bộ nhớ nói trên.

4.2.3. Hiện thực các nhóm lệnh I/M/F

Nhóm số nguyên cơ sở (I). Các phép ALU được hiện thực bằng toán tử JavaScript trên số nguyên 32 bit có dấu, với toán tử $|$ 0 để ép kết quả về 32 bit có dấu sau mỗi phép tính (mô phỏng tràn vòng của số học 32 bit). Phép dịch dùng $\ll$, $\gg$ (số học) và $\ggg$ (logic) với lượng dịch lấy 5 bit thấp; so sánh không dấu dùng $\ggg$ 0 để diễn giải toán hạng như số không dấu. Các lệnh lui/auipc xử lý immediate 20 bit cao; rẽ nhánh và nhảy tính địa chỉ đích tương đối PC. Lệnh ecall chuyển sang xử lý dịch vụ hệ thống (mục 4.2.4); ebreak dừng lỗi xử lý.

Nhóm nhân/chia số nguyên (M). Phép nhân thấp mul dùng Math.imul để lấy 32 bit thấp đúng theo ngữ nghĩa nhân 32 bit. Ba lệnh lấy phần cao mulh/mulhsu/mulhu được hiện thực bằng số nguyên lớn BigInt: tích 64 bit được tính rồi dịch phải 32 bit ($\gg 32n$), với việc chuyển toán hạng sang miền có dấu hoặc không dấu phù hợp từng biến thể. Phép chia/lấy dư tuân theo đặc tả RISC-V [1] cho các trường hợp biên: chia cho 0 trả về toàn bit 1 (tức -1) cho div/divu, và trả về số bị chia cho rem/remu; trường hợp tràn $\text{INT_MIN} / (-1)$ trả về INT_MIN cho div và 0 cho rem.

```
MULH: (BigInt(rs1) * BigInt(rs2)) >> 32n // có dấu × có dấu
DIV:  rs2 == 0 → -1; rs1==INT_MIN && rs2==-1 → INT_MIN; ngược lại [rs1/rs2]
```

Nhóm dấu phẩy động đơn (F). Các phép số học cơ bản (fadd.s, fsub.s, fmul.s, fdiv.s) dùng trực tiếp toán tử số học JavaScript, kết quả tự được rút về độ chính xác đơn khi ghi vào Float32Array; phép chia cho 0 trả về $\pm\infty$ hoặc NaN theo dấu của tử số. Các lệnh nhân–cộng hợp nhất (fmadd.s, fmsub.s, fnmsub.s, fnmadd.s) tính theo công thức tương ứng. Các thao tác trên bit (dấu, di chuyển bit, phân loại) dùng ArrayBuffer + DataView để truy cập biểu diễn nhị phân 32 bit của số thực: nhóm fsgnj.s/fsgnjn.s/fsgnjx.s ghép bit dấu của toán hạng thứ hai vào phần độ lớn của toán hạng thứ nhất; fmv.x.w/fmv.w.x sao chép nguyên trạng 32 bit giữa thanh ghi số nguyên và thanh ghi dấu phẩy động. Lệnh phân loại fclass.s trả về một trong 10 lớp (mỗi lớp ứng một bit trong kết quả): $-\infty$, số âm chuẩn, số âm dưới chuẩn, -0 , $+0$, số dương dưới chuẩn, số dương chuẩn, $+\infty$, NaN báo hiệu (signaling) và NaN im lặng (quiet), được xác định từ trường mũ và phần định trị của biểu diễn IEEE 754. Các

lệnh so sánh `feq.s/flt.s/fle.s` trả về 0 khi có toán hạng là NaN; `fmin.s/fmax.s` xử lý NaN bằng cách trả về toán hạng không-NaN còn lại.

Các lệnh chuyển đổi `fcvt.w.s/fcvt.wu.s` (số thực $\rightarrow$ số nguyên) áp dụng chế độ làm tròn theo trường `rm`: chế độ làm tròn về số chẵn gần nhất (`rne`) dùng `Math.round` và chế độ làm tròn về 0 (`rtz`) dùng `Math.trunc`; các chế độ còn lại quy về `rne`. Kết quả còn được kẹp (`saturate`) vào dải số nguyên hợp lệ khi nguồn là NaN hoặc vượt biên. Chiều ngược lại `fcvt.s.w/fcvt.s.wu` chuyển số nguyên có dấu/không dấu sang số thực.

Cần nêu rõ các giới hạn của hiện thực dấu phẩy động, nhất quán với mục 1.3.2: hệ thống chưa hiện thực thanh ghi điều khiển/trạng thái dấu phẩy động (FCSR), các cờ ngoại lệ (`fflags`) và việc chọn chế độ làm tròn động (`frm`); việc làm tròn chỉ phân biệt hai chế độ `rne` và `rtz` (các chế độ khác quy về `rne`), và tính tương thích bit-exact với chuẩn IEEE 754 [10] ở mọi trường hợp biên chưa được kiểm chứng đầy đủ. Lệnh `amoadd.w` (cộng nguyên tử, thuộc mở rộng RV32A) cũng được hiện thực như một mở rộng minh họa nằm ngoài tên đề tài RV32IMF; cơ chế nguyên tử ở tầng bus được trình bày ở mục 4.4.

4.2.4. Xử lý dịch vụ hệ thống (`syscall`)

Lệnh `ecall` gọi tới `handleSyscall`, trong đó mã dịch vụ được lấy từ thanh ghi `a7` (`x17`) và các đối số từ `a0–a2` (`x10–x12`), theo quy ước tương thích với môi trường RARS [16]. Bảng 4.4 liệt kê các dịch vụ được hỗ trợ.

Bảng 4.4. Các dịch vụ hệ thống được hỗ trợ (`'a7'` chọn dịch vụ)

<code>a7</code>	Dịch vụ	Đối số	Hành vi
1	In số nguyên	<code>a0</code> = giá trị	Xuất giá trị số nguyên ra console
4	In chuỗi	<code>a0</code> = địa chỉ chuỗi	Đọc chuỗi kết thúc null từ bộ nhớ và xuất ra console
64	Ghi ra mô tả tệp	<code>a0=fd</code> , <code>a1</code> =địa chỉ đệm, <code>a2</code> =số byte	Khi <code>fd=1</code> (<code>stdout</code>): xuất <code>a2</code> byte ra console; trả số byte đã ghi vào <code>a0</code>
93	Kết thúc	<code>a0</code> = mã thoát	Dừng lỗi xử lý, phát mã thoát qua <code>onSyscallExit</code>
0	Halt ngằm	—	Dừng lỗi xử lý

a7	Dịch vụ	Đối số	Hành vi
100	MMU – ánh xạ trang	a0=VA, a1=PA, a2=cờ quyền	Gọi mapPage (bit 0=R, 1=W, 2=X, 3=cacheable)
101	MMU – hủy ánh xạ	a0=VA	Gọi unmapPage
102	MMU – xóa ánh xạ	—	Gọi clearMappings

Các dịch vụ in/ghi không tác động trực tiếp lên giao diện mà phát chuỗi kết quả qua hàm gọi lại onSyscallOutput; lớp giao diện gắn hàm này để đưa kết quả vào khung console (mục 4.8). Ba dịch vụ 100/101/102 cho phép chương trình hợp ngữ điều khiển bảng ánh xạ của đơn vị quản lý bộ nhớ ngay trong lúc chạy: lỗi xử lý truy cập đối tượng MMU qua tham chiếu lowerPort.lower và gọi các phương thức mapPage/unmapPage/clearMappings tương ứng (mục 4.3.2). Mã dịch vụ không hỗ trợ sẽ phát thông báo lỗi qua onSyscallError và dừng lỗi xử lý.

4.3. Hệ thống bộ nhớ

Hệ thống bộ nhớ gồm ba lớp hiện thực độc lập: bộ nhớ chính, đơn vị quản lý bộ nhớ kèm TLB, và phân cấp bộ nhớ đệm. Các lớp này trao đổi qua giao diện yêu cầu/phản hồi thống nhất đã mô tả ở mục 3.3.2.

4.3.1. Bộ nhớ chính

Bộ nhớ chính được hiện thực trong lớp Mem. Vùng lưu trữ là một đối tượng JavaScript ánh xạ địa chỉ byte sang giá trị byte (`mem = {}`), tức lưu trữ thưa (*sparse*): chỉ những byte thực sự được ghi mới tồn tại trong đối tượng, nhờ đó không gian địa chỉ 32 bit được mô phỏng mà không cần cấp phát toàn bộ. Hàm tiện ích `readSizedValue/writeSizedValue` đọc/ghi 1/2/4 byte theo thứ tự little-endian.

Để biểu diễn chi phí truy cập bộ nhớ vật lý, Mem mang một độ trễ cấu hình được (*latency*); trong cấu hình SoC, độ trễ này được đặt bằng 20 chu kỳ. Khi nhận một yêu cầu, bộ nhớ ghi nhận thời điểm sẵn sàng `readyCycle = cycle + latency`; mỗi tick chỉ phục vụ yêu cầu khi đã đủ độ trễ. Với yêu cầu kích thước lớn hơn 4 byte (yêu cầu lấp

khối kiểu fill phục vụ refill của bộ nhớ đệm), bộ nhớ chuyển sang chế độ truyền chuỗi (burst): trả về dữ liệu theo từng "nhịp" (beat) 4 byte, mỗi nhịp cách nhau burstBeatLatency chu kỳ, kèm các trường beatIndex/beatCount/lastBeat để bên nhận ghép lại thành một dòng đầy đủ. Đáp ứng cho yêu cầu đọc là AccessAckData (kèm dữ liệu), cho yêu cầu ghi là AccessAck. Ngoài luồng tick, lớp Mem còn cung cấp directRead/directWrite cho các truy cập trực tiếp (phục vụ refill bộ nhớ đệm, cầu nối bus và các khung quan sát/gỡ lỗi).

4.3.2. Đơn vị quản lý bộ nhớ và TLB

Đơn vị quản lý bộ nhớ được hiện thực trong lớp MMU, nằm trên đường yêu cầu giữa lõi xử lý và bộ nhớ đệm. MMU không được cấp một bước tick riêng (mục 3.3.1): nó thực hiện dịch địa chỉ ngay trên đường yêu cầu đi xuống. Bảng 4.5 tóm tắt cấu hình MMU.

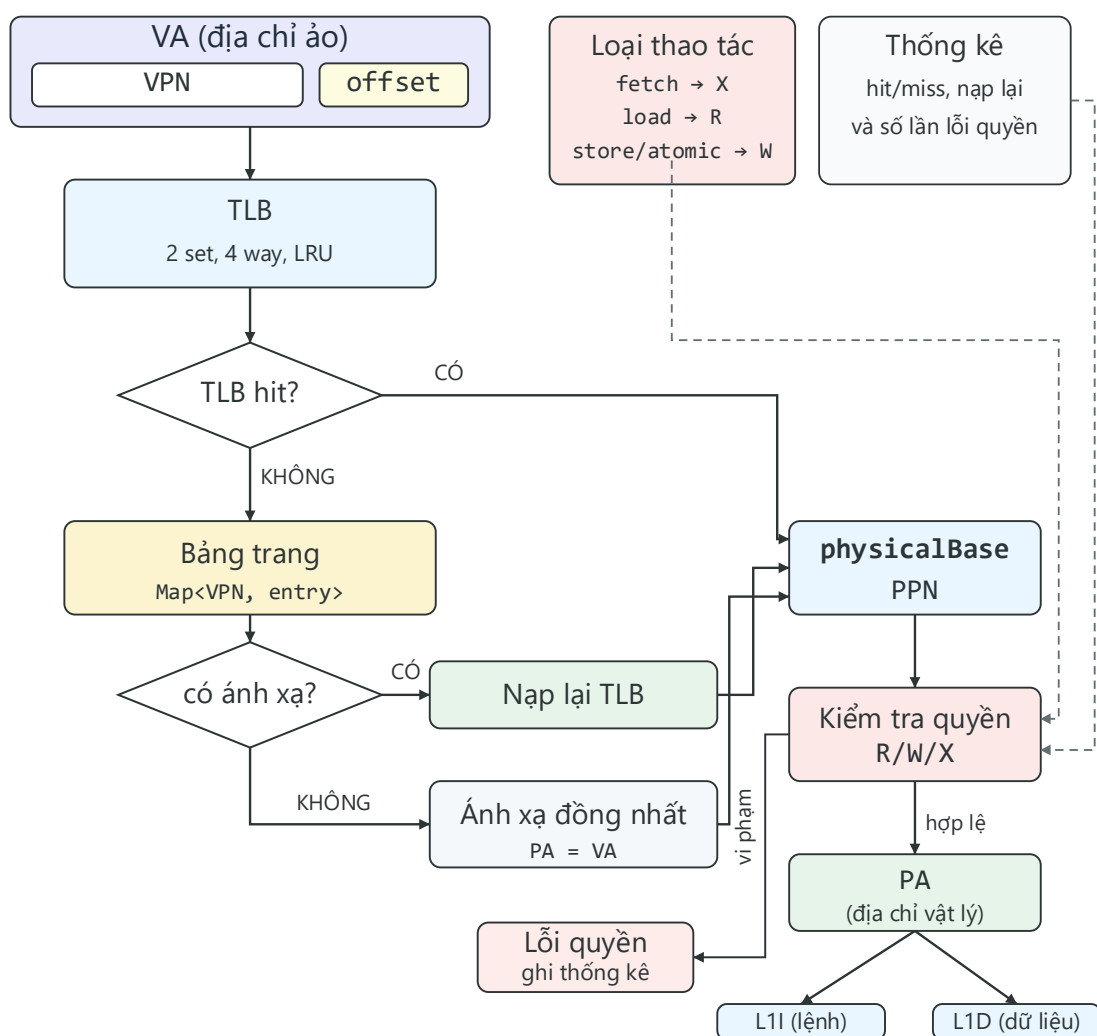
Bảng 4.5. Cấu hình đơn vị quản lý bộ nhớ và TLB

Tham số	Giá trị mặc định	Tùy chọn cấu hình (UI)
Kích thước trang	4096 byte	1024 / 2048 / 4096 / 8192
Số mục TLB	8	4 / 8 / 16 / 32
Số đường (way) TLB	4	2 / 4 / liên kết đầy đủ
Số tập (set) TLB	2 (= số mục / số đường)	(suy ra)
Chính sách thay thế TLB	LRU (theo lastReference)	—
Quyền truy cập mỗi trang	R / W / X	—
Thuộc tính lưu đệm	cacheable theo từng trang	—
Bảng trang	Map (phần mềm)	—
Lịch sử dịch địa chỉ	32 mục gần nhất	—

Bảng trang được biểu diễn bằng cấu trúc Map ánh xạ số trang ảo (VPN) sang một mục chứa địa chỉ nền vật lý cùng các quyền R/W/X và thuộc tính lưu đệm. TLB được hiện thực dạng tập-kết hợp (set-associative) với các khối vật lý cố định; chỉ số tập được tính bằng $VPN \bmod \text{số_tập}$, và khi một tập đầy thì khối ít được dùng gần nhất (LRU) bị thay thế. Quá trình dịch (translateAddress) diễn ra theo thứ tự: tra TLB

trước; nếu trượt thì tra bảng trang phần mềm và nạp lại kết quả vào TLB; nếu trang chưa được ánh xạ thì áp dụng cơ chế ánh xạ đồng nhất (identity) trả về địa chỉ vật lý bằng địa chỉ ảo — nhờ vậy các chương trình bare-metal không thiết lập bảng trang vẫn chạy được. Phân loại quyền được suy từ loại thao tác bus: nạp lệnh kiểm tra quyền thực thi (X), tải kiểm tra quyền đọc (R), lưu và thao tác nguyên tử kiểm tra quyền ghi (W); vi phạm quyền ném lỗi và được ghi nhận vào thống kê. Thuộc tính lưu đệm của mỗi truy cập được xác định kết hợp giữa quyền lưu đệm của trang và một vị từ `cacheabilityPredicate` do SoC cung cấp (đánh dấu các vùng MMIO và vùng thanh ghi DMA là không lưu đệm).

MMU thu thập thống kê chi tiết (số lần dịch, trúng/trượt TLB, trúng bảng trang, số lần dùng ánh xạ đồng nhất, số lần nạp lại/đuôi TLB, số lỗi quyền theo từng loại) và lưu lịch sử 32 lần dịch gần nhất; cả hai phục vụ khung quan sát MMU trên giao diện. Sau khi dịch, MMU chuyển tiếp yêu cầu xuống cổng dưới với địa chỉ đã thành địa chỉ vật lý, đồng thời đính kèm địa chỉ ảo gốc để đường phản hồi trả về đúng địa chỉ mà lỗi xử lý chờ. Vì MMU phục vụ hai luồng tách biệt (nạp lệnh và truy cập dữ liệu), nó giữ hai cổng dưới riêng (`instructionLowerPort`, `dataLowerPort`) tương ứng L1I và L1D (mục 3.3.2). Các tham số cấu hình (kích thước trang, kích thước/độ kết hợp TLB) được đọc từ `localStorage` của trình duyệt, cho phép thay đổi qua giao diện.



Hình 4.5: Minh họa cấu trúc MMU và đường dịch địa chỉ.

4.3.3. Phân cấp bộ nhớ đệm L1I/L1D/L2

Cả ba bộ nhớ đệm dùng chung một hiện thực trong lớp SimpleCache, chỉ khác nhau về tham số cấu hình. Bảng 4.6 trình bày cấu hình ba bộ nhớ đệm; lưu ý các giá trị độ trễ là số chu kỳ dùng để biểu diễn tương quan chi phí truy cập giữa các cấp [12], không phải số liệu định thời vật lý (mục 1.3.2).

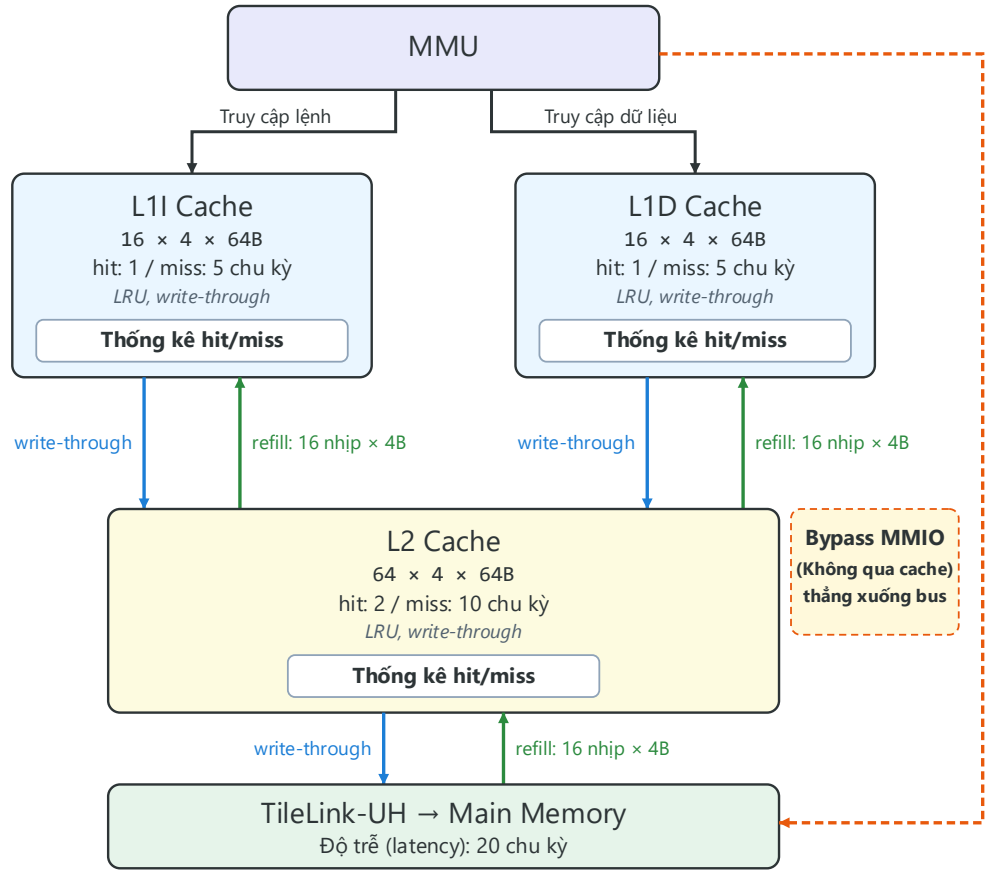
Bảng 4.6. Cấu hình bộ nhớ đệm L1I/L1D/L2

Tham số	L1I	L1D	L2
Số tập (set)	16	16	64
Số đường (way)	4	4	4

Tham số	L1I	L1D	L2
Kích thước khối (block)	64 byte	64 byte	64 byte
Dung lượng	4 KB	4 KB	16 KB
Độ trễ khi trúng (hit)	1 chu kỳ	1 chu kỳ	2 chu kỳ
Độ trễ khi trượt (miss)	5 chu kỳ	5 chu kỳ	10 chu kỳ
Chính sách ghi	write-through	write-through	write-through
Chính sách thay thế	LRU	LRU	LRU

Địa chỉ vật lý được phân rã thành ba phần theo cách quy ước: các bit thấp là độ dịch trong khối (offset, 6 bit với khối 64 byte), kế đến là chỉ số tập (4 bit với 16 tập của L1, 6 bit với 64 tập của L2), phần còn lại là thẻ (tag). Mỗi khối lưu cờ hợp lệ, cờ đã sửa đổi, thẻ, mốc tham chiếu LRU và một mảng byte dữ liệu. Khi nhận yêu cầu, nếu vùng địa chỉ không lưu đệm (vị từ isCacheable trả về sai, ví dụ vùng MMIO) hoặc bộ nhớ đệm bị tắt, yêu cầu được chuyển thẳng (bypass) xuống cấp dưới; ngược lại, bộ nhớ đệm tra khối theo tập và thẻ: nếu trúng thì phục vụ ngay với độ trễ trúng; nếu trượt thì chọn khối nạn nhân (LRU) và khởi động quá trình nạp lại (refill).

Quá trình nạp lại được hiện thực bất đồng bộ qua các trạng thái pendingFill/pendingResponse/pendingBurstResponse và tiến triển theo từng tick: bộ nhớ đệm phát một yêu cầu lấp khối xuống cấp dưới và nhận dữ liệu về theo từng nhịp 4 byte cho đến khi đủ một dòng khối (với khối 64 byte là 16 nhịp), rồi mới đánh dấu khối hợp lệ và phục vụ yêu cầu gốc. Chính sách ghi là write-through [11]: mỗi lần ghi trúng vừa cập nhật khối trong bộ nhớ đệm vừa ghi xuyên xuống cấp dưới; thao tác nguyên tử được phục vụ theo kiểu đọc–sửa–ghi tại chỗ rồi ghi xuyên xuống dưới. Bộ nhớ đệm thu thập thống kê số lần đọc/ghi, số lần trúng/trượt và tổng số chu kỳ, phục vụ khung quan sát Cache và việc đánh giá hiệu năng ở Chương 5.



Hình 4.6: Minh họa phân cấp bộ nhớ đệm trong hệ thống.

4.4. Hệ thống bus TileLink

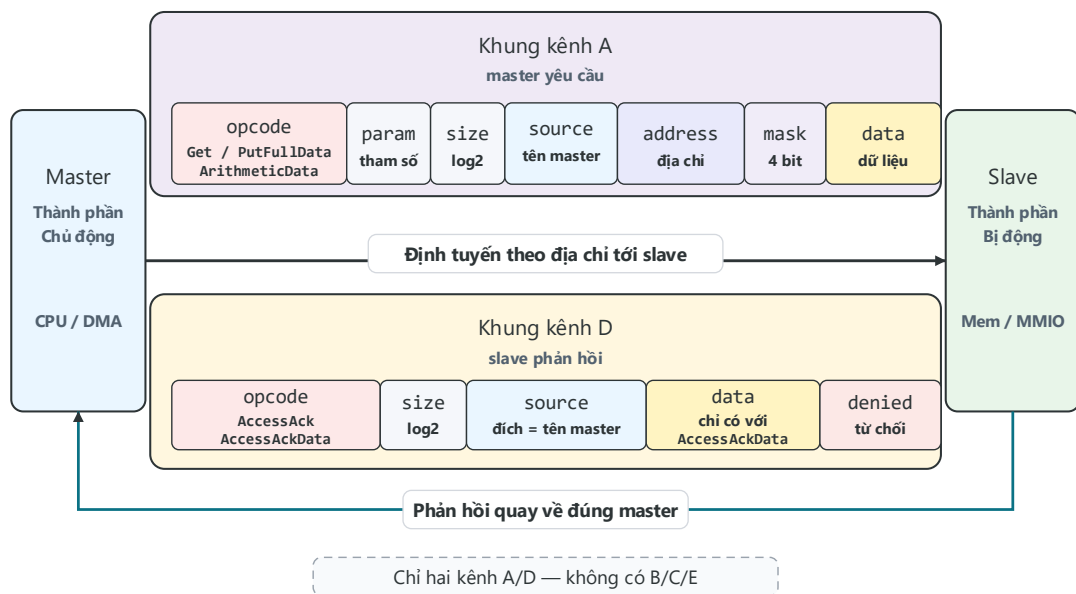
Hệ thống kết nối được hiện thực thành bốn lớp: một lõi định nghĩa dùng chung, một lớp cơ sở chứa logic định tuyến và hàng đợi, hai phân hệ bus UH/UL kế thừa lớp cơ sở, và cầu nối giữa hai bus. Phần lý thuyết giao thức TileLink đã trình bày ở mục 2.6; ở đây trình bày hiện thực bên trong.

4.4.1. Lỗi giao thức: mã thao tác, mặt nạ và ảnh chụp kênh

Lỗi giao thức định nghĩa mã thao tác kênh A (TL_A_Opcode: PutFullData=0, PutPartialData=1, ArithmeticData=2, LogicalData=3, Get=4, Intent=5) và kênh D (TL_D_Opcode: AccessAck=0, AccessAckData=1, HintAck=2, ...), cùng các tham số cho thao tác số học/logic (TL_Param_Arithmetic, TL_Param_Logical). Hàm computeTileLinkMask sinh mặt nạ byte (byte lane mask) theo địa chỉ và kích thước

truyền: 4 bit cho một từ 4 byte, hoặc một/hai bit cho truy cập byte/nửa từ tùy theo vị trí trong từ. Một nhóm hàm trợ giúp phân loại thao tác (đọc/ghi/nguyên tử), đọc/ghi giá trị theo kích thước, và áp dụng phép nguyên tử (applyTileLinkAtomic cho các phép cộng/min/max/and/or/xor/swap). Hai hàm snapshotAChannel/snapshotDChannel tạo ảnh chụp tức thời của hai kênh A/D dưới dạng tập tin hiệu (valid, opcode, param, size, source, address, mask, data...), phục vụ việc trực quan hóa giao dịch trên sơ đồ SoC (mục 4.7).

Cần nhấn mạnh, nhất quán với mục 1.3.2 và 2.6: hệ thống chỉ hiện thực mô hình giao dịch trên hai kênh A (yêu cầu) và D (phản hồi); không hiện thực các kênh B/C/E cùng cơ chế bảo đảm nhất quán bộ nhớ đệm của TileLink-C [2].



Hình 4.7: Minh họa cấu trúc một giao dịch trên hai kênh A và D.

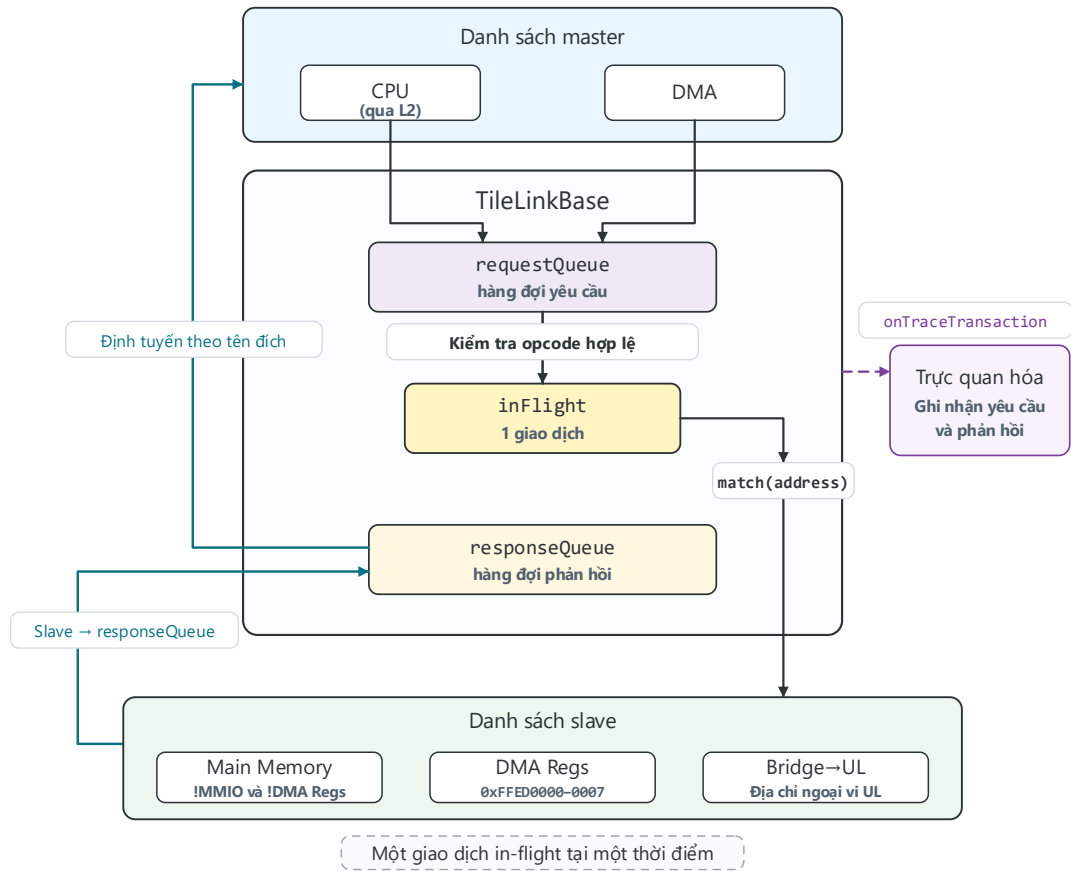
4.4.2. Định tuyến master/slave và mô hình giao dịch

Lớp cơ sở TileLinkBase chứa logic chung của hai bus. Mỗi bus giữ: một hàng đợi yêu cầu (requestQueue), một giao dịch đang xử lý (inFlight), một hàng đợi phản hồi (responseQueue), một sổ đăng ký master (masters, ánh xạ tên sang thành phần chủ động) và một danh sách slave (slaves, mỗi mục gồm tên, thành phần đích và một hàm

phán đoán địa chỉ match). Mỗi tick của bus xử lý một giao dịch tại một thời điểm: nếu chưa có giao dịch nào đang xử lý và hàng đợi yêu cầu không rỗng, bus lấy yêu cầu kế tiếp, kiểm tra tính hợp lệ của mã thao tác đối với phân hệ bus, chọn slave bằng cách tìm mục đầu tiên có match(địa chỉ) đúng, rồi chuyển yêu cầu tới slave đó; ở chiều phản hồi, bus lấy một phản hồi từ hàng đợi và định tuyến về đúng master theo tên đích. Giao dịch được coi là kết thúc khi phản hồi cuối cùng (lastBeat) đã được trả, khi đó inFlight được giải phóng cho giao dịch kế tiếp.

Lớp cơ sở còn cung cấp các hàm truy cập trực tiếp directRead/directWrite (chọn slave theo địa chỉ rồi đọc/ghi thẳng vào đích, dùng cho refill bộ nhớ đệm và cầu nối) cùng peek/poke (truy cập phục vụ gỡ lỗi không đếm là giao dịch). Một móc nối onTraceTransaction được gọi tại mỗi yêu cầu/phản hồi/truy cập trực tiếp để lớp trực quan hóa ghi nhận dấu vết giao dịch (mục 4.7).

Về tính "đa master": hệ thống có hai thành phần cùng đăng ký làm master và cùng phát yêu cầu là lõi xử lý (qua đường L2 → TileLink-UH) và bộ điều khiển DMA. Tuy nhiên, đúng như đã nêu ở mục 1.3.2 và 3.2.1, cơ chế điều phối hiện được mô hình hóa ở mức hàng đợi yêu cầu với một giao dịch được xử lý tại một thời điểm, chưa phải là một cơ chế phân xử (arbitration) đa master đầy đủ; báo cáo trình bày đặc điểm này một cách trung thực.



Hình 4.8: Minh họa cấu trúc bus và mô hình giao dịch.

4.4.3. Hai phân hệ bus TileLink-UH và TileLink-UL

Hai phân hệ bus kế thừa TileLinkBase và chỉ khác nhau ở tập mã thao tác được phép. TileLink-UH cho phép đủ sáu mã thao tác kênh A, kể cả thao tác số học và logic nguyên tử, phục vụ truy cập bộ nhớ chính, thanh ghi DMA và cầu nối; TileLink-UL chỉ cho phép bốn mã thao tác (đọc/ghi và lệnh gợi ý), phù hợp với các thiết bị ngoại vi đơn giản. Bảng 4.7 đối chiếu hai tập mã thao tác. Việc kiểm tra được thực hiện trong `_validateRequest` của lớp cơ sở: yêu cầu dùng mã thao tác không nằm trong tập cho phép của phân hệ bus sẽ bị từ chối kèm thông báo lỗi.

Bảng 4.7. Mã thao tác kênh A được phép trên TileLink-UH và TileLink-UL

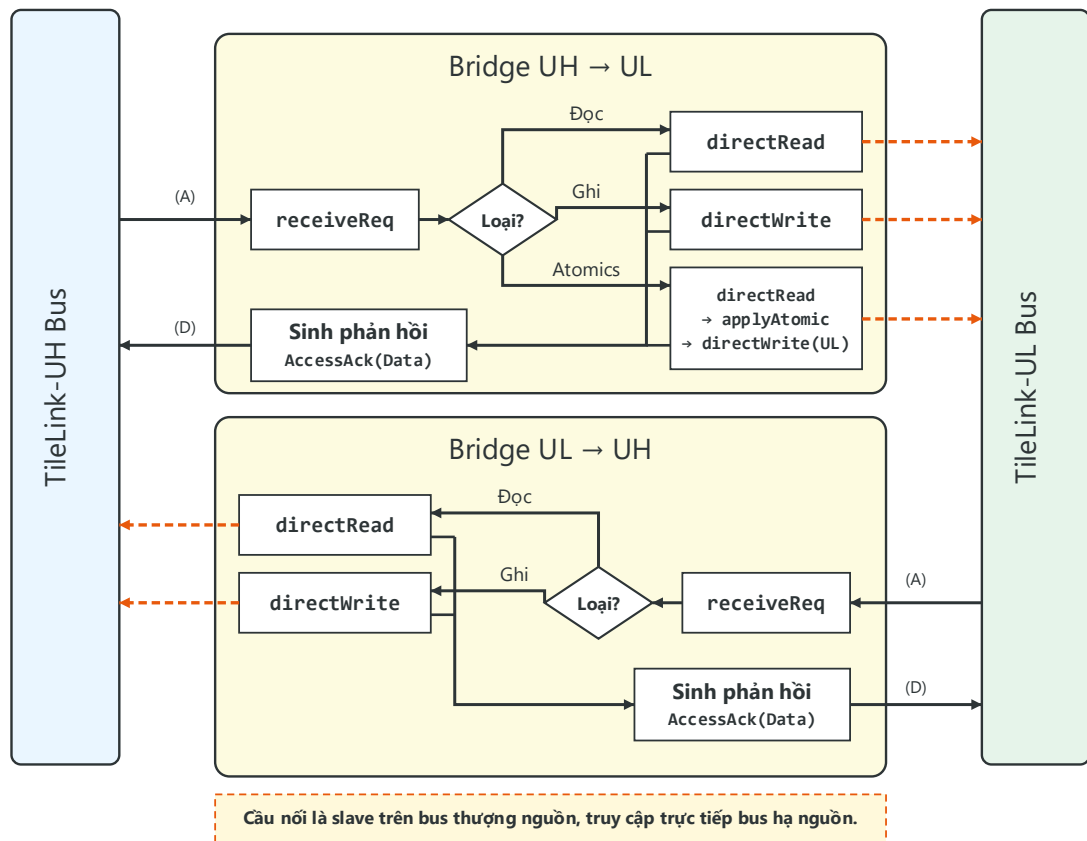
Mã thao tác kênh A	TileLink-UH	TileLink-UL
PutFullData (ghi đầy đủ)	✓	✓

Mã thao tác kênh A	TileLink-UH	TileLink-UL
PutPartialData (ghi một phần)	✓	✓
Get (đọc)	✓	✓
Intent (gợi ý)	✓	✓
ArithmeticData (nguyên tử số học)	✓	—
LogicalData (nguyên tử logic)	✓	—

Cần lưu ý về nhãn hiển thị trên giao diện: bus UH được dán nhãn "Coherent Bus (High-Speed)" và bus UL được dán nhãn "Low-Speed/MMIO" trong khung sơ đồ. Tuy nhiên, như đã phân tích ở mục 2.6 và 3.2.1, đây là mô hình ở mức giao dịch trên hai kênh A/D, không phải bus có nhất quán bộ nhớ đệm theo nghĩa đầy đủ của TileLink-C [2]; trong báo cáo, UH được hiểu là bus hiệu năng cao (cho phép nguyên tử và truyền chuỗi) và UL là bus nhẹ cho ngoại vi.

4.4.4. Cầu nối UH↔UL

Cầu nối được hiện thực trong lớp TileLinkBridge. Mỗi cầu nối có một bus thượng nguồn và một bus hạ nguồn; SoC tạo hai cầu nối một chiều ghép lại: một cầu UH→UL (để giao dịch từ phía bộ nhớ/lõi xử lý tới được ngoại vi) và một cầu UL→UH (để DMA trên bus UL truy cập được bộ nhớ chính). Cầu nối được đăng ký như một slave trên bus thượng nguồn: khi nhận một yêu cầu, nó không đẩy lại vào hàng đợi của bus hạ nguồn mà gọi trực tiếp directRead/directWrite của bus hạ nguồn để hoàn tất giao dịch ngay, rồi sinh phản hồi tương ứng (AccessAckData cho đọc, AccessAck cho ghi) trả về bus thượng nguồn. Với yêu cầu nguyên tử, cầu nối thực hiện đọc–sửa–ghi (read-modify-write) bằng cách đọc giá trị hiện tại từ bus hạ nguồn, áp dụng phép nguyên tử rồi ghi lại.



Hình 4.9: Minh họa luồng giao dịch qua cầu nối.

4.5. Bộ điều khiển DMA

Bộ điều khiển truy cập bộ nhớ trực tiếp được hiện thực bằng ba lớp phối hợp: DMADescriptor (mô tả truyền), DMARegisters (mô hình thanh ghi và hàng đợi mô tả), và DMAController (máy trạng thái truyền).

Mô tả truyền. Một DMADescriptor gồm ba từ 32 bit: địa chỉ nguồn (sourceAddr), địa chỉ đích (destAddr) và từ cấu hình (configWord). Từ cấu hình đóng gói nhiều trường: số phần tử cần truyền numElements (24 bit thấp), cờ đảo byte bswap (bit 27), chế độ địa chỉ nguồn srcMode (bit 28–29) và chế độ địa chỉ đích dstMode (bit 30–31). Mỗi chế độ địa chỉ vừa quy định cách tiến địa chỉ vừa quy định kích thước phần tử: chế độ 0 (địa chỉ cố định, phần tử 1 byte), chế độ 1 (cố định, 4 byte), chế độ 2 (tăng dần theo byte, phần tử 1 byte), chế độ 3 (tăng dần theo từ, phần tử 4 byte).

Bản đồ thanh ghi. Bộ điều khiển trình diện hai thanh ghi ánh xạ bộ nhớ trong vùng 0xFFED0000: thanh ghi điều khiển/trạng thái CTRL tại 0xFFED0000 và thanh ghi nạp mô tả DESC tại 0xFFED0004. Bảng 4.8 trình bày bố cục bit. Ghi đủ ba từ liên tiếp vào DESC sẽ ghép thành một mô tả và đẩy vào hàng đợi mô tả (FIFO) sâu 8 mục; ghi CTRL với bit cho phép và bit khởi động sẽ kích hoạt truyền.

Bảng 4.8. Bản đồ thanh ghi DMA

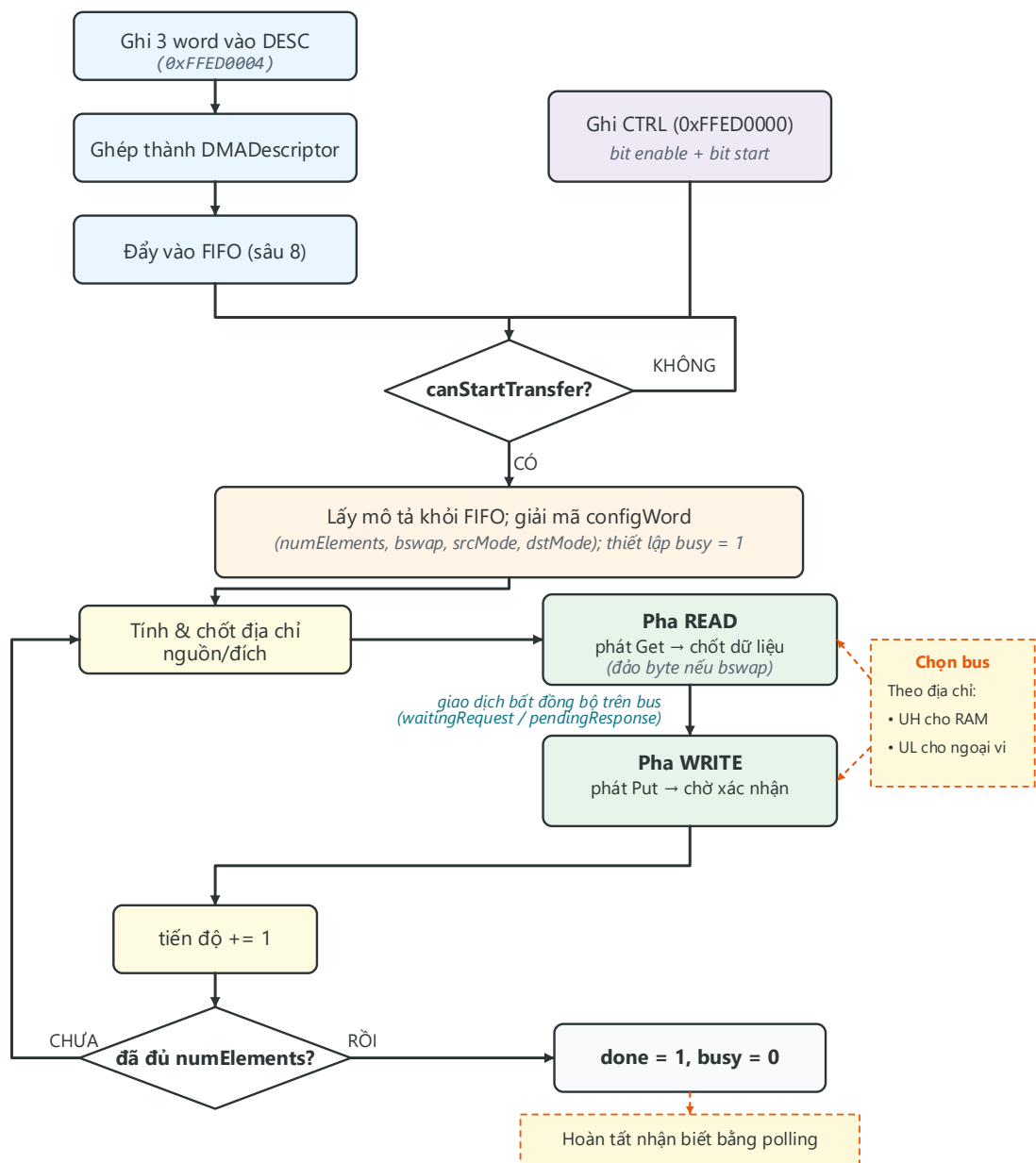
Thanh ghi	Offset	Bit	Ý nghĩa
CTRL (đọc)	0xFFED0000	0	đã cho phép (enabled)
		1	đã yêu cầu khởi động (start requested)
		2	hoàn tất (done)
		19:16	log2(độ sâu FIFO)
		27	FIFO rỗng
		28	FIFO đầy
		29	lỗi (error)
		30	hoàn tất (done)
		31	đang bận (busy)
CTRL (ghi)	0xFFED0000	0	cho phép/tắt (tắt sẽ reset)
		1	khởi động truyền
		27	xác nhận và xóa cờ done/error
DESC (ghi)	0xFFED0004	31:0	một từ mô tả (ghi đủ 3 từ → một mô tả vào FIFO)

Máy trạng thái truyền. Mỗi tick, nếu điều kiện khởi động thỏa (đã cho phép, không bận, FIFO không rỗng, đã yêu cầu khởi động), bộ điều khiển lấy mô tả kế tiếp khỏi FIFO, giải mã cấu hình và chuyển sang trạng thái bận. Việc truyền diễn ra theo từng phần tử với hai pha đọc và ghi, mỗi pha là một giao dịch bất đồng bộ trên bus (tương tự mô hình của lõi xử lý ở mục 4.2.2): với mỗi phần tử, bộ điều khiển tính địa chỉ nguồn/đích theo chế độ địa chỉ và chốt lại (latch); pha đọc phát một Get tới địa chỉ nguồn rồi chờ phản hồi, chốt dữ liệu nhận được (có đảo byte nếu cờ bswap bật); pha ghi phát một Put ghi dữ liệu đã chốt tới địa chỉ đích rồi chờ xác nhận, sau đó tăng tiến

độ. Khi đã truyền đủ số phần tử, bộ điều khiển đặt cờ done, hạ cờ busy và gọi hàm gọi lại nếu có.

Một điểm thiết kế đáng nêu là DMA là master trên cả hai phân hệ bus: với mỗi địa chỉ, một hàm chọn bus (`selectLinkForAddress`) định tuyến giao dịch sang TileLink-UL nếu địa chỉ thuộc vùng ngoại vi, ngược lại sang TileLink-UH (cho bộ nhớ chính). Nhờ vậy, một lần truyền có thể di chuyển dữ liệu giữa bộ nhớ chính (trên UH) và một thiết bị ngoại vi như ma trận LED (trên UL). Việc đọc/ghi thanh ghi điều khiển của DMA lại đến từ phía bus UH (DMA là slave trên UH cho vùng `0xFFED0000`); yêu cầu này được hoãn lại (`pendingRegReq`) và phục vụ trong tick kế tiếp để trả phản hồi đúng cơ chế bus.

Đúng như giới hạn đã nêu ở mục 1.3.2, bộ điều khiển DMA không phát tín hiệu ngắt tới lõi xử lý; chương trình nhận biết hoàn tất truyền bằng cách đọc thăm dò (polling) bit done/busy của thanh ghi CTRL.



Hình 4.10: Mô tả quy trình truyền của bộ điều khiển DMA.

4.6. Hệ thống ngoại vi và MMIO

4.6.1. Tổng quan kiến trúc ngoại vi

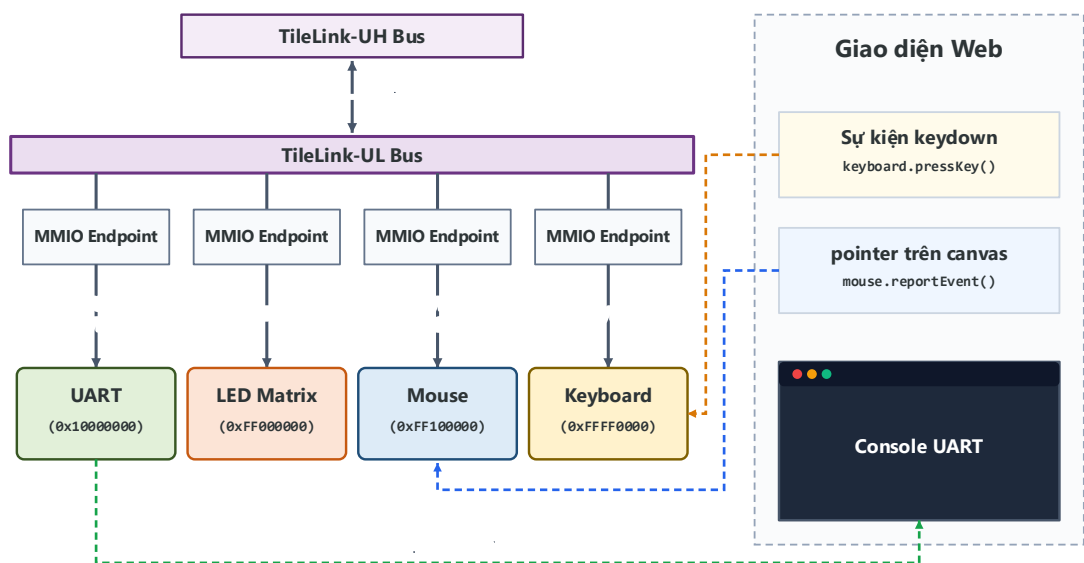
Hệ thống có bốn thiết bị ngoại vi gắn vào TileLink-UL: UART, ma trận LED, bàn phím và chuột. Mỗi thiết bị được trình diện trên bus qua một endpoint MMIO tạo bởi hàm createMMIOEndpoint: endpoint này nhận yêu cầu bus, gọi hàm đọc/ghi tương

ứng của thiết bị, rồi trả phản hồi AccessAckData/AccessAck. Cách bố trí này tách bạch logic thiết bị (trong từng mô-đun ngoại vi) khỏi việc đấu nối bus. Bảng 4.9 liệt kê bốn thiết bị ngoại vi cùng vùng địa chỉ; để tham chiếu đầy đủ, vùng thanh ghi DMA cũng được liệt kê dù DMA không phải thiết bị ngoại vi theo nghĩa thông thường.

Bảng 4.9. Danh sách thiết bị ngoại vi và vùng địa chỉ

Thiết bị	Địa chỉ nền	Kích thước	Bus	Mô tả ngắn
UART	0x10000000	20 byte (0x14)	TileLink-UL	Giao tiếp nối tiếp dạng console (TX/RX/STATUS/CTRL/BAUD)
Ma trận LED	0xFF000000	4096 byte (32×32×4)	TileLink-UL	Hiển thị lưới 32×32 điểm ảnh từ VRAM
Chuột	0xFF100000	20 byte (0x14)	TileLink-UL	Tọa độ con trỏ và trạng thái nút
Bàn phím	0xFFFF0000	8 byte (0x08)	TileLink-UL	Bộ đệm ký tự nhập (đọc thăm dò)
(Thanh ghi DMA)	0xFFED0000	8 byte (0x08)	TileLink-UH	CTRL/DESC của bộ điều khiển DMA (mục 4.5)

Lớp giao diện nối các sự kiện đầu vào của trình duyệt vào thiết bị tương ứng: sự kiện gõ phím được chuyển thành mã ký tự đưa vào bộ đệm bàn phím; sự kiện con trỏ trên vùng canvas LED được chuyển thành tọa độ/nút cho thiết bị chuột; và kết quả truyền của UART được hiển thị ra khung console.



Hình 4.11: Minh họa kiến trúc ngoại vi và các đường sự kiện từ giao diện.

4.6.2. UART

Thiết bị UART trình diện năm thanh ghi trong vùng 20 byte kể từ địa chỉ nền 0x10000000. Bảng 4.10 trình bày bản đồ thanh ghi.

Bảng 4.10. Bản đồ thanh ghi UART

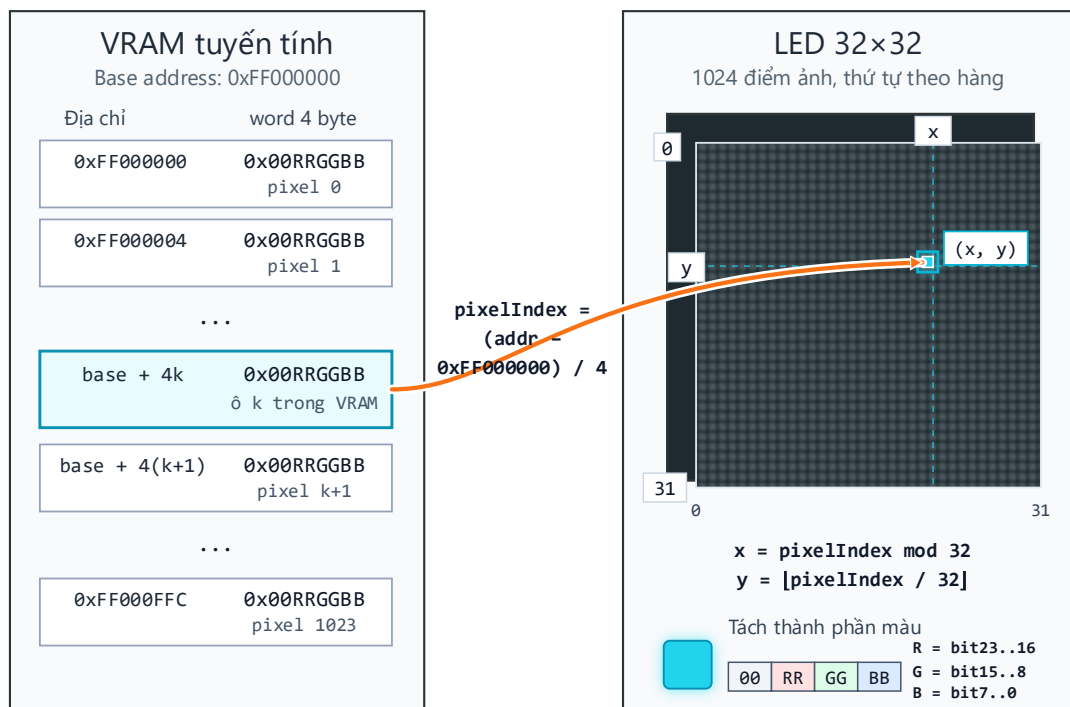
Thanh ghi	Offset	Quyền	Ý nghĩa
TX	0x00	Ghi	Ghi một ký tự để truyền đi
RX	0x04	Đọc	Đọc và tiêu thụ một ký tự từ bộ đệm nhận
STATUS	0x08	Đọc	Bit 0: TX sẵn sàng; bit 1: RX có dữ liệu; bit 2: cho phép ngắt TX; bit 3: cho phép ngắt RX
CTRL	0x0C	Đọc/Ghi	Bit 0: cho phép ngắt TX; bit 1: cho phép ngắt RX
BAUD	0x10	Đọc/Ghi	Hệ số chia (divisor) tốc độ truyền

Việc truyền được mô hình hóa có tính tới tốc độ baud: thanh ghi BAUD lưu hệ số chia, và tốc độ baud được tính bằng $\text{tần_số_ngoại_vi} / (\text{lấy_mẫu_vượt} \times \text{hệ_số_chia})$, với tần số ngoại vi 48 MHz và hệ số lấy mẫu vượt (oversampling) 16 — hệ số chia mặc định 26 cho tốc độ xấp xỉ 115200 baud. Khi chương trình ghi một ký tự vào TX,

thiết bị đặt cờ TX-bận và tính số chu kỳ cần để truyền xong một khung (mười bit: một bit start, tám bit dữ liệu, một bit stop) dựa trên tần số CPU 100 MHz; cờ TX-sẵn-sàng chỉ được dựng lại sau khi đếm hết số chu kỳ này trong các lần tick. Nhờ đó, mô hình phản ánh được việc UART không thể truyền tức thời mà tốn thời gian tỉ lệ nghịch với tốc độ baud. Các bit cho phép ngắt trong thanh ghi CTRL/STATUS hiện chỉ tồn tại ở mức cờ trạng thái; hệ thống chưa có đường ngắt thực sự tới lõi xử lý (mục 1.3.2), nên chương trình giao tiếp UART theo cơ chế đọc thăm dò thanh ghi STATUS.

4.6.3. Ma trận LED

Ma trận LED mô phỏng một bảng 32×32 điểm ảnh với bộ nhớ hiển thị riêng (VRAM) là một mảng Uint32Array 1024 phần tử, ánh xạ vào vùng 4096 byte kể từ địa chỉ nền 0xFF000000. Mỗi điểm ảnh chiếm 4 byte và lưu màu theo định dạng 0x00RRGGBB. Khi nhận một thao tác ghi từ (writeWord), thiết bị chuẩn hóa địa chỉ về dạng không dấu, tính độ dịch so với địa chỉ nền, suy ra chỉ số điểm ảnh $\text{pixelIndex} = \text{offset} / 4$, rồi từ đó tính tọa độ $x = \text{pixelIndex} \bmod 32$ và $y = \lfloor \text{pixelIndex} / 32 \rfloor$; giá trị màu được tách thành ba thành phần R/G/B và vẽ lên phần tử canvas. Để hiệu quả, thiết bị chỉ vẽ lại đúng điểm ảnh vừa thay đổi (cập nhật từng phần) thay vì vẽ lại toàn bộ; điểm ảnh sáng được vẽ kèm hiệu ứng phát sáng (glow) qua thuộc tính bóng đổ của canvas.



Hình 4.12: Minh họa ánh xạ giữa VRAM và điểm ảnh.

4.6.4. Bàn phím và chuột

Thiết bị bàn phím tuân theo quy ước MMIO của môi trường RARS [16], trình diện hai thanh ghi chỉ-đọc trong vùng 8 byte kể từ 0xFFFF0000: thanh ghi điều khiển CTRL tại offset 0x00 (bit 0 báo "có dữ liệu", bằng 1 khi bộ đệm không rỗng) và thanh ghi dữ liệu DATA tại offset 0x04 (đọc sẽ lấy ký tự cũ nhất khỏi bộ đệm). Sự kiện gõ phím trên giao diện đầy mã ký tự vào bộ đệm; chương trình đọc thăm dò CTRL chờ bit sẵn sàng rồi đọc DATA. Thiết bị chuột trình diện năm thanh ghi trong vùng 20 byte kể từ 0xFF100000. Bảng 4.11 trình bày bản đồ thanh ghi của cả hai thiết bị.

Bảng 4.11. Bản đồ thanh ghi bàn phím và chuột

Thiết bị	Thanh ghi	Offset	Quyền	Ý nghĩa
Bàn phím	CTRL	0x00	Đọc	Bit 0: bộ đệm có dữ liệu (sẵn sàng)
	DATA	0x04	Đọc	Lấy mã ASCII ký tự cũ nhất khỏi bộ đệm
Chuột	X	0x00	Đọc	Tọa độ X gần nhất (theo điểm ảnh canvas)

Thiết bị	Thanh ghi	Offset	Quyền	Ý nghĩa
	Y	0x04	Đọc	Tọa độ Y gần nhất
	BTN	0x08	Đọc	Bitmap nút: bit 0 trái, bit 1 phải, bit 2 giữa
	STATUS	0x0C	Đọc/Ghi	Bit 0: có sự kiện di chuyển; bit 1: có nhấp/đổi nút (ghi để xóa cờ)
	CTRL	0x10	Đọc/Ghi	Dành riêng (cho phép ngắt trong tương lai)

Thiết bị chuột nhận tọa độ và trạng thái nút từ sự kiện con trỏ trên vùng canvas LED (qua reportEvent): tọa độ được cắt về 16 bit, mặt nạ nút lấy ba bit thấp, và cờ trạng thái được dựng theo loại sự kiện (di chuyển hoặc nhấp). Việc ghi vào thanh ghi STATUS có ngữ nghĩa ghi-1-đề-xóa (xóa đúng các bit được đặt trong giá trị ghi), cho phép chương trình xác nhận đã xử lý sự kiện. Cũng như các ngoại vi khác, bàn phím và chuột được giao tiếp theo cơ chế đọc thăm dò.

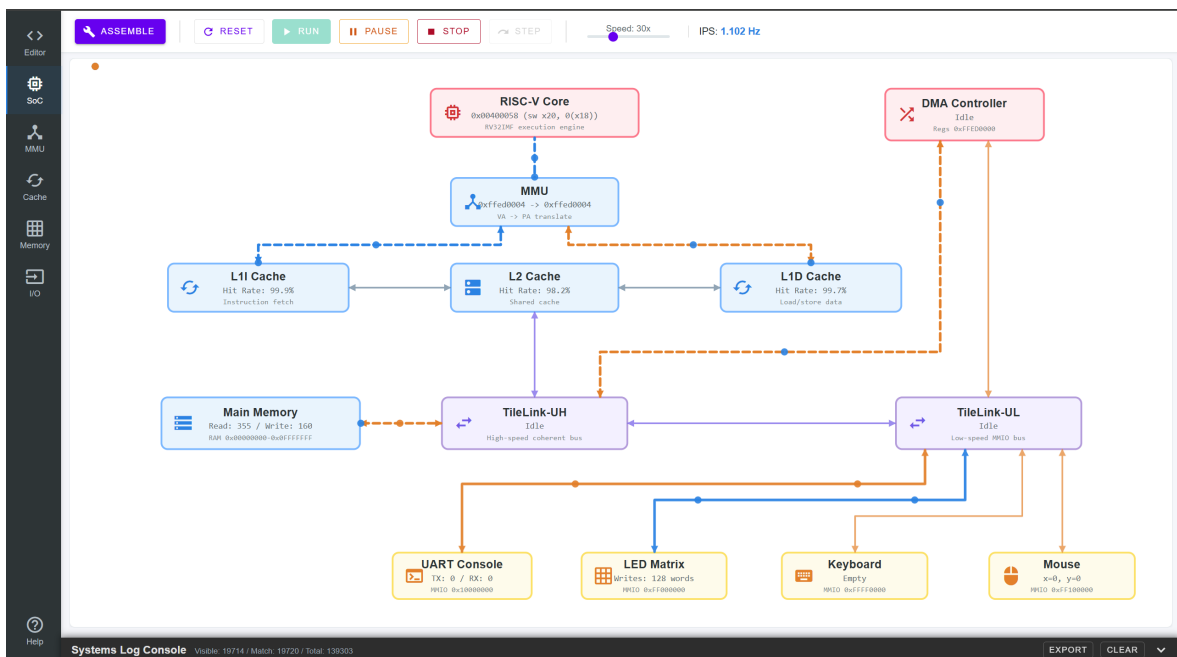
Về thiết bị truyền thông CAN: tuy được nhắc tới trong định hướng ban đầu, hệ thống hiện chưa có mô-đun/mã nguồn cho CAN — đây là nội dung chưa xác định từ source hiện tại và chỉ được đề ngỏ cho hướng phát triển (Chương 6).

4.7. Trực quan hóa và gỡ lỗi

4.7.1. Sơ đồ SoC động

Sơ đồ SoC động được hiện thực trong mô-đun soc_diagram.js. Cấu trúc sơ đồ được mô tả khai báo bằng hai bảng dữ liệu: danh sách nút (SOC_NODES, mỗi nút gồm tên, tọa độ, kích thước, nhóm màu, biểu tượng, mô tả, chú giải bay (tooltip) và liên kết tới khung nhìn tương ứng) và danh sách cạnh (SOC_EDGES, mỗi cạnh gồm điểm đầu/cuối theo dạng nút:cổng, nhóm bus và các điểm gãy để định tuyến). Sơ đồ được kết xuất thành SVG với khung nhìn (viewBox) cố định; mỗi khối được vẽ bằng phần tử foreignObject chứa biểu tượng và nhãn trạng thái, và mỗi cạnh được vẽ bằng một đường gấp khúc trực giao (orthogonal) sinh bởi hàm generateOrthogonalPath (ghép các đoạn ngang/dọc qua các điểm gãy).

Khi mô phỏng chạy, lớp khởi tạo trung tâm ghi nhận dấu vết giao dịch trên các đường nối (qua móc onTraceTransaction ở mục 4.4.2 và lớp bao công), lưu vào một cấu trúc trace gồm danh sách đường đang hoạt động (mỗi đường có thời điểm hết hiệu lực) và hàng đợi giao dịch gần nhất. Hàm updateSocTraceHighlights đọc cấu trúc này để làm nổi bật các đường đang có giao dịch: đường được tô lớp active (và active-write nếu là giao dịch ghi), đồng thời một chấm tròn chạy dọc theo đường (hiệu ứng "pulse" bằng animateMotion, kéo dài 0,65 giây và được tiết chế tần suất để tránh quá dày) minh họa hướng di chuyển của giao dịch. Mỗi khối còn hiển thị trạng thái tóm tắt (ví dụ bộ đếm chương trình của lõi xử lý, tỉ lệ trúng của bộ nhớ đệm) và có chú giải bay khi trở tới. Lưu ý nhãn chú giải của nút bus UH ghi "high-speed coherent bus" mang tính mô tả giao diện; bản chất là mô hình giao dịch như đã nêu ở mục 4.4.3.



Hình 4.13: Minh họa khung nhìn sơ đồ SoC khi có giao dịch trên bus.

4.7.2. Nhật ký hệ thống và các khung quan sát trạng thái

Nhật ký hệ thống được hiện thực bằng cách thay thế (wrap) các hàm console.log/warn/error/info để mọi thông điệp nhật ký do các mô-đun phát ra đều được bắt lại, gắn nhãn rồi đưa vào một kho nhật ký. Mỗi dòng nhật ký được phân loại

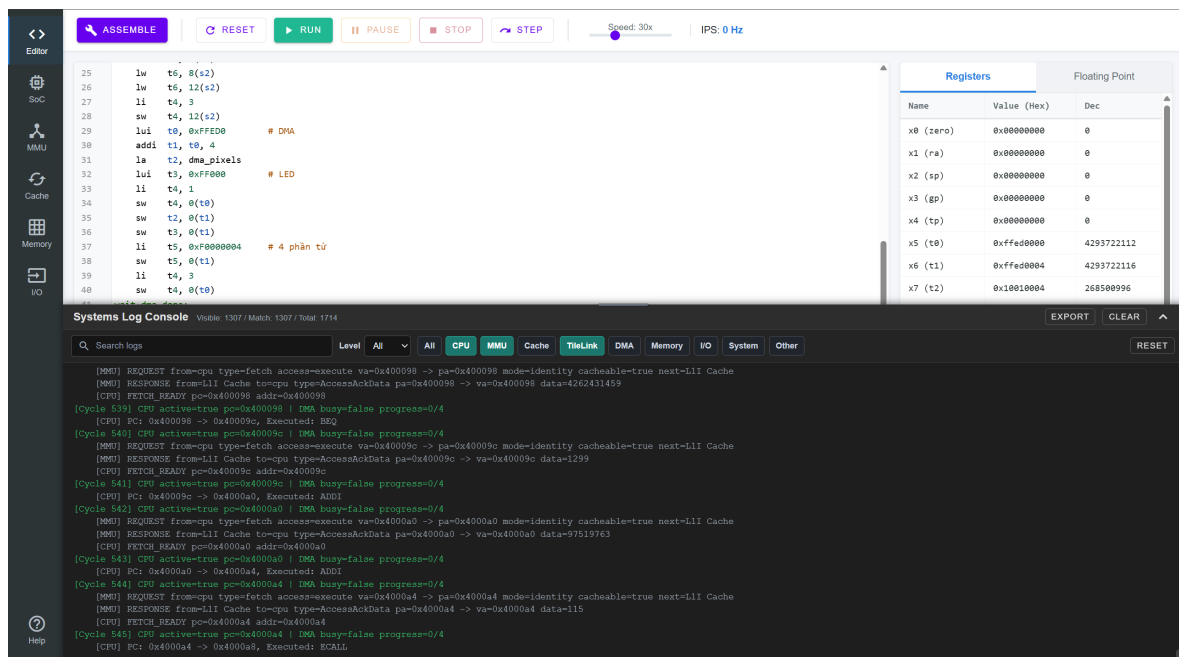
vào một hoặc nhiều nhóm trong số chín nhóm bằng hàm `inferLogModules`, dựa trên nhãn trong ngoặc vuông ở đầu dòng và nội dung văn bản; nhóm chính của dòng là nhóm không phải "khác" đầu tiên. Bảng 4.12 liệt kê chín nhóm và tiêu chí phân loại.

Bảng 4.12. Các nhóm nhật ký và tiêu chí phân loại

Nhóm	Tiêu chí phân loại (rút gọn)
cpu	Dòng bắt đầu bằng [Cycle ...] hoặc chứa từ khóa "cpu"
mmu	Chứa "mmu"
cache	Chứa "cache" hoặc dạng L1I/L1D/L2 cache
tilelink	Chứa "tilelink" (kể cả tilelink-uh/-ul)
dma	Chứa "dma"
memory	Chứa "main memory"
io	Chứa "uart"/"keyboard"/"mouse"/"led matrix"/"io map"
system	Reset, "simulation halted", lỗi assemble/run/step, hoặc liên quan syscall/arch/ui
other	Không khớp nhóm nào ở trên

Kho nhật ký hỗ trợ cơ chế đăng ký (subscribe) để giao diện cập nhật tăng dần, cùng các thao tác chụp nhanh, xóa và xuất văn bản; lịch sử được giới hạn (cắt bớt khi vượt ngưỡng) để kiểm soát bộ nhớ. Trên giao diện, terminal nhật ký cho phép lọc theo nhóm mô-đun, theo mức (log/warn/error) và theo chuỗi tìm kiếm; để giữ hiệu năng khi số dòng lớn, giao diện giới hạn số dòng hiển thị trong DOM và nạp theo từng đợt qua cơ chế vẽ lại theo khung hình.

Ngoài terminal nhật ký, giao diện còn có hai khung quan sát chuyên biệt phản ánh trạng thái nội bộ của hệ thống bộ nhớ: khung MMU (hiển thị tổng quan và cấu hình, bản đồ địa chỉ, bảng trang phần mềm, bảng TLB và lịch sử dịch địa chỉ) và khung Cache (hiển thị trạng thái từng tập/đường cùng thống kê trùng/trượt của L1I, L1D, L2). Hai khung này lấy dữ liệu trực tiếp từ các đối tượng MMU và SimpleCache đã trình bày ở mục 4.3.



Hình 4.14: Minh họa terminal nhật ký với bộ lọc theo mô-đun.

4.8. Vòng điều khiển thực thi trên giao diện

Toàn bộ tương tác điều khiển thực thi được hiện thực trong mô-đun giao diện javascript.js, đóng vai trò cầu nối giữa các nút điều khiển và lõi mô phỏng. Trạng thái chạy được giữ trong đối tượng runState (gồm các cờ đang chạy/tạm dừng, định danh khung hình, bộ đếm chu kỳ, giới hạn chu kỳ tối đa 500.000, bảng địa chỉ điểm dừng, và các biến đo tốc độ).

Các thao tác điều khiển handleAssemble khởi tạo lại SoC, thiết lập các hàm gọi lại dịch vụ hệ thống và UART, gọi trình biên dịch rồi nạp chương trình; handleRun bắt đầu chạy liên tục; handlePause tạm dừng/tiếp tục; handleStop dừng hẳn; handleStep tiến đúng một lệnh; handleReset đưa hệ thống về trạng thái đầu nhưng giữ nguyên mã nguồn trong trình soạn thảo.

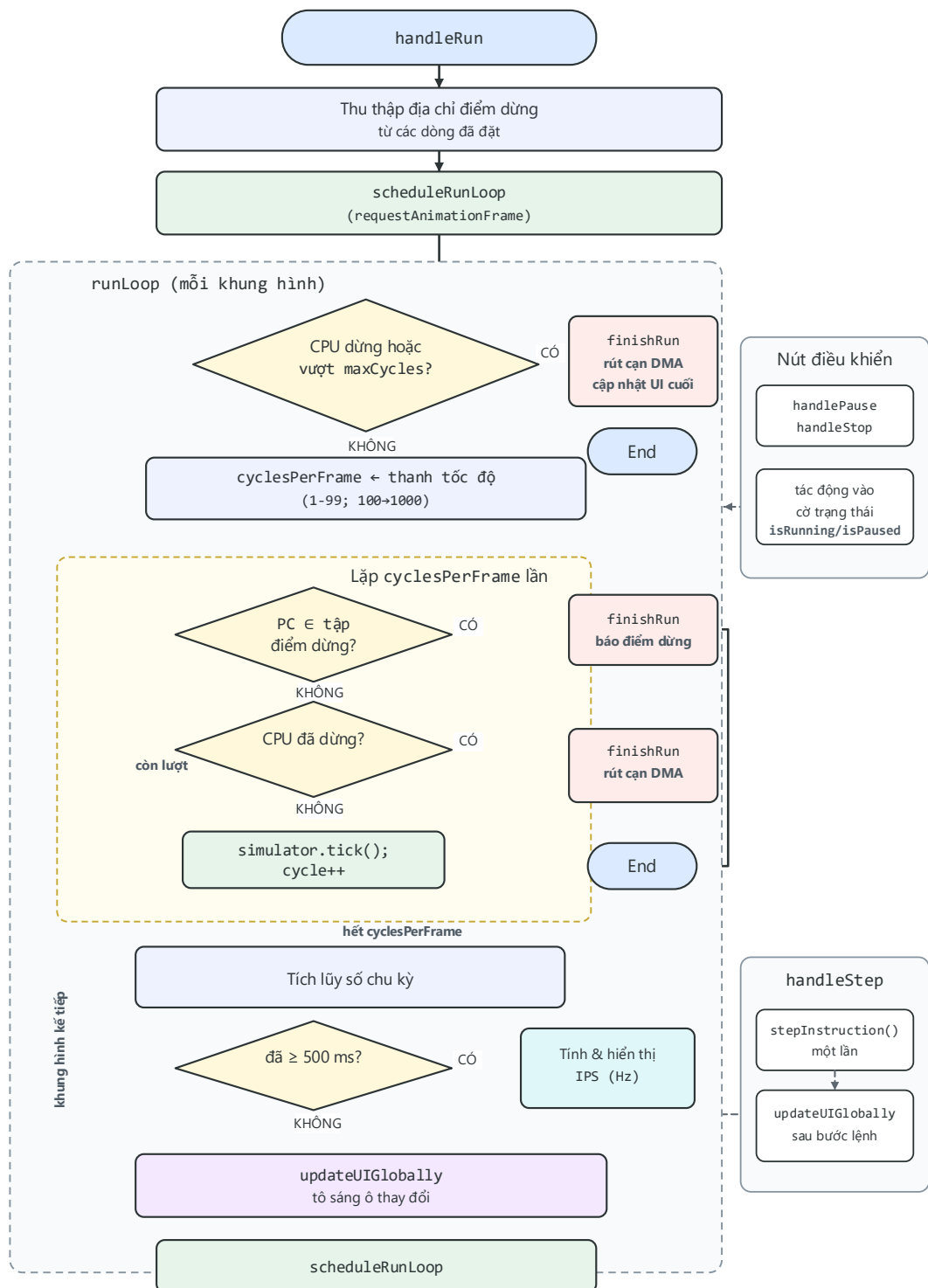
Chế độ chạy liên tục dùng cơ chế vẽ lại theo khung hình của trình duyệt (requestAnimationFrame): hàm runLoop được gọi mỗi khung hình và thực thi một số chu kỳ nhất định. Số chu kỳ mỗi khung hình (cyclesPerFrame) lấy theo giá trị thanh trượt tốc độ (1–99); riêng vị trí cao nhất của thanh trượt (giá trị 100) được ánh xạ

thành 1000 chu kỳ mỗi khung hình để cho phép chạy nhanh. Trong vòng lặp nội bộ của mỗi khung hình, trước mỗi chu kỳ hệ thống kiểm tra điều kiện dừng: gặp điểm dừng tại địa chỉ hiện hành, lỗi xử lý dừng, hoặc vượt giới hạn chu kỳ tối đa. Sau mỗi khung hình, hệ thống cập nhật toàn bộ giao diện và lên lịch khung hình kế tiếp.

Người học đặt điểm dừng theo dòng trên cột đặt điểm dừng của trình soạn thảo (gutter của CodeMirror). Khi bắt đầu chạy, hàm `collectBreakpointAddresses` chuyển tập dòng có điểm dừng thành tập địa chỉ lệnh (dựa trên bố trí địa chỉ của trình biên dịch), nhờ đó vòng lặp chỉ cần kiểm tra bộ đếm chương trình hiện hành có nằm trong tập địa chỉ điểm dừng hay không.

Đo tốc độ thực thi. Hệ thống tích lũy số chu kỳ đã thực thi và, mỗi khi khoảng thời gian thực trôi qua đạt ít nhất 500 ms, tính tốc độ thực thi ước lượng bằng số chu kỳ chia cho thời gian thực rồi quy về mỗi giây, hiển thị ở chỉ số IPS trên thanh công cụ (đơn vị Hz). Chỉ số này phản ánh số chu kỳ mô phỏng thực thi trên mỗi giây thời gian thực, dùng làm thước đo tương đối của tốc độ tương tác.

Cập nhật giao diện có làm nổi bật thay đổi. Hàm `updateUIGlobally` vẽ lại các khung quan sát (thanh ghi số nguyên/dấu phẩy động, bộ nhớ lệnh, vùng dữ liệu, Cache, MMU, sơ đồ SoC). Khi một ô thanh ghi có giá trị khác lần vẽ trước, ô đó được gán lớp làm nổi bật (highlight) và lớp này được gỡ sau khoảng 500 ms, giúp người học dễ nhận ra thay đổi sau mỗi bước thực thi. Khi dừng chạy, nếu DMA còn giao dịch dở dang, hệ thống tiếp tục tiến chu kỳ cho đến khi DMA hoàn tất trước khi cập nhật giao diện lần cuối.



Hình 4.15: Minh họa vòng lặp chạy mô phỏng trên giao diện.

4.9. Một số thách thức hiện thực và cách xử lý

Mục này tổng hợp một số vấn đề kỹ thuật phát sinh khi hiện thực và cách hệ thống đã giải quyết; các giới hạn về phạm vi đã được nêu trực tiếp tại từng mô-đun ở trên và được tổng hợp ở Chương 5–6.

Nhân 64 bit để lấy phần cao. JavaScript không có kiểu số nguyên 64 bit gốc và số thực 64 bit không biểu diễn chính xác tích của hai số 32 bit. Hệ thống dùng BigInt để tính tích chính xác rồi dịch phải 32 bit cho nhóm lệnh mulh*, đồng thời dùng Math.imul cho phần thấp mul (mục 4.2.3).

Mô hình bộ nhớ bất đồng bộ giữ nguyên PC. Vì một truy cập bộ nhớ trải dài nhiều chu kỳ, lõi xử lý và bộ điều khiển DMA dùng chung mô hình yêu cầu–chờ–phản hồi–thực hiện lại: lệnh/giao dịch chưa hoàn tất giữ nguyên con trỏ chương trình (hoặc tiến độ) và được "thực hiện lại" khi phản hồi về (mục 4.2.2, 4.5).

Tính mặt nạ byte cho ghi một phần. Ghi byte/nửa từ cần xác định đúng các làn byte (byte lane) bị tác động; hàm computeTileLinkMask sinh mặt nạ theo địa chỉ và kích thước để mô hình ghi một phần đúng vị trí (mục 4.4.1).

Ghi little-endian nhất quán. Trình biên dịch ghi mã máy, bộ nhớ và các thiết bị đều thống nhất thứ tự byte nhỏ trước, qua các hàm đọc/ghi theo kích thước dùng chung (mục 4.1.1, 4.3.1).

Biểu diễn IEEE 754 bằng Float32Array/DataView. Việc dùng Float32Array cho tệp thanh ghi dấu phẩy động bảo đảm độ chính xác đơn, còn ArrayBuffer/DataView cho phép thao tác trực tiếp trên bit để hiện thực các lệnh sao chép dấu, di chuyển bit và phân loại (mục 4.2.3).

Mô phỏng đơn luồng vẫn giữ giao diện đáp ứng. Vòng lặp chạy chia công việc theo từng khung hình requestAnimationFrame và đếm nhật ký theo đợt, nhờ đó mô phỏng tiến triển mà giao diện vẫn mượt và terminal nhật ký không bị nghẽn khi khối lượng thông điệp lớn (mục 4.7.2, 4.8).

Tổng hợp lại, Chương 4 đã trình bày hiện thực chi tiết của toàn bộ các thành phần đã thiết kế ở Chương 3: trình biên dịch hợp ngữ hai lượt với bảng mã hóa và bản đồ thanh ghi; lõi xử lý RV32IMF với mô hình truy cập bộ nhớ bất đồng bộ và xử lý dịch vụ hệ thống; hệ thống bộ nhớ gồm bộ nhớ chính, MMU/TLB và phân cấp bộ nhớ đệm; hệ thống bus TileLink với lõi giao thức, định tuyến, hai phân hệ UH/UL và cầu nối; bộ điều khiển DMA; bốn thiết bị ngoại vi MMIO; các thành phần trực quan hóa và vòng điều khiển trên giao diện. Trên cơ sở hiện thực này, Chương 5 trình bày quy trình và kết quả kiểm thử, đánh giá hệ thống.

Chương 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Phương pháp và môi trường kiểm thử

Để xác minh hệ thống, đề tài kết hợp ba lớp kiểm thử. Thứ nhất là kiểm thử đơn vị và kiểm thử khối: mỗi script Node.js trong thư mục test/ khởi tạo trực tiếp một mô-đun (assembler, MMU, TileLink, DMA) hoặc một cấu hình tích hợp nhỏ, rồi so sánh kết quả với giá trị kỳ vọng bằng các phép kiểm tra (assertion). Thứ hai là đối chiếu mã hóa lệnh: lấy mã máy do trình biên dịch hợp ngữ của đề tài sinh ra so khớp từng từ lệnh 32 bit với mã máy do GNU RISC-V binutils tạo ra, dùng bộ chương trình kiểm thử chuẩn riscv-tests làm dữ liệu đầu vào [17], [18]. Thứ ba là đối chiếu thực thi: chạy các chương trình riscv-tests trên trình mô phỏng tham chiếu Spike để khẳng định hành vi thực thi đúng [8].

Môi trường kiểm thử gồm Node.js để chạy các script kiểm thử, GNU RISC-V binutils (riscv64-unknown-elf-as, objcopy, objdump) làm công cụ đối chiếu mã hóa, Spike làm trình mô phỏng tham chiếu, và một bản checkout cục bộ của riscv-tests để cung cấp dữ liệu kiểm thử ISA.

5.2. Kiểm thử đơn vị các mô-đun

Mỗi mô-đun chính có một script kiểm thử riêng và đều chạy qua (pass) toàn bộ phép kiểm tra. Script `assembler_verify` kiểm các đường mã hóa dễ sai như cú pháp `jalr`, các lệnh giả `jr/ret/call/li`, mã hóa `fence`, các lệnh dấu phẩy động có `rounding mode` và các directive dữ liệu; kết quả mong đợi được viết dưới dạng từ lệnh hex hoặc byte trong bộ nhớ nên đây là kiểm thử tĩnh định ở mức trình biên dịch. Script `mmu_basic_verify` kiểm logic dịch địa chỉ của MMU: identity fallback khi chưa có ánh xạ, ánh xạ địa chỉ ảo sang địa chỉ vật lý qua bảng trang rồi hit TLB ở lần sau, thay thế LRU trong TLB, lỗi quyền truy cập và đường request/response.

Script `tilelink_verify` kiểm fabric ở cấu hình tích hợp nhỏ gồm cache, memory, TileLink-UH/UL, bridge và DMA: đọc/ghi qua memory, ghi một phần (partial write) đúng byte lane, lệnh nguyên tử, và định tuyến hai chiều qua bridge. Script `dma_verify`

kiểm DMA qua ba kịch bản: gọi trực tiếp API copy trong RAM, để CPU ghi các thanh ghi DMA bằng chương trình hợp ngữ rồi poll bit BUSY cho tới khi xong, và chế độ byte-swap. Cả bốn script trên đều chạy thành công.

5.3. Đối chiếu mã hóa lệnh với GNU binutils

Đây là phép kiểm chứng quan trọng nhất đối với trình biên dịch hợp ngữ. Ý tưởng là so sánh mã máy do đề tài sinh ra với mã máy do một công cụ chuẩn, đã được kiểm chứng rộng rãi, tạo ra cho cùng một chương trình; nếu hai bên trùng khớp ở mọi lệnh thì có cơ sở tin rằng phần mã hóa lệnh của đề tài là đúng. Công cụ tham chiếu được chọn là GNU RISC-V binutils, và dữ liệu đầu vào là bộ riscv-tests — bộ chương trình kiểm thử ISA chính thức của cộng đồng RISC-V [17], [18].

Quy trình thực hiện như sau: với mỗi tập tin ELF đã biên dịch sẵn trong riscv-tests (các nhóm rv32ui, rv32um, rv32uf), dùng objdump tách từng lệnh kèm mã máy gốc; chuẩn hóa các lệnh nhảy tương đối theo PC (như beq, jal) về offset tương ứng tại địa chỉ đang xét; biên dịch lại từng lệnh bằng trình biên dịch của đề tài tại đúng địa chỉ đó; rồi so khớp từ lệnh 32 bit. Các lệnh không thuộc phạm vi RV32IMF của đề tài — chủ yếu là lệnh CSR và lệnh đặc quyền xuất hiện trong đoạn khởi tạo môi trường của riscv-tests — được đánh dấu bỏ qua (skipped) và thống kê riêng, không dùng để che lỗi.

Kết quả: trên 61 tập tin ELF của riscv-tests, đề tài đối chiếu 17.978 từ lệnh thuộc phạm vi RV32IMF và tất cả đều trùng khớp với GNU binutils, không có lệnh nào sai (0 mismatch); 1.456 lệnh CSR/đặc quyền nằm ngoài phạm vi được bỏ qua. Ngoài ra, một bộ chương trình mẫu cục bộ gồm 83 từ lệnh (đại diện cho các nhóm R/I/load-store/điều khiển của RV32I, RV32M và RV32F) cũng trùng khớp hoàn toàn.

Bảng 5.1 minh họa một vài lệnh tiêu biểu thuộc các nhóm RV32I, RV32M và RV32F cùng mã máy 32 bit; với mỗi lệnh, mã máy do đề tài sinh ra trùng khớp với mã máy do GNU binutils tạo ra.

Bảng 5.1. Ví dụ mã hóa lệnh do đề tài sinh ra, đối chiếu với GNU binutils

Lệnh hợp ngữ	Mã máy đề tài sinh	Mã máy GNU binutils
<code>addi x1, x2, -2048</code>	<code>0x80010093</code>	<code>0x80010093</code>
<code>mul x1, x2, x3</code>	<code>0x023100B3</code>	<code>0x023100B3</code>
<code>lw x5, 0(x6)</code>	<code>0x00032283</code>	<code>0x00032283</code>
<code>fadd.s f1, f2, f3</code>	<code>0x003170D3</code>	<code>0x003170D3</code>

5.4. Đối chiếu thực thi với Spike trên riscv-tests

Bên cạnh việc kiểm tra mã hóa tĩnh, đề tài dùng trình mô phỏng tham chiếu Spike để kiểm tra hành vi thực thi [8]. Script `verify_riscv_tests_spike` chạy lần lượt các ELF `rv32ui`, `rv32um`, `rv32uf` của `riscv-tests` trên Spike với cấu hình ISA `RV32IMF_zicclsm`, rồi phân loại `pass/fail` theo cơ chế kết thúc của `riscv-tests`. Phần mở rộng `zicclsm` được bật để hỗ trợ các bài kiểm tra truy cập bộ nhớ không căn hàng (ví dụ `rv32ui-p-ma_data`); nếu chạy với `RV32IMF` thuần, Spike có thể phát sinh trap `misaligned load/store`.

Kết quả: toàn bộ 61/61 chương trình `riscv-tests` thuộc ba nhóm `RV32I`, `RV32M` và `RV32F` đều chạy thành công trên Spike, không có chương trình nào `timeout` hay kết thúc với mã lỗi.

5.5. Nhận xét và hạn chế

Kết quả kiểm thử cho thấy các mô-đun trọng tâm của hệ thống đã có bằng chứng đúng đắn rõ ràng: trình biên dịch hợp ngữ sinh mã trùng khớp với GNU `binutils` trên gần mười tám nghìn lệnh, các chương trình ISA chuẩn chạy đúng trên Spike, và các mô-đun `MMU`, `TileLink`, `DMA` đều qua kiểm thử đơn vị. So với khóa luận trước của Nguyễn Gia Bảo Ngọc [9], đề tài mở rộng phạm vi tập lệnh sang `RV32M` và `RV32F`, đồng thời bổ sung quy trình đối chiếu định lượng với GNU `binutils` và Spike.

Tuy nhiên, phạm vi kiểm thử cũng cho thấy một số giới hạn cần nêu trung thực. Kết quả chỉ khẳng định tính đúng đắn trong phạm vi tập con `RV32IMF` ở chế độ người dùng: các lệnh `CSR` và lệnh đặc quyền nằm ngoài phạm vi nên được bỏ qua khi đối chiếu, và `fence` mới được mã hóa chứ chưa có nhánh thực thi riêng. Đối với `TileLink`, kiểm thử chỉ xác minh mô hình giao dịch `A/D` cùng các opcode

đọc/ghi/nguyên tử đã hiện thực, chưa bao phủ kênh B/C/E hay coherence đầy đủ. Cuối cùng, hệ thống chưa có cơ chế ngắt phần cứng tới CPU nên DMA và các ngoại vi đều hoạt động theo cơ chế polling, và đề tài chưa xây dựng bộ benchmark hiệu năng định lượng có quy trình đo lặp lại — các bộ đếm cache/TLB và số chu kỳ trong trình mô phỏng hiện chỉ phục vụ quan sát.

Chương 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Khóa luận đã xây dựng được trình mô phỏng hệ thống trên chip (SoC) dựa trên kiến trúc tập lệnh RISC-V RV32IMF [1] và giao thức TileLink [2], chạy trực tiếp trên trình duyệt web. Hệ thống tích hợp trình biên dịch hợp ngữ, lõi xử lý, MMU/TLB, bộ nhớ đệm L1I/L1D/L2, bus TileLink-UH/UL, bộ điều khiển DMA, bốn thiết bị ngoại vi (UART, ma trận LED, bàn phím, chuột) và các khung trực quan hóa trạng thái.

So với năm mục tiêu kỹ thuật ở mục 1.2.2, khóa luận đã hoàn thành 5/5 mục tiêu, tương đương khoảng 100% số mục tiêu đề ra trong phạm vi mô phỏng đã xác định:

- Lõi xử lý: hỗ trợ tập con RV32I/M/F, tệp thanh ghi số nguyên và dấu phẩy động, chu trình nạp, giải mã và thực thi lệnh.
- Hệ thống bus: mô phỏng TileLink-UH/UL ở mức giao dịch trên hai kênh A/D, có cầu nối giữa hai phân hệ.
- DMA: vận chuyển khối dữ liệu độc lập với luồng thực thi chính của CPU.
- Quản lý bộ nhớ: hiện thực MMU, TLB và phân cấp bộ nhớ đệm L1I/L1D/L2.
- Biên dịch, ngoại vi và trực quan hóa: tích hợp assembler hai lượt, các ngoại vi tương tác và các khung quan sát hệ thống.

Kết quả kiểm thử cho thấy các mô-đun trọng tâm đều chạy qua kiểm thử đơn vị; phần mã hóa lệnh đối chiếu 17.978 từ lệnh với GNU RISC-V binutils [18] và không có sai khác; 61/61 chương trình thuộc bộ riscv-tests [17] chạy thành công trên Spike [8]. So với khóa luận liên quan trước đó [9], hệ thống đã mở rộng thêm RV32M, RV32F, tệp thanh ghi dấu phẩy động, nhiều ngoại vi hơn và quy trình kiểm chứng định lượng rõ ràng hơn.

Nhìn chung, khóa luận đã đạt mục tiêu tổng quát ở mục 1.2.1: xây dựng một công cụ mô phỏng SoC RISC-V trên nền web phục vụ học tập và giảng dạy kiến trúc máy tính ở mức hệ thống.

6.2. Hạn chế

Hệ thống còn các hạn chế chính sau:

- Chỉ hỗ trợ tập con RV32IMF; chưa hiện thực CSR, chế độ đặc quyền, FCSR/fflags/frm đầy đủ và thực thi fence.
- TileLink mới dừng ở mức giao dịch A/D; chưa có kênh B/C/E và cơ chế nhất quán bộ nhớ đệm TileLink-C [2].
- Điều phối bus vẫn theo hàng đợi một giao dịch, chưa phải bộ phân xử đa master đầy đủ.
- Chưa có cơ chế ngắt phần cứng tới CPU; DMA và ngoại vi chủ yếu dùng đọc thăm dò.
- Chưa có bộ benchmark hiệu năng định lượng lặp lại; số liệu hiện phục vụ quan sát trong mô phỏng.
- Thiết bị truyền thông CAN [19] chưa được hiện thực.

6.3. Hướng phát triển

Các hướng phát triển tiếp theo gồm:

- Bổ sung CSR, chế độ đặc quyền, FCSR, cờ ngoại lệ và chế độ làm tròn động cho nhóm lệnh dấu phẩy động.
- Thực thi fence như một thao tác có hiệu lực trong lõi xử lý.
- Mở rộng TileLink với các kênh B/C/E, cơ chế TileLink-C và bộ phân xử đa master đầy đủ.
- Bổ sung bộ điều khiển ngắt và đường IRQ từ DMA, UART, bàn phím, chuột tới CPU.
- Mở rộng hệ thống ngoại vi, trong đó có CAN [19].
- Xây dựng benchmark hiệu năng định lượng cho bộ nhớ đệm, DMA và tốc độ mô phỏng.
- Hoàn thiện log kiểm chứng chính thức, mở rộng độ phủ kiểm thử và xây dựng thêm bài thực hành phục vụ giảng dạy.

TÀI LIỆU THAM KHẢO

- [1] A. Waterman và K. Asanović (chủ biên), "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Document Version 20191213," RISC-V Foundation, tháng 12/2019. [Trực tuyến]. Có tại: <https://riscv.org/technical/specifications/>
- [2] SiFive, Inc., "SiFive TileLink Specification, Version 1.8.1," 2020. [Trực tuyến]. Có tại: <https://www.sifive.com/documentation/tilelink/>
- [3] B. Nikolic, Z. Radivojevic, J. Djordjevic và V. Milutinovic, "A Survey and Evaluation of Simulators Suitable for Teaching Courses in Computer Architecture and Organization," IEEE Transactions on Education, tập 52, số 4, trang 449–458, tháng 11/2009.
- [4] K. Vollmar và P. Sanderson, "MARS: An education-oriented MIPS assembly language simulator," trong Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education (SIGCSE '06), Houston, TX, Hoa Kỳ, 2006, trang 239–243.
- [5] "RISC-V Interpreter," Đại học Cornell, môn học CS 3410. [Trực tuyến]. Có tại: <https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/> (truy cập: 19/01/2026).
- [6] R. Giorgi và G. Mariotti, "WebRISC-V: A Web-Based Education-Oriented RISC-V Pipeline Simulation Environment," trong Proceedings of the Workshop on Computer Architecture Education (WCAE '19), New York, NY, Hoa Kỳ, 2019, trang 1–6.
- [7] M. B. Petersen, "Ripes: A Visual Computer Architecture Simulator," trong Proceedings of the 2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE), 2021, trang 1–8.
- [8] "Spike RISC-V ISA Simulator (riscv-software-src/riscv-isa-sim)," RISC-V Software. [Trực tuyến]. Có tại: <https://github.com/riscv-software-src/riscv-isa-sim> (truy cập: 19/01/2026).

- [9] N. G. B. Ngọc, "Phát triển trình mô phỏng SoC," Khóa luận tốt nghiệp, Trường Đại học Công nghệ Thông tin, Đại học Quốc gia TP. Hồ Chí Minh, 2025.
- [10] "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019 (bản sửa đổi của IEEE Std 754-2008), trang 1–84, tháng 7/2019.
- [11] D. A. Patterson và J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition, tái bản lần 2. Cambridge, MA, Hoa Kỳ: Morgan Kaufmann, 2020.
- [12] J. L. Hennessy và D. A. Patterson, Computer Architecture: A Quantitative Approach, tái bản lần 6. Cambridge, MA, Hoa Kỳ: Morgan Kaufmann, 2019.
- [13] Mozilla, "MDN Web Docs." [Trực tuyến]. Có tại: <https://developer.mozilla.org/> (truy cập: 19/01/2026).
- [14] M. Haverbeke, "CodeMirror — thư viện soạn thảo mã trên nền web." [Trực tuyến]. Có tại: <https://codemirror.net/> (truy cập: 19/01/2026).
- [15] Google, "Material Components for the Web (MDC Web)." [Trực tuyến]. Có tại: <https://github.com/material-components/material-components-web> (truy cập: 19/01/2026).
- [16] "RARS — RISC-V Assembler and Runtime Simulator (TheThirdOne/rars)," kho mã nguồn GitHub. [Trực tuyến]. Có tại: <https://github.com/TheThirdOne/rars> (truy cập: 19/01/2026).
- [17] "riscv-software-src/riscv-tests," RISC-V Software, kho mã nguồn GitHub. [Trực tuyến]. Có tại: <https://github.com/riscv-software-src/riscv-tests> (truy cập: 19/01/2026).
- [18] "GNU Binutils," GNU Project, và "riscv-gnu-toolchain," RISC-V Collaboration. [Trực tuyến]. Có tại: <https://www.gnu.org/software/binutils/> và <https://github.com/riscv-collab/riscv-gnu-toolchain> (truy cập: 19/01/2026).

- [19] Robert Bosch GmbH, "CAN Specification, Version 2.0," Stuttgart, Đức, 1991; và ISO 11898, "Road vehicles — Controller area network (CAN)," Tổ chức Tiêu chuẩn hóa Quốc tế (ISO).